



BSDArchitect™ Process Guide

Software Version 2015.4

December 2015

**© 1999-2015 Mentor Graphics Corporation
All rights reserved.**

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

U.S. GOVERNMENT LICENSE RIGHTS: The software and documentation were developed entirely at private expense and are commercial computer software and commercial computer software documentation within the meaning of the applicable acquisition regulations. Accordingly, pursuant to FAR 48 CFR 12.212 and DFARS 48 CFR 227.7202, use, duplication and disclosure by or for the U.S. Government or a U.S. Government subcontractor is subject solely to the terms and conditions set forth in the license agreement provided with the software, except for provisions which are contrary to applicable mandatory federal laws.

TRADEMARKS: The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the owner of the Mark, as applicable. The use herein of a third-party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at: www.mentor.com/trademarks.

The registered trademark Linux[®] is used pursuant to a sublicense from LMI, the exclusive licensee of Linus Torvalds, owner of the mark on a world-wide basis.

Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: www.mentor.com
SupportNet: supportnet.mentor.com/

Send Feedback on Documentation: supportnet.mentor.com/doc_feedback_form

Table of Contents

Chapter 1

Overview	11
Understanding Boundary Scan	11
Benefits of Boundary Scan	11
BSDArchitect Boundary Scan Overview	11
Boundary Scan Insertion With BSDArchitect	18
Top-Down Design Flows Using BSDArchitect	19
BSDArchitect Design Flows	19
BSDArchitect User Interface	20
Running Batch Mode Using Dofiles	20
Generating a Log File	21
Running UNIX Commands	21
Interrupting the Session	22
Exiting the Session	22
Resetting the State of BSDArchitect	22

Chapter 2

External Boundary Scan Synthesis	23
BSDArchitect External Flow	23
BSDArchitect Output Model	24
BSDArchitect Output Example	26
Design Issues	30
Logic Type of Entity Ports	30
Escaped Identifiers for Verilog	30
Handling Tri-State and Bidirectional Ports	30
Limitations	36
Recommended Practices	38
Preparing for Boundary Scan Insertion	38
Boundary Scan Example Design	38
Creating the HDL Description	39
Creating the Package Mapping File	39
Invoking BSDArchitect	39
Setting Up the Boundary Scan Specification	39
Boundary Scan Insertion Using BSDArchitect	40
The Example Design	40
Running BSDArchitect in an External Flow	40

Chapter 3

Internal Boundary Scan Synthesis	43
Understanding BSDArchitect Internal Flow	43
Inputs	43
Outputs	44

Building I/O Pad Definitions	44
Invoking BSDArchitect	44
Example on Boundary Scan Insertion.	45
Reporting Circuit Components	46
Adding I/O Pads to Ports Without Pads.	47
Reporting Pad Cells	48
Adding Boundary Scan Cells.	49
Boundary Scan Cell Placement.	49
Adding Boundary Scan Cells to Control Ports	51
Boundary Scan Cell Placement for Control Ports.	52
Loading Pin Maps	55
Making Top-Level Port Connections.	56
Placing and Connecting External Registers.	57
Sharing Functional Pins With TDO.	58
Uniquification	59
Unsupported Commands	60
 Chapter 4	
Boundary Scan Customizations.	61
Creating Customizations	61
Technology Mapping.	62
Libraries	62
Technology Mapping Syntax	63
Design Rules Checking for Technology Mapping	65
Defining Boundary Scan Cells	66
Defining I/O Pads	99
Using a Pin Mapping File	111
Pin Map File Contents	112
Pin Map File Syntax	113
Pin Mapping Example.	116
Using User-Defined Instructions	117
Connecting Internal Scan Circuitry	118
Adding Dummy Scan Pins	119
Specifying the Scan Architecture and Testing Mode	120
Specifying the Internal Scan Chain.	123
Defining the Scan Instruction	123
Combining Multiple Chains	124
 Chapter 5	
Output Testing and Verification	125
Understanding BSDArchitect Output Files	125
HDL Description.	125
Top-Level Interface Description.	126
HDL Test Bench.	126
BSDL Description.	127
Synthesis Script.	127
Parametric Tests	128
Test Vectors	128

Table of Contents

ATPG Setup Files	128
Using BSDArchitect in a BSDL Flow	129
Inserting Boundary Scan Circuitry	130
Defining a Test Setup Procedure	131
Example Test Setup for STIL Test Vectors	133
Creating Output Files	135
Verifying the HDL Boundary Scan Circuitry	136
Verifying the Gate-Level Boundary Scan Logic	138
Initializing TAP States for Gate-Level Simulations	140
AC Port Verification	140
AC Test Receiver Verification	140
AC Selection Cell Verification	143
Troubleshooting the Test Bench Simulation	144
Boundary Scan Test Architecture	144
JTAG Logic Tests	145
Parametric Testing	161
 Chapter 6	
Existing Boundary Scan Register Integration	167
Working with Existing Blocks	169
I/O's Mapped to the Top Level of the Device	170
I/O's Not Mapped to the Top Level of the Device	171
How the Commands Relate to Existing Blocks	172
Specifying the Routing Order With Existing Block BSRs	173
 Appendix A	
Getting Help	175
Documentation	175
Mentor Graphics Support	176
 Appendix B	
Cell Type Assignments for Bidirectional Ports	177
Background	177
Understanding Cell Type Assignments	177
Cell Type Correspondence	178
 Appendix C	
Setting Up the 1149.6 Hardware	179
AC Boundary Scan Cells Support	179
AC Pad Cells	179
Support for EXTEST_PULSE and EXTEST_TRAIN Instructions	180
Setting Up the AC Boundary Scan Ports	180
Adding AC Boundary Scan Cells	180
Specifying AC/DC Selection Cells	181
Specifying Observe Only Cells	182
Summary of Modifications to Report Commands	182

Index

Third-Party Information

End-User License Agreement

List of Examples

Example 3-1. Internal Boundary Scan Input	46
Example 3-2. Internal Boundary Scan Output.	46
Example 3-3. Boundary Scan Cell Placement on Single Data Port.	50
Example 3-4. Boundary Scan Cell Placement on Multiple Data Ports	51
Example 3-5. No Top-Level Signal for a Tri-State Control Port	52
Example 3-6. Single Control Signal Without Inverter/Buffer.	53
Example 3-7. Single Control Signal With Inverter	53
Example 3-8. Multiple Control Signals Without Inverter/Buffer	54
Example 3-9. Multiple Control Signals With Inverter.	55
Example 3-10. Design Core Example	57
Example 3-11. Resulting Design Core.	58
Example 3-12. Pin Sharing Example.	59

List of Figures

Figure 1-1. Boundary Scan Chips on Board	12
Figure 1-2. Boundary Scan Architecture	13
Figure 1-3. State Diagram of TAP Controller Circuitry	16
Figure 2-1. BSDArchitect External Design Flow	23
Figure 2-2. Boundary Scan Output Model.	25
Figure 2-3. BSDArchitect Output Files.	28
Figure 2-4. “my_mod_top.v” Output.	28
Figure 2-5. “my_mod_bscan_unbuff.v” Output	29
Figure 2-6. “my_mod_bscan.v” Output.	30
Figure 2-7. Handling of Enable Signals Not Used in Core	34
Figure 2-8. Handling of Enable Signals Used in Core.	35
Figure 2-9. Accessing the Enable	36
Figure 3-1. BSDArchitect Internal Flow Inputs and Outputs	43
Figure 3-2. BSDArchitect Tool Flow	45
Figure 3-3. Top-Level Port Connections	57
Figure 4-1. Generic BC_1 Cell	67
Figure 4-2. Generic BC_2 Cell	69
Figure 4-3. Generic BC_2_A Cell	70
Figure 4-4. Generic BC_2_A_EXT Cell	72
Figure 4-5. Generic BC_2_B Cell	73
Figure 4-6. Generic BC_3 Cell	74
Figure 4-7. Generic BC_4 Cell	75
Figure 4-8. Generic BC_5 Cell	76
Figure 4-9. Generic BC_7 Cell	78
Figure 4-10. Generic BC_7_LOW Cell.	80
Figure 4-11. Generic BC_8 Cell	82
Figure 4-12. Generic BC_9 Cell	83
Figure 4-13. Generic BC_10 Cell	85
Figure 4-14. BC_X Cell	86
Figure 4-15. Generic AC_1 Cell	87
Figure 4-16. Generic AC_2 Cell	88
Figure 4-17. Generic AC_7 Cell	90
Figure 4-18. Generic AC_7_LOW Cell.	92
Figure 4-19. Generic AC_8 Cell	94
Figure 4-20. Generic AC_9 Cell	95
Figure 4-21. Generic AC_10 Cell	97
Figure 4-22. Generic AC_SelX Cell	98
Figure 4-23. Generic AC_SelU Cell	99
Figure 4-24. Basic Input Pad	100
Figure 4-25. Basic Output Pad.	101
Figure 4-26. Basic Tri-State Pad	102

List of Figures

Figure 4-27. Basic Bidirectional Pad	103
Figure 4-28. H-Type Open Collector Output Pad	104
Figure 4-29. L-Type Open Collector Output Pad	105
Figure 4-30. Generic Differential Input and Output Pads	106
Figure 4-31. AC Input Pad	108
Figure 4-32. AC Differential Input Pad	108
Figure 4-33. AC Bidi Pad	109
Figure 4-34. AC Bidi Differential Pad	110
Figure 4-35. Clocking Circuitry Created for Mux-DFF Architecture	120
Figure 4-36. Clocking Circuitry Created for Clocked Scan Architecture	121
Figure 4-37. Default Architecture for Testing Mode	122
Figure 4-38. Internal Scan Instruction Connections	123
Figure 4-39. Connection of Multiple Scan Chains	124
Figure 5-1. Connections Between an AC Input Pad with Test Receiver, the Corresponding BC_4 Boundary Scan Cell, and the TAP Controller	143
Figure 5-2. Test Architecture	145
Figure 6-1. Internal Flow Connections of the Existing Block's BSR Segment With the Design's New Boundary Scan Cells	168
Figure 6-2. External Flow Connections of the Existing Block's BSR Segment with the Design's New Boundary Scan Cells	169
Figure 6-3. Example of Routing Order With Multiple Existing Blocks in the External Flow	174

List of Tables

Table 3-1. Control Port Notation 51

Table 4-1. Library Attributes vs. Command Line Arguments 63

Table 4-2. H-Type Open Collector Truth Table 104

Table 4-3. L-Type Open Collector Truth Table 105

Table 4-4. Differential Pad Truth Tables 106

Table B-1. BC_2_* to BC_7* Cell Type Correspondence 178

Understanding Boundary Scan

Boundary scan, sometimes referred to as JTAG (for Joint Test Action Group, the committee that formulated the IEEE standard 1149.1 that describes boundary scan), is a DFT technique that facilitates the testing of printed circuit board (and MCM) interconnect circuitry and, to a limited extent, the chips on those boards. Boundary scan test structures greatly improve board-level testing, thus shortening the manufacturing test and diagnostics processes.

The most recent revision to the standard occurred in 2001 and is known as IEEE 1149.1-2001.

Benefits of Boundary Scan

As the usefulness of in-circuit test diminishes owing to the increasing popularity of surface mount devices, boundary scan provides the same benefits, but without requiring physical access to each electrical network. Adding boundary scan logic to your board lets you detect the vast majority of board manufacturing process faults using boundary scan test methods. These faults include wrong components, missing components, incorrectly oriented components, components with stuck pins, shorts, opens, and blown wire bonds.

Although your engineering costs may increase slightly because of the additional silicon and ports used for the boundary scan circuitry, implementing the IEEE 1149.1 standard can dramatically reduce a design's manufacturing costs.

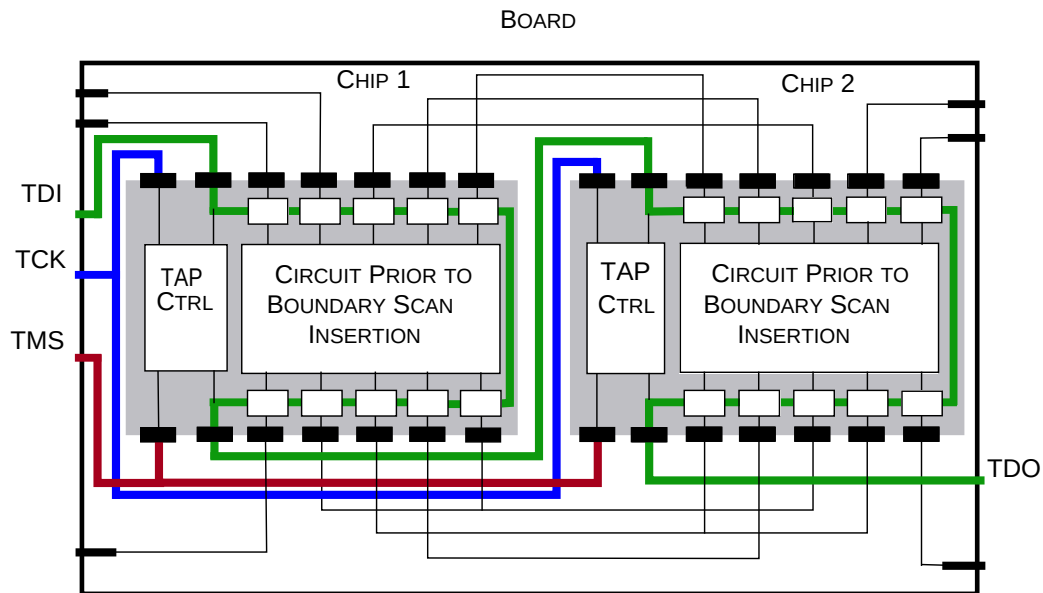
BSDArchitect Boundary Scan Overview

When used on a board, boundary scan provides access to the input and output ports of the chips by linking them together into a long scan path. Data shifts along the scan path, starting at the edge-connector input TDI (test data in) and ending at the edge-connector output TDO (test data out). In between, the scan path connects all the devices on the board that contain boundary scan circuitry. The TDO of one chip feeds the TDI of the next, all the way around the board. The inputs, TCK (test clock) and TMS (test mode select), connect in parallel to each boundary scan device in the scan path. With this configuration you can test board interconnections, perform a snapshot of normal system data, or test individual chips. The TAP (test access port) controller is a state machine that controls the operation of the boundary scan circuitry.

Boundary scan circuitry's primary use is in board-level testing, but it can also control circuit-level test structures, such as BIST or internal scan. By adding boundary scan circuitry to your design, you create a standard interface for accessing and testing chips at the board level.

Figure 1-1 shows a board containing two chips with boundary scan circuitry.

Figure 1-1. Boundary Scan Chips on Board



Simple Boundary Scan Architecture

This section discusses boundary scan circuitry in detail.

Figure 1-2. Boundary Scan Architecture

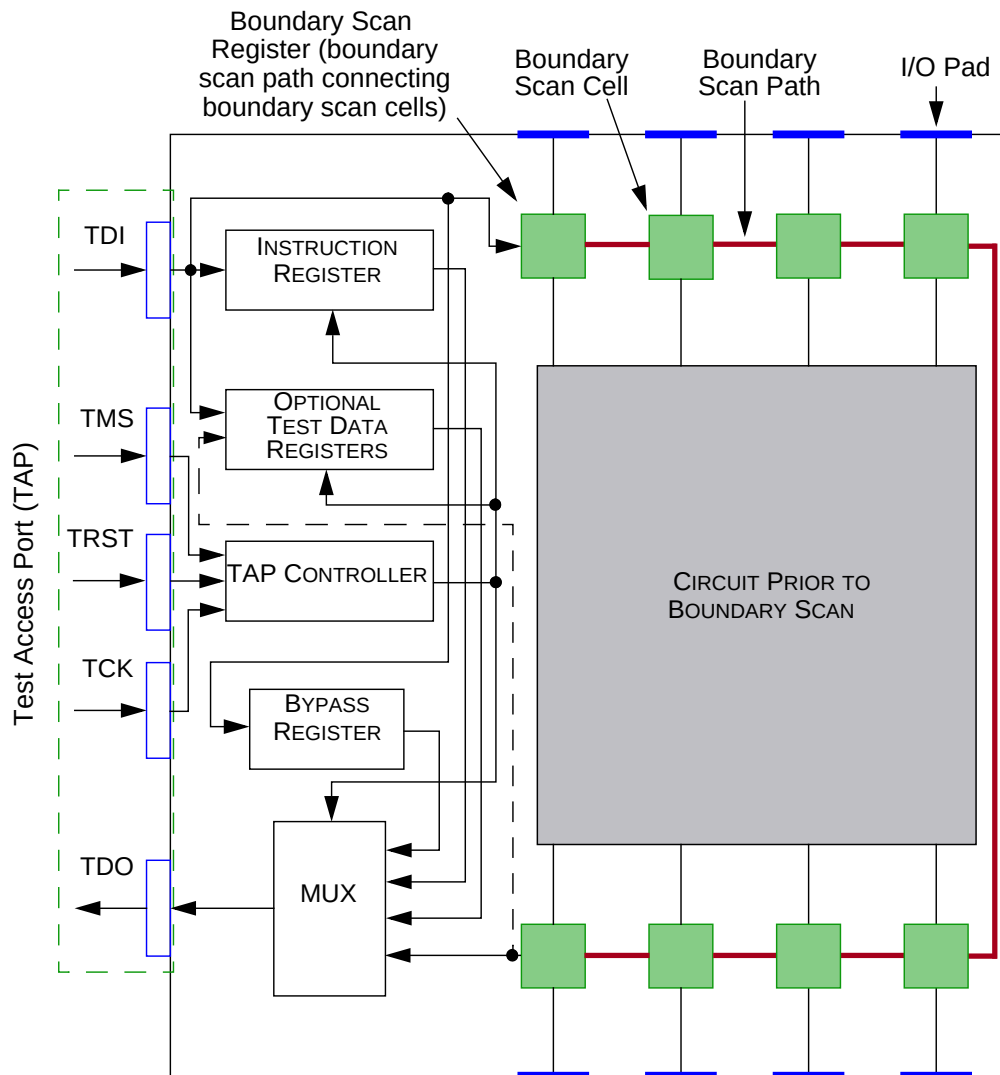


Figure 1-2 shows the general configuration of a chip after the addition of boundary scan logic. This simple boundary scan architecture contains the following components:

- **Circuit Prior to Boundary Scan** — This is the application logic of the original design before adding boundary scan logic. This logic might already contain internal scan circuitry (or at least internal scan ports so boundary scan circuitry can connect to the internal scan circuitry).
- **Boundary Scan Cells** — These contain memory elements for capturing data from the circuit, loading data into the circuit, or serially shifting data to the next scan cell in the path. The tool places boundary scan cells between the core logic and each input,

bidirectional, and 2- or 3-state output pin. Boundary scan cells collectively comprise a parallel-in, parallel-out shift register that runs along the periphery, or boundary, of the original design.

- **Test Access Port (TAP)** — The TAP consists of a minimum of four pins for the four signals that make up the test bus. These signals include the test clock (TCK), the test data input (TDI), the test data output (TDO), and the test mode selector (TMS). Also shown is an optional asynchronous test reset (TRST).
- **TAP Controller** — The TAP controller is a finite state machine that controls the operation of the instruction and test data registers. The TAP controller's state depends on the value of the TMS line at each clock pulse.
- **Boundary Scan Register** — Considered the main test data register, the boundary scan register is a virtual shift register (consisting of the connection of the individual boundary scan cells) that can, either serially or in parallel, load and unload input and output data for the circuit.
- **Bypass Register** — The bypass register shortens the serial path between TDI and TDO to one cell when there is no requirement to test a particular device. This shortened path in effect bypasses the chip, allowing more efficient data shifting to other ICs in the chain.
- **Optional Test Data Registers:**
 - **Device Identification Register** — This register contains a device identification code or programming code used to check that the board has proper chips.
 - **Data-Specific Registers** — These registers allow access to the chip's test support features, such as BIST and internal scan paths.
- **Instruction Register** — The instruction register controls the boundary scan circuitry by connecting a specific test data register between the TDI and TDO pins. It controls the operation affecting the data in that register, using a predefined set of instructions. Some instructions are mandatory, and others are optional.

The mandatory instructions include:

- **EXTEST** — This instruction tests circuitry external to the ICs themselves, such as board interconnect. EXTEST is the main test instruction for boundary scan testing.
- **SAMPLE/PRELOAD** — This instruction takes data from the chips's I/O pads and latches it in the boundary scan register (during normal board operation). The IEEE Std. 1149.1-2001 has redefined this instruction as two separate instructions (SAMPLE and PRELOAD).
- **SAMPLE** — This instruction takes data from the chip's I/O pads and latches it in the boundary scan register (during normal board operation).
- **PRELOAD** — This instruction allows the loading of data into the boundary scan register before selecting another instruction.

- **BYPASS** — This instruction enables bypassing of chips not being tested. For example, if you only want to test one chip in a board with 20 chips in the boundary scan chain, all chips except the chip under test would be bypassed. Consequently, shifted data needs to go through only one extra shift to pass through each of the chips not being tested (as opposed to the entire boundary scan register).
- **EXTEST_PULSE** — This instruction tests circuitry external to the ICs themselves, for designs that have AC pins. This instruction is mandatory only when using AC cell types.

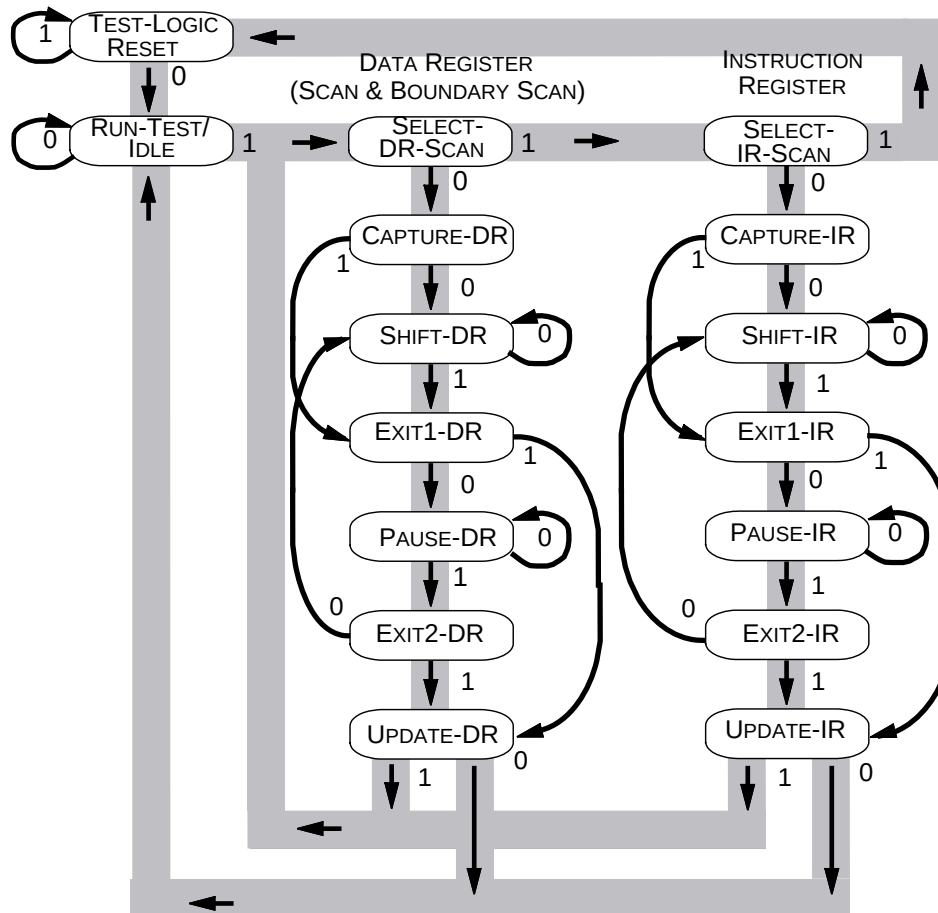
The optional instructions include:

- **INTTEST** — This instruction tests a chip's internal circuitry by applying a test vector to, and capturing the output response from, the application logic.
- **IDCODE** — This instruction connects the device identification register between TDI and TDO. The device identification register contains the device ID code, which is normally used to determine if the chip belongs on the board.
- **USERCODE** — This instruction also selects the identification register, but the information placed in that register is now user-defined and is meant to expand on the IDCODE information.
- **CLAMP** — This instruction forces static 1s or 0s on selected nodes in order to create a testable situation or block interfering signals.
- **HIGHZ** — This instruction forces an IC's output and bidirectional pins into a high-impedance state. In this condition, an in-circuit tester can test it safely without the potential for overdrive damage.
- **RUNBIST** — This instruction executed the circuit's internal BIST procedure.
- **EXTEST_TRAIN** — This instruction, which is available when using AC cell types, operates similarly to EXTEST_PULSE but can use multiple TCK cycles. For more detail, refer to IEEE 1149.6, available through the IEEE.

TAP Controller State Machine

Figure 1-3 shows the finite state machine for the TAP controller of a IEEE 1149.1 circuit.

Figure 1-3. State Diagram of TAP Controller Circuitry



The TMS signal (the value shown adjacent to each state transition) controls the state transitions. The rising edge of the TCK clock captures the TAP controller inputs. Therefore, to avoid race conditions, the signals whose states require the positive edge of TCK are retimed to propagate out of the TAP controller at the negative edge of TCK.

The following example pseudocode of the TAP controller state machine illustrates when the controller needs to retime the signals to avoid the possibility of race conditions:

```
always @ (negedge trstl or negedge tck)
if (trstl == 1'b0)
begin
    resetl <= 1'b0;
    enable <= 1'b0;
    capturedr <= 1'b0;
    shiftldr <= 1'b0;
    shiftlir <= 1'b0;
end
else
```

```
begin
    resetl <= reset_d;
    enable <= enable_d;
    capturedr <= capturedr_d;
    shiftdr <= shiftdr_d;
    shiftir <= shiftir_d;
end
```

Signals in which the event happens at the positive edge of TCK (where the TAP controller needs to retime the signals to avoid race conditions) are as follows:

- **resetl** — The Test-Logic-Reset state and the resetl signal is retimed to occur at the negative edge of TCK.
- **enable** — This signal is the control signal of the TDO tristate buffer. The enable signal is retimed to occur at the negative edge of TCK since the TDO signal needs to occur at the falling edge of TCK.
- **capturedr** (Capture-DR state)
- **shiftdr** (Shift-DR state)
- **shiftir** (Shift-IR state)

The states in which the event happens at the negative edge of TCK (where the TAP controller does not need to retime the signals) are as follows:

- **captureir** (Capture-IR state)
- **updatedr** (Update-DR state)
- **updateir** (Update-IR state)

The following signals are derived from tap states by default, which the tool generates in the RTL and also the logic of these signals. The signals are as follows:

```
clockir = ((pstate == capture_IR) || (pstate == shift_IR)) ? 1'b1 : 1'b0;
clockdr = ((pstate == capture_DR) || (pstate == shift_DR)) ? 1'b1 : 1'b0;
```

And if clock gating is ON, the signals are as follows:

```
clockir = ((pstate == capture_IR) || (pstate == shift_IR)) ? tck : 1'b1;
clockdr = ((pstate == capture_DR) || (pstate == shift_DR)) ? tck : 1'b1;
```

This section is only an overview of boundary scan architecture. For a more comprehensive description of the boundary scan standard, refer to IEEE 1149.1-2001, available through the IEEE.

Boundary Scan Insertion With BSDArchitect

BSDArchitect™ is the Mentor Graphics boundary scan insertion tool. The tool creates and interconnects RTL-level boundary scan logic compliant with the IEEE 1149.1-2001 standard.

Features of the BSDArchitect tool:

- **Instruction support** — Full support of required IEEE standards 1149.1 instructions.
- **Extension support** — Support of base extensions to IEEE 1149.1, such as the Device ID register.
- **Compliant VHDL** — Generation of VHDL ('93 standard) output that is compliant with ModelSim® (VHDL), Synopsys' Design Compiler, and other industry synthesis tools.
- **Compliant Verilog** — Generation of Verilog (IEEE 1364-2001) that is compliant with the ModelSim (Verilog), Synopsys' Design Compiler, and other industry synthesis tools.
- **RTL-level boundary scan generation** — Insertion and interconnection of boundary scan circuitry at the RTL level, moving generation of test circuitry to earlier in the design process.
- **Customized boundary scan** — Generation of default or user-customized boundary scan architectures.
- **I/O pad synthesis** — Generation of generic I/O pads or technology-specific I/O pads.
- **Automatic connection** — Automatic connection of boundary scan to internal scan logic.
- **Test bench generation** — Generation of a test bench, which allows testing of the boundary scan logic after interconnection with the core application logic.
- **Test vector generation** — Generation of boundary scan test vectors in WGL.
- **Setup file generation** — Generation of ATPG setup files, for designs with generated boundary scan circuitry controlling internal scan circuitry.
- **Compliant BSDL** — Production of BSDL output that is compliant with IEEE standard IEEE 1149.1-2001 specification.
- **Generic element mapping** — Mapping of boundary scan elements to generic boundary scan library cells (which enhances re-targetability of the boundary scan circuitry).
- **Technology-specific element mapping** — Mapping of boundary scan, I/O pad, and TAP controller elements to technology-specific library cells. Technology mapping provides support for replacing generic boundary scan cells, I/O pads, and the TAP controller by equivalent technology-specific cells.

For information on using BSDArchitect in your design flow, refer to [BSDArchitect Design Flows](#). For detailed information on the BSDArchitect command set, refer to the [BSDArchitect Reference Manual](#).

Top-Down Design Flows Using BSDArchitect

For information on using BSDArchitect in an ATPG process flow, refer to “[Top-Down Design Flow with DFT](#)” in the *Scan and ATPG User’s Manual*.

For information on using BSDArchitect in a BIST process flow, refer to the [MBISTArchitect Process Guide](#) and the [LBISTArchitect Process Guide](#).

In the overall design process, the generation of boundary scan logic typically occurs after the insertion of internal scan chains and before test pattern generation. However, because BSDArchitect works at the RTL level, it allows you (but does not require you) to perform boundary scan insertion earlier in the design process.

Note



It is important for you to check with your vendor early on in your design process for specific requirements and restrictions that may affect your DFT strategies. For example, the vendor's test equipment may only be able to handle single scan chains, have memory limitations, or have special timing requirements that affect the way you generate scan circuitry and test patterns.

BSDArchitect Design Flows

There are three design flows for BSDArchitect:

- External Flow

For the External flow, the boundary scan logic is added to the design externally. BSDArchitect creates a new top-level module adding another level of hierarchy to the design (sometimes called the *wrapper*). An instance of the original design core is placed in the new module along with the TAP controller instance. The I/O pad cells and boundary scan cells are placed between the original design core and the new top-level ports. For more information, refer to “[External Boundary Scan Synthesis](#)” on page 23.

- Internal Flow

For the Internal flow, the boundary scan logic is added to the design internally. The TAP controller instance is placed within the original top-level of the design and the boundary scan cells are inserted between the I/O pads and the core logic. For more information, refer to “[Internal Boundary Scan Synthesis](#)” on page 43.

Note



Upon invocation, BSDArchitect looks for existing pads in the design and (if specified) in the library file. If any pad is found, BSDArchitect uses the Internal flow. If no pads are found, BSDArchitect uses the External flow. Once BSDArchitect is running, you can change the tool flow from External to Internal and vice versa with the [Set Bscan Flow](#) command, if necessary.

- BSDL Flow

For the BSDL flow, the boundary scan logic is already present in the design. The BSDArchitect tool checks the logic for compliance to the IEEE 1149.1-2001 standard and generates patterns and the test bench. For more information, refer to “[Using BSDArchitect in a BSDL Flow](#)” on page 129.

BSDArchitect User Interface

You invoke BSDArchitect from the command line using the [bsdarchitect](#) invocation command.

Commands follow the 3-2-1 minimum typing convention. As a shortcut (in most cases) you need only type the first three characters of the first command word, the first two characters of the second command word, and the first character of the third command word. In cases where the 3-2-1 rule leads to ambiguity between commands, such as with Set Bscan Capture and Set Bscan Controller (both reducing to “set bs c”), you need to specify the additional characters to alleviate the ambiguity. Here, the minimum typing for the Set Bscan Capture command becomes “set bs ca,” and for the Set Bscan Controller command it becomes “set bs co.”

You also should be aware that when you issue commands with very long argument lists, you can use the “\” line continuation character. For example, you can specify the Add Technology Cell command within a dofile (or at the system mode prompt) as follows:

```
add technology cell vendor_pad_bidi bidirectional \  
-inputs PI E D SLOWDRV \  
-outputs PONF PO \  
-inouts IO \  
-connect PO/din D/dout E/oe IO/pin PI/local_pi PONF/open SLOWDRV/pwr \  
-nonexistent TH
```

For more information on dofile scripts, see the “[Running Batch Mode Using Dofiles](#)” section.

Running Batch Mode Using Dofiles

You can run the tool in batch mode by using a *dofile* to pipe commands into the application. Dofiles let you automatically control the operations of the tool. The dofile is a text file that you create that contains a list of application commands that you want to run, but without entering them individually. If you have a large number of commands, or a common set of commands that

you use frequently, you can save time by placing these commands in a dofile. You can specify a dofile at invocation by using the `-Dofile` switch or by typing `Dofile` at the command line.

If you place all commands, including the `Exit` command, in a dofile, you can run the entire session as a batch process. Once you generate a dofile, you can run it at invocation. For example, to run the tool as a batch process using the commands contained in *my_dofile.do*, enter:

```
shell> <Tessent_Tree_Path>/bin/bsdarchitect example_ckt -top top_design \  
-dofile my_dofile.do
```

The following shows an example BSDArchitect dofile:

```
add tristate port -in IO_IN -out IO_OUT -ctrl IO_CTRL \  
-Active high  
add port mapping IO_OUT_t IO  
add tristate port -out PAD -ctrl PAD_EN -Active high  
load pin map pin_map  
run  
report bscan cell  
save bscan -unbuffered -replace  
exit
```

By default, if the tool encounters an error when running one of the commands in the dofile, it stops dofile execution; however, you can turn this setting off by using the [Set Dofile Abort](#) command.

Generating a Log File

Log files provide a useful way to examine the operation of the tool, especially when you run the tool in batch mode using a dofile. If errors occur, you can examine the log file to see exactly what happened. The log file contains all Tessent® application operations and any notes, warning, or error messages that occur during the session.

You can generate log files by using the `-Logfile` switch when you invoke the tool.

```
<Tessent_Tree_Path>/bin/bsdarchitect example_ckt -top top_design -verilog -logfile log1
```

Running UNIX Commands

You can run UNIX operating system commands within Tessent applications by using the `System` command. For example, the following command executes the UNIX operating system command `ls` within a Tessent application session:

```
BSDA> system ls
```

Interrupting the Session

To interrupt the invocation of a Tessent product and return to the operating system, enter Control-C. You can also use the Control-C key sequence to interrupt the current operation and return control to the tool.

Exiting the Session

To exit a Tessent application and return to the operating system, you can execute the **File > Exit** menu item, click on the Exit button in the Control Panel, or enter Exit at the command line:

```
BSDA> exit
```

If you have generated circuitry that you have not saved, the tool will prompt you as to whether or not to save the information.

Resetting the State of BSDArchitect

At times, you may find it necessary to disregard all the commands you have entered and start over from the beginning. The Reset State command lets you do this. The effect of this command is to reset all the command arguments and values to their default values, which is equivalent to exiting the tool and invoking the tool again on the same design. The tool allows only one synthesis run per session—unless you issue the Reset State command to reset the tool back to its system defaults.

Chapter 2

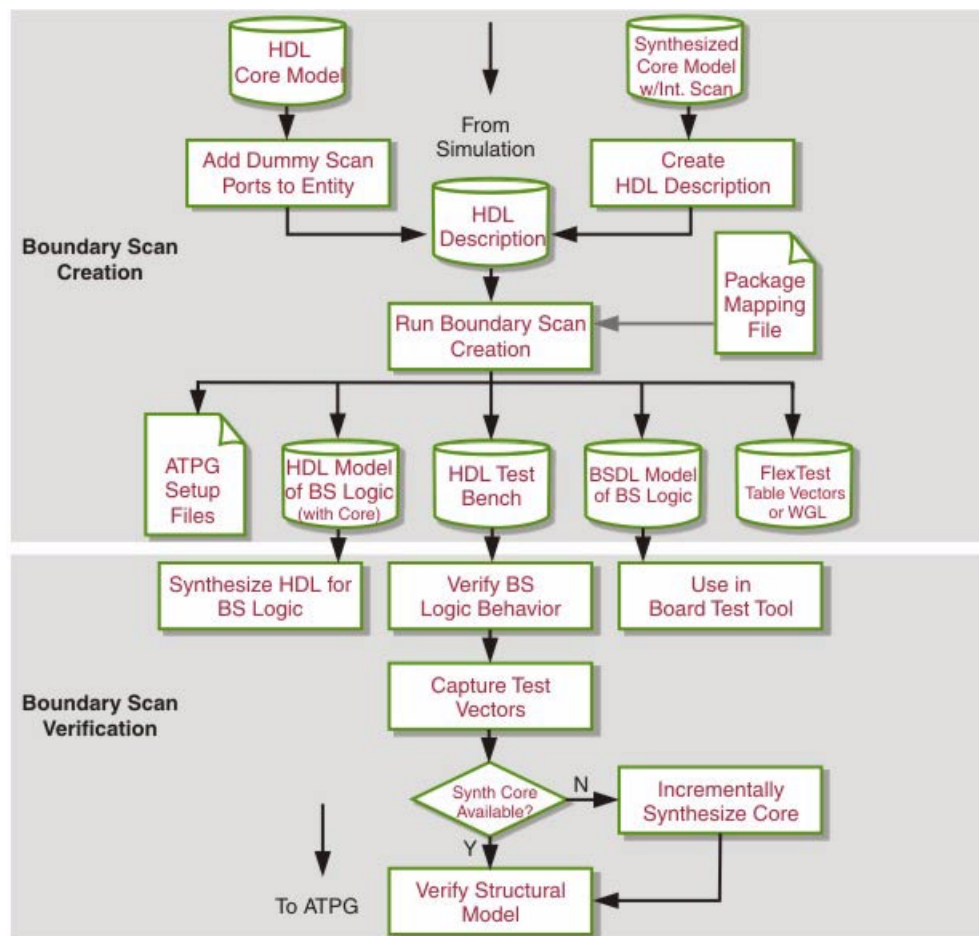
External Boundary Scan Synthesis

This chapter outlines how to use BSDArchitect tool to synthesize and verify boundary scan circuitry for your design using the External flow.

BSDArchitect External Flow

Figure 2-1 shows the External design flow for BSDArchitect.

Figure 2-1. BSDArchitect External Design Flow



The External design flow begins with an HDL model, normally a *Verilog module*. If you plan to insert internal scan circuitry into your design, you need to modify the entity or module description to include the scan ports that will eventually be part of the design. If your design is

already synthesized and contains scan circuitry, you only need to create an HDL entity of the top-level of your design to use as input to BSDArchitect.

If you are using custom library packages for a VHDL design, you need to supply a package mapping file (*dft.map*) in your working directory that contains the packages/libraries used in the core entity description.

You can now run BSDArchitect to create and insert boundary scan circuitry into your design.

BSDArchitect can create a number of different output types:

- A buffered or unbuffered HDL description with boundary scan inserted *<prefix>_bscan.<suffix>*
- A description of the inputs and outputs for the top-level boundary scan core entity/module *<prefix>_top.<suffix>*
- A buffered HDL test bench *<prefix>_tb.<suffix>*
- A BSDL description of the boundary scan circuitry which you can use with board testing tools *<prefix>_bscan.bsd*
- ATPG setup files *<prefix>.testproc* and *<prefix>.dofile*
- WGL format test vectors

You use the HDL test bench to verify that the boundary scan logic generated is correct. The test bench contains vectors used as stimulus for the boundary scan circuitry. You can choose to have BSDArchitect write these vectors into a separate file in WGL format.

Once you verify that the boundary scan circuitry is compliant, you can synthesize the Verilog model of the boundary scan circuitry to a gate-level model. If the circuit contains internal scan circuitry controlled by the boundary scan circuitry, you can use BSDArchitect to generate two ATPG setup files: a dofile, for setting up scan circuitry information, and a test procedure file, for specifying the operation of the scan circuitry. The “[Test Procedure Files](#)” section in the *Scan and ATPG User’s Manual* discusses test procedure files in more detail. You can re-use the HDL test bench to verify correct operation and IEEE compliance of the synthesized design.

The second half of this section discusses how to use these outputs for boundary scan verification further along in the design flow.

BSDArchitect Output Model

BSDArchitect produces a boundary scan design model in Verilog. This model contains a new level of hierarchy (sometimes called the *wrapper*), instantiating the boundary scan circuitry and connecting it to the core design.

Note



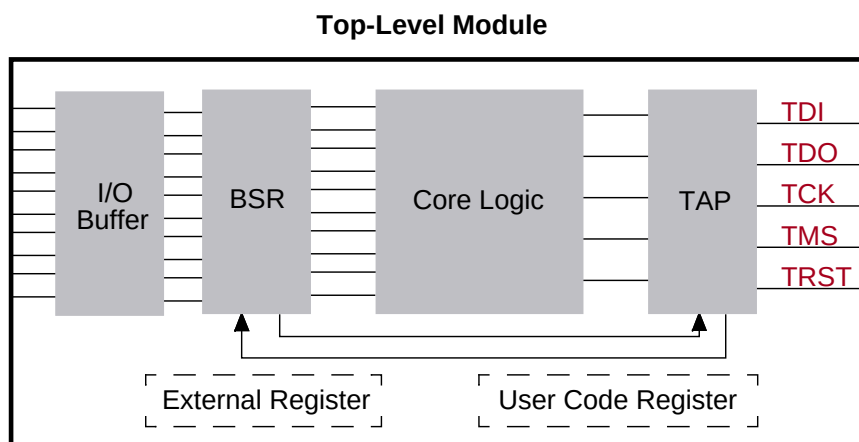
Using the **Set Bscan Flow** command with the Internal option prevents creation of the wrapper, and you can configure pads on all ports.

Following boundary scan insertion, the top-level module typically contains the following components by default in the External flow:

- Core Logic block
- Boundary Scan Register instance(s)
- TAP block
- External register (optional)
- User Code register (optional)
- I/O Buffer Block instance(s)

Figure 2-2 shows the top-level model that BSDArchitect produces.

Figure 2-2. Boundary Scan Output Model



In case of technology mapping, even if ASIC vendor boundary scan cells contain I/O pads, the boundary scan cells are placed in the BSR instance only.

If a USERCODE instruction is added using the Set Bscan Usercode -Port command, BSDArchitect generates another top level block for the device identification register.

The tool can also generate an unbuffered top-level design (no I/O Buffer-block instances). However, the following must be taken into account when generating an unbuffered top-level design.

- If you add tri-state ports, the top-level unbuffered output will have interface signals for the tri-state's control ports.
- If you add bidirectional ports, the tool will make available their control and input signals at the top-level.
- If you add differential ports, they will not be used in the top-level module and only one corresponding signal will be made available at the top-level.

To save unbuffered outputs, use the Save Bscan command with the -Unbuffered switch. The tool will not generate an RTL test bench or a BSDL description.

BSDArchitect Output Example

The Design

The design, *example.v*, is shown below:

```
module my_mod (CLK, D, EN1, IN_IO, OUT_IO, PAD_IN, PAD_OUT,Q);
  input CLK;
  input D;

  input [3:0] IN_IO; // inputs on bidi port
  output [3:0] OUT_IO; // outputs of bidi port

  output EN1; // bidi port enable
  input [1:0] PAD_IN; // bidi pin
  output [1:0] PAD_OUT; // bidi pin

  output Q;

endmodule
```

The Pinmap File

The following example shows a pin mapping file named *pinmap*. For information on the pinmap file, refer to [“Using a Pin Mapping File”](#).

```
--
-- Example pin map file. File name "pin_map".
--
-- Syntax:
--
-- logical_name PINID -p pad_name -c cell_names
--
-- The command to load the pinmap file is:
--
-- load pin map pin_map
--
--
CLK      clk      -c BC_4      -inv;
D        d_id     -c BC_3;
```

```
--
// note that you only specify the out pin for BC_7
//
PAD_OUT[0]  PAD_O[0]      -c BC_7;
PAD_OUT[1]  PAD_O[1]      -c BC_1;
PAD_IN[1]   PAD_I[1]      -c BC_1;
EN1         VOID          -c BC_2_A;
--
-- The ports for the I/O Bus IN_IO and OUT_IO are
-- added. The control for this port is an external
-- signal IO_CTRL (not in original design IO).
--
-- logical_name PINID -p pad_number -c cell_names
--
IO[0]  out[0]      -c BC_7;
IO[1]  out[1]      -c BC_1;
IN_IO[1] IN[1]     -c BC_1;
IO[2]  out[2]      -c BC_7;
IO[3]  out[3]      -c BC_7;
--
--
Q       DFFQ       -c BC_1;
--
-- The format for specifying a differential port's pin
--
-- ID's. If the core port is "SUM", the pin map will be:
--
-- SUM  S1 S2  -c BC_1;
--
-- where S1 and S2 are the pin ID's for the differential
-- port.
--
-- If a pad with no connections is desired. Use the value
-- "VOID" for both the port_name and PINID fields.
--
-- VOID  VOID -p 1;
-- VOID  VOID -p a2;
--
```

The Output

Figure 2-3 shows the four basic BSDArchitect output files generated by a default run.

Figure 2-3. BSDArchitect Output Files.

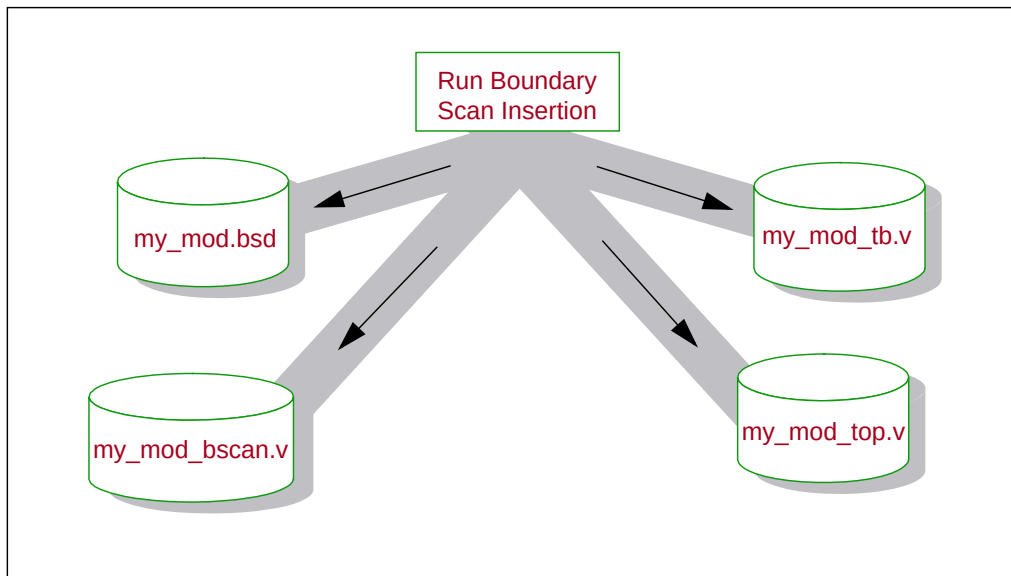


Figure 2-4. "my_mod_top.v" Output

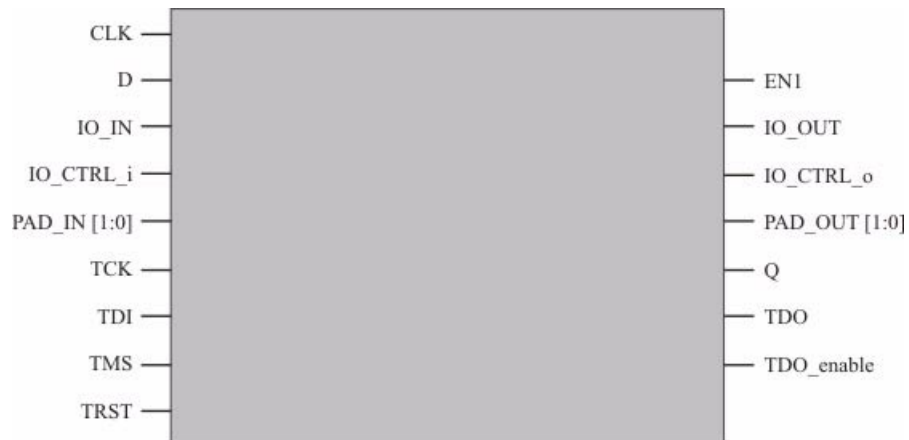


Figure 2-5. “my_mod_bscan_unbuff.v” Output

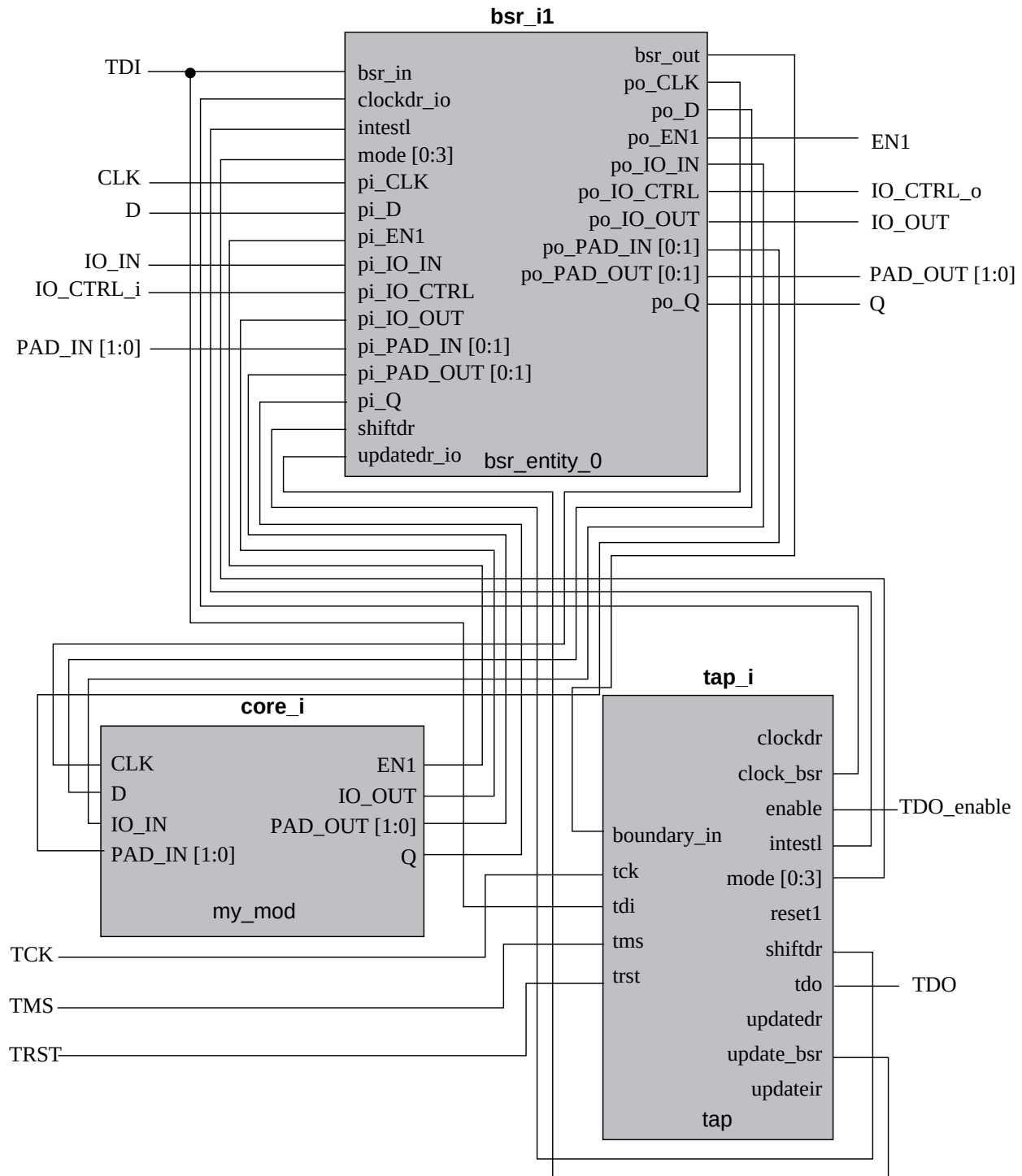
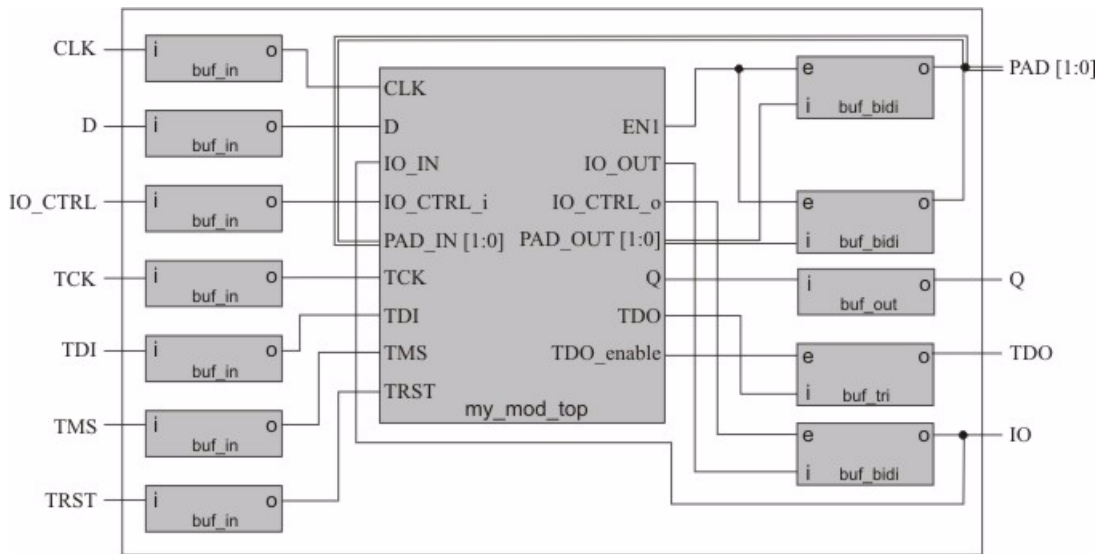


Figure 2-6. “my_mod_bscan.v” Output



The following sections discuss design and library issues you should understand before proceeding with boundary scan insertion.

Design Issues

The following subsections discuss design issues and the methods for handling these issues to achieve proper boundary scan insertion.

Logic Type of Entity Ports

You must declare all the ports in the VHDL entity to be of type `std_ulogic` or `std_logic`.

Escaped Identifiers for Verilog

BSDArchitect supports the use of Verilog escaped identifiers. BSDArchitect preserves the name specified in the port list. Therefore, if the port list name contains an escaped identifier, the BSDArchitect output name will also contain the escaped identifier. BSDArchitect considers legal Verilog names with and without escaped identifiers to be unique. That is, BSDArchitect does not equate “net1” with “\net1”.

Handling Tri-State and Bidirectional Ports

If your design contains bidirectional and/or tri-state devices, you should be aware of several important issues:

- **Width of enables**
BSDArchitect handles scalar, vector, or vector slice enables for each tri-state bus and each bidirectional bus.
- **Widths of bidirectional signal**
The widths of bidirectional input and output signals must be the same. BSDArchitect checks width constraints and reports errors when violations occur.
- **Specifying tri-state and bidirectional ports**
You must specify all tri-state and bidirectional enable signals. This means that for tri-state ports, you must specify the output and control signals, and for bidirectional ports, you must specify the input, output, and control signals. You can do so either by using the `mgc_bs_port_info` attribute in VHDL or ‘`specparam`’ statement in Verilog, or from the BSDArchitect session using the [Add Tristate Port](#) command. For best performance, use the `mgc_bs_port_info` attribute or the `specparam` statement. However, to work from the interactive BSDArchitect session, use the Add Tristate Port command.

Either method allows you to specify the input and output signals as a scalar (single port or indexed signal), vector (buses or indexed signals), or vector slices (a range of ports or indexed signals). You can also specify the control (enable) signal using the same criteria, as well as its (their) active level, either “`active_high`” or “`active_low`” (the default).

The following paragraphs provide examples of using `mgc_bs_port_info` in VHDL and Verilog, and of using the Add Tristate Port command from a BSDArchitect session.

A VHDL example of the `mgc_bs_port_info` attribute for a bidirectional port is:

```
attribute mgc_bs_port_info : string;
attribute mgc_bs_port_info of example_core : entity is
  "(d_i/in, d_o/out, d_ctrl/ctrl/active_high)";
```

A VHDL example of the `mgc_bs_port_info` attribute for a bidirectional port using VHDL range indicators is:

```
attribute mgc_bs_port_info : string;
attribute mgc_bs_port_info of example_core : entity is
  "(d_i(0 to 1)/in, d_o(0 to 1)/out, d_ctrl(1)/ctrl/active_high)";
```

NOTE: Using mismatched range indicators in `mgc_bs_port_info` will map as expected. For example, if you used the following ‘`downto`’ and ‘`to`’ range indicators, `d_i(0)` is mapped to `d_o(1)` as you might expect—and `d_i(1)` is mapped to `d_o(0)`:

```
(d_i(1 downto 0)/in, d_o(0 to 1)/out, d_ctrl...)
```

A VHDL example for a tri-state port is:

```
attribute mgc_bs_port_info : string;
attribute mgc_bs_port_info of example_core : entity is
  "(a_o/out, a_ctrl/ctrl/active_low)";
```

A VHDL example for handling multiple bidirectional or tri-state ports is:

```
library ieee;
use ieee.std_logic_1164.all;

entity tri is
    port(data_in : in std_logic_vector(31 downto 0);
          data_out : out std_logic_vector(31 downto 0);
          ctrl_bus : out std_logic_vector(1 downto 0);
          ctrl : out std_logic);

    attribute mgc_bs_port_info : string;
    attribute mgc_bs_port_info of tri : entity is \
"(data_in(31 downto 24)/in, data_out(31 downto 24)/out,\
ctrl_bus(1)/ctrl) (data_in(23 downto 16)/in,\
data_out(23 downto 16)/out, ctrl_bus(0)/ctrl)\
(data_in(15 downto 0)/in, data_out(15 downto 0)/out,\
ctrl/ctrl/active_high)";

end tri;

architecture only of tri is
begin
end only;
```

Do not use quotes, ampersands, or newlines between the parentheses defining the signals in the entity description. Line continuation characters and multiple attributes (with same name) are not allowed in VHDL. Specify multiple ports by placing them in a single continuous line as shown above.

Because Verilog simulators do not support user-defined attributes, you must use the specparam method to specify tri-state and bidirectional information.

A Verilog example of the mgc_bs_port_info specparam for specifying multiple tri-state ports is:

```
module aha(co,sum,c,b,a,d_i,d_o,d_c);
    input a,b,c;
    output sum,co;
    input d_i;
    output d_o, d_c;

    specify
        specparam mgc_bs_port_info = "(d_i/in, d_o/out, d_c/ctrl)";
    endspecify

endmodule
```

An Add Tristate Port command example for a bidirectional port is:

add tristate port -in d_i -out d_o -ctrl d_ctrl -active high

An Add Tristate Port command example for a bidirectional port using VHDL range indicators is:

add tristate port -in "d_in(3 downto 2)" -out "d_out(0 to 1)" -ctrl d_ctrl(1) -active high

An Add Tristate Port command example for a tri-state port is:

add tristate port -out a_o -ctrl a_ctrl -active low

- **Naming conventions**

You can specify your own naming convention by entering the name in the *mgc_bs_port_info* attribute. BSDArchitect adds one level of hierarchy to the design and names the top-level tri-state or bidirectional signals to match the output port name. Use the following syntax if you want to define a unique name for the top-level port:

```
<top_level_port_name>/top
```

For example:

```
attribute mgc_bs_port_info of example_core:entity is  
  "(d_i/in, d_o/out, d_ctrl_o/ctrl/active_high, d_out/top)";
```

- **Types of enable signals**

Control signals for designs that contain tri-state or bidirectional ports fit into one of two categories: internally generated or externally supplied. The following list describes the different categories of enable signals:

- **Internally generated control signals**

If the core logic's internal circuitry generates the control signal for a bidirectional or tri-state output, you need to specify the control signal in two different ways: as a port of mode "out" in the port map of the entity description and by specifying it with the *mgc_bs_port_info* attribute.

- **Externally supplied control signals**

There are two cases for externally supplied control signals. In the first case (see "Case A" that follows), the core logic does *not* use the enable signals that control these tri-state and/or bidirectional ports. In the second case (see "Case B" that follows), the core logic *does* use the enable signals that control these ports. BSDArchitect handles each case differently.

- **Case A (core does not use enable)**

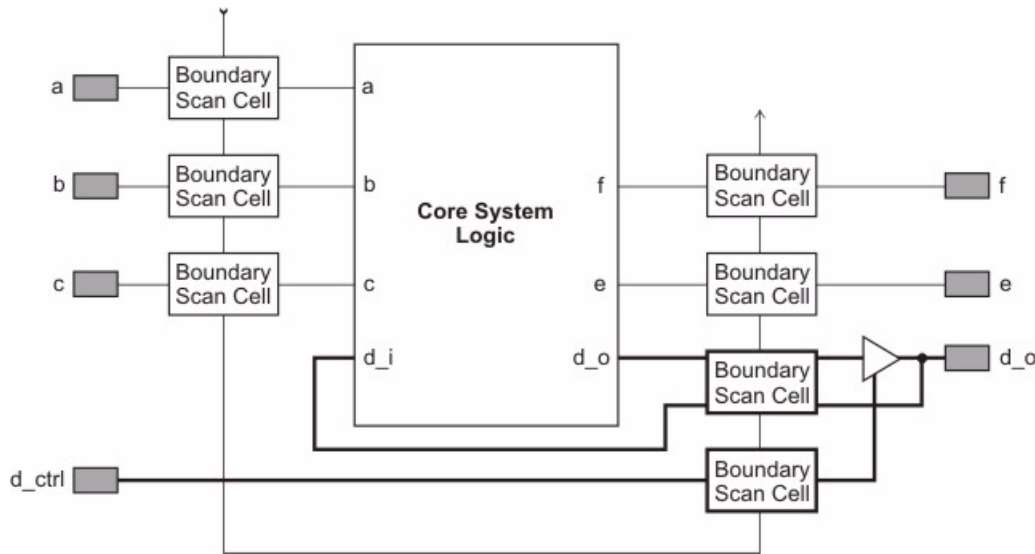
Because the internal circuitry does not use the enable, you must specify the enable signal in the *mgc_bs_port_info* attribute, but not in the core logic's entity port map. For example, if your design has three input ports (a, b, c), one bidirectional port (d), and two output ports (e, f), all of type *std_ulogic*, your entity description should look like this:

```
entity example_core is port (  
  a: in std_ulogic;  
  b: in std_ulogic;  
  c: in std_ulogic;  
  d_i: in std_ulogic;  --d_i and d_o are bi-dir port  
  d_o: out std_ulogic;
```

```
e: out std_ulogic;  
f: out std_ulogic);  
attribute mgc_bs_port_info : string;  
attribute mgc_bs_port_info of example_core:entity is  
"(d_i/in, d_o/out, d_ctrl/ctrl/active_high)";  
end example_core;
```

In this case, when BSDArchitect processes the *mgc_bs_port_info* attribute in the entity and finds an enable signal that is not in the port declaration, it adds the signal at the top level and uses it as an enable, as shown in [Figure 2-7](#).

Figure 2-7. Handling of Enable Signals Not Used in Core



In this case, the tool places a single boundary scan cell on the enable signal. You can use your own naming convention for the top-level bidirectional signal name by following the “[Naming conventions](#)” on page 33.

- **Case B (core uses enable)**

Because the enable signal controls the output ports as well as some of the internal core circuitry, you should include the enable signal in the port map for the core logic. For example, if your design has three input ports (a, b, c), a bidirectional port (d), and two output ports (e, f), all of type *std_ulogic*, your entity description should look like this:

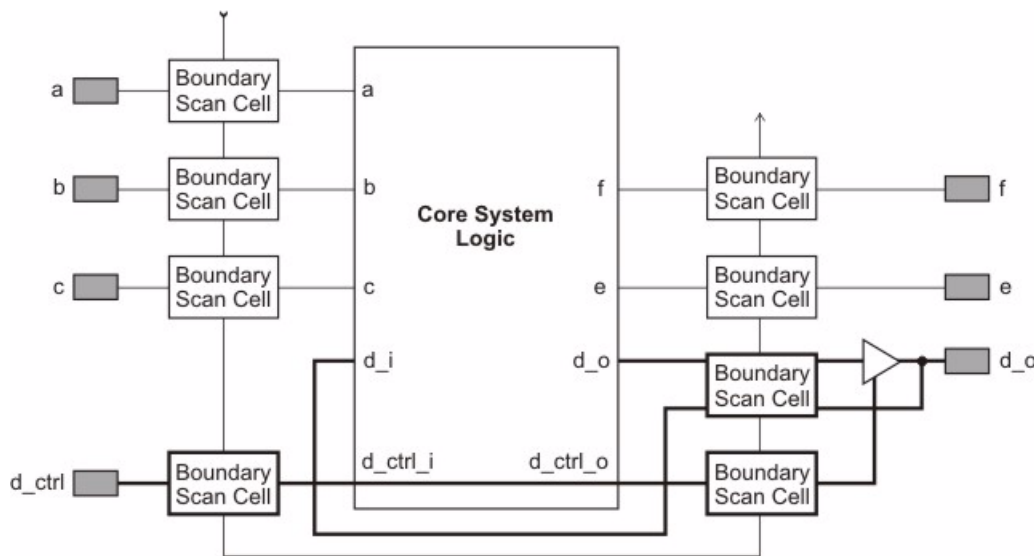
```

entity example_core is port (
  a: in std_ulogic;
  b: in std_ulogic;
  c: in std_ulogic;
  d_i: in std_ulogic;  --d_i and d_o are bi-dir ports
  d_ctrl_i: in std_ulogic;
  d_o: out std_ulogic;
  d_ctrl_o: out std_ulogic;
  e: out std_ulogic;
  f: out std_ulogic);
attribute mgc_bs_port_info : string;
attribute mgc_bs_port_info of example_core:entity is
  "(d_i/in, d_o/out, d_ctrl_o/ctrl/active_high)";
end example_core;

```

You should connect `d_ctrl_o` to `d_ctrl_i` inside the core. When BSDArchitect processes the entity and finds the enable signal in the port map, it places one boundary scan cell on the input side (`d_ctrl_i`) and one boundary scan cell on the output side (`d_ctrl_o`), as shown in [Figure 2-8](#).

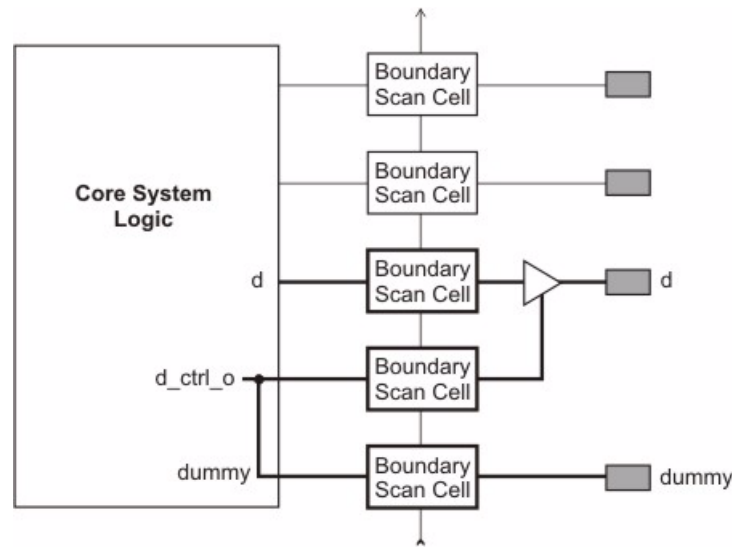
Figure 2-8. Handling of Enable Signals Used in Core



- **Accessing core-generated enable signals at the top level**

[Figure 2-9](#) shows an example of an enable signal generated by the core. As is also the case in both [Figure 2-7](#) and [Figure 2-8](#), when BSDArchitect generates a bidirectional or tri-state port within the boundary scan circuitry, the enable signal is not available as an output at the top level. If you need to access this enable, you must connect a dummy signal to the enable signal within the core circuitry and specify the name of the signal as an output of the core logic in the port map.

Figure 2-9. Accessing the Enable



Limitations

You should be aware of the following limitations when you use BSDArchitect:

- **No testing of user-supplied instructions unless core register length is specified.**

The test bench only tests user-supplied instructions when the core register length is known. The test bench tests all other user-supplied instructions.

- **No testing of INTEST and RUNBIST instructions.**

The test bench does not test the INTEST or the RUNBIST instructions.

- **Support for only one internal scan interconnection method.**

BSDArchitect supports interconnection with internal scan circuitry, but currently there is only one method for achieving this. If your design contains internal scan, refer to [“Connecting Internal Scan Circuitry”](#) on page 118 for the recommended method of connecting the internal scan circuitry with the boundary scan circuitry.

Support for only mux-DFF and clocked-scan internal scan architectures. Currently, the tool supports only the mux-DFF and clocked-scan internal scan architectures.

- **VHDL extended identifier support limitation.**

Extended identifiers for VHDL are not supported at this time.

- **Timing issues BSDArchitect does not handle.**

You are responsible for designing the clock distribution tree and ensuring that no timing problems arise due to clock skew. BSDArchitect itself does not check for these problems.

- **Verilog support has some limitations or differences from the standard.**

There are some differences between the Verilog supported by BSDArchitect and that defined by the standard.

Specifically, BSDArchitect does not support the following:

- **Non-optional specify_output_terminal_descriptor in list_of_path_outputs.**

For example, BSDArchitect will not parse the path declaration `/***/` if written as follows:

```
(clear *> q, nq) = (tRiseCtlQ, tFallCtlQ); /***/  
(clear, preset *> q) = (tRiseCtlQ, tFallCtlQ); /***/
```

- **Optional if-expression in edge_sensitive_path_description (non_optional).**

For example, you cannot use the following syntax:

```
( posedge clock => ( q +: in ) ) = ( 10, 8 );
```

Additionally, it is a good idea to put port declarations before any other module items. Thus, if the port declarations are first in the file, you need not worry if the remainder of the file contains unsupported constructs or syntax.

- **Adding pull-up resistors to the boundary scan ports.**

The IEEE 1149.1-2001 standard specification states that boundary scan ports TDI, TMS, and TRST (if used) require pull-up resistors to power up and keep these signals at known states when they are idle. BSDArchitect produces an output VHDL file that supports this requirement. After completing synthesis and optimization, you must manually add I/O ports that contain these pull-up resistors to the design.

- **Power-up reset circuitry.**

BSDArchitect does not create power-up reset circuitry. You can initialize a simulator to simulate the circuitry BSDArchitect creates. However, you must eventually add circuitry to the test logic that controls the power-up reset of the TAP controller to avoid bus contention and possible damage to the on-chip logic.

- **Clocking for internal scan chains.**

If you have a single internal scan chain that spans a design, and this design contains various components controlled by clocks of different frequencies, you must make sure that the clocking for your internal scan chain functions properly. BSDArchitect does not perform this type of checking.

- **HDL test bench operation.**

The HDL test bench that the BSDArchitect tool creates operates the AC cells in DC mode, and the tests are digital. However, the tool does activate the EXTEST_PULSE and EXTEST_TRAIN instructions to verify generation of the pulse and train signals.

Recommended Practices

The following are recommended practices:

- **Using TRST in the TAP (1149.1 Recommended Practice)**

To ensure deterministic operation of the test logic when testing the reset TAP controller state, you should hold TMS at 1 while TRST changes from a 0 to a 1. This makes sure that the test logic responds predictably when the signal applied to TRST changes from 0 to 1. If rising edges occur simultaneously at TRST and TCK (when a logic 0 is applied to TMS) a race condition occurs. This can cause the TAP controller to either remain in the Test-Logic-Reset state or enter the Run-Test/Idle state.

- **Using BSDArchitect-reserved identifiers**

- Do not use the names for JTAG ports, IEEE instructions, and TAP signals as port names in your HDL model or netlist. For a list of these names, refer to tables C-5 through C-7 in [Appendix C](#) of the *BSDArchitect Reference Manual*. The JTAG port names may be used in your design only if they define ports intended for use with the generated TAP controller.
- Do not use the BSDArchitect-generated component names for boundary scan cells, JTAG modules, buffers, and pad cells as module names in your design. Refer to tables C-1 through C-4 in [Appendix C](#) of the *BSDArchitect Reference Manual* for a complete list of generated names.

Preparing for Boundary Scan Insertion

The following subsections discuss the tasks you must perform to prepare for boundary scan insertion.

Boundary Scan Example Design

This section uses an example full-adder called “aha” to demonstrate the boundary scan design process. The VHDL description of the “aha” component follows.

```
library ieee;
use ieee.std_logic_1164.all;

entity aha is
    port( s:out std_ulogic;
          co:out std_ulogic;
          a:in std_ulogic;
          b:in std_ulogic;
          c:in std_ulogic);
end aha;

architecture rtl of aha is
begin
```

```
s<=a xor b xor c;  
co <=(a and b) or (a and c) or (b and c);  
end rtl;
```

Creating the HDL Description

BSDArchitect requires an HDL definition (in either VHDL or Verilog) as a high-level interface to your design. You should specify the top level module using the `-top` argument at invocation when more than one entity/module is present in a design.

Creating the Package Mapping File

BSDArchitect optionally supports placing a package mapping file called *dft.map* in your working directory for using custom library packages for VHDL designs. This mapping file specifies any packages/libraries used in the core entity description. By default, without a package mapping file, BSDArchitect uses the following required standard library packages:

For VHDL:

```
<Tessent_Tree_Path>/lib/tecnhnology/vhdl_src/stdlogic.vhd  
<Tessent_Tree_Path>/lib/tecnhnology/vhdl_src/standard.vhd
```

If you do not specify the location of the `std_logic_1164` and `standard` packages in your custom *dft.map* file, BSDArchitect searches these locations by default.

For more information on the syntax of this file, refer to “[Package Mapping File Syntax](#)” in the *BSDArchitect Reference Manual*.

Invoking BSDArchitect

To invoke BSDArchitect, enter the following at the shell prompt:

```
shell> <Tessent_Tree_Path>/bin/bsdarchitect hdl_design_name -top top_design_name
```

After BSDArchitect invokes, it displays the following prompt and the command line:

```
BSDA>
```

You can now proceed to set up for and insert boundary scan logic. The invocation for VHDL input and Verilog input is identical. BSDArchitect reads in either format, and later produces output in the same format used as input.

Setting Up the Boundary Scan Specification

You have two choices for setting up the boundary scan logic: using system defaults, or customizing the boundary scan architecture. If you choose to use the system defaults,

BSDArchitect requires no prior setup. You just execute the boundary scan insertion by issuing the Run command.

The boundary scan specification consists of a set of commands that customize your particular boundary scan implementation. Refer to “[Boundary Scan Customizations](#)” for more information. You can either enter commands interactively at the BSDA> command line prompt, store the setup commands in a file and run it using the Dofile command. For more information on dofiles, refer to “[Running Batch Mode Using Dofiles](#)” on page 20.

Boundary Scan Insertion Using BSDArchitect

The following example tells BSDArchitect to create a standard boundary scan architecture for the same example Verilog design, *example.v*.

The Example Design

The HDL description for the design, *example.v*, is the following:

```
module my_mod (CLK, D, EN1, IN_IO, OUT_IO, PAD_IN, PAD_OUT,Q);
    input CLK;
    input D;

    input [3:0] IN_IO; // inputs on bidi port
    output [3:0] OUT_IO; // outputs of bidi port

    output EN1;          // bidi port enable
    input [1:0] PAD_IN; // bidi pin
    output [1:0] PAD_OUT; // bidi pin

    output Q;

endmodule
```

Running BSDArchitect in an External Flow

1. First you invoke BSDArchitect at the shell:

```
shell> <Tessent_Tree_Path>/bin/bsdarchitect example.v -verilog -top core
```

2. Then enter the following commands at the “BSDA>” prompt:

```
load pin map pin_map
run
report bscan cell
report bscan status
save bscan -replace
exit
```

This example loads a pin mapping file (for an example, see “[The Pinmap File](#)”), runs the boundary scan insertion with all system defaults (using the Run command with no prior setup)

and reports on the logic that BSDArchitect adds. BSDArchitect then saves the buffered output (*my_mod_bscan.v*), the interface description file (*my_mod_top.v*), the test bench file (*my_mod_tb.v*), and the BSDL description file (*my_mod_bscan.bsd*).

Note



You must save the output netlist in the same format (VHDL or Verilog) as was the input netlist that you read into the tool. For more information see the [Set Output Format](#) command.

The following report shows the default architecture created in this session.

```

-----
                          BSDArchitect  Report
-----
Design : my_mod
-----
Asynchronous TAP Reset : Yes
Device Identification Register : No
User Identification Register : No
Instruction Register Width : 4

INSTRUCTION REPORT:
-----
Instruction                                Target Register                                Opcode
-----
EXTEST                                    BOUNDARY                                    0000
BYPASS                                    BYPASS                                    1111
SAMPLE                                    BOUNDARY                                    0001
PRELOAD                                    BOUNDARY                                    0001
-----

BSDArchitect Port Report
-----
Design : my_mod
-----
No.  Top-Port    BSC Mode    Type    Cell-Name BSR    Instance Internal
-----
1    CLK          YES Observe   bc_4    BC_4      bsr_i1 NO
2    D            YES Both     bc_3    BC_3      bsr_i1 NO
3    PAD_OUT[0]   YES Both     bc_7    BC_7      bsr_i1 NO
4    PAD_OUT[1]   YES Both     bc_1    BC_1      bsr_i1 NO
5    PAD_IN[1]    YES Both     bc_1    BC_1      bsr_i1 NO
6    EN1          YES Both     bc_2_a  BC_2_A    bsr_i1 NO
7    Q            YES Both     bc_1    BC_1      bsr_i1 NO
8    IO_OUT       YES Both     bc_1    bc_1      bsr_i1 NO
9    IO_IN        YES Both     bc_1    bc_1      bsr_i1 NO
10   PAD_IN[0]    YES Both     bc_1    bc_1      bsr_i1 NO
-----

Total no. of BS cells : 10
Note: Port ordering is from TDO to TDI.

```


Chapter 3

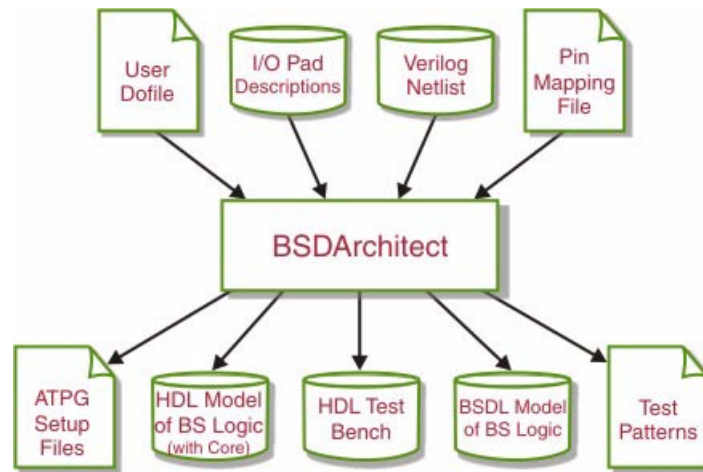
Internal Boundary Scan Synthesis

This chapter outlines how to use BSDArchitect to synthesize boundary scan circuitry for your design using the Internal flow. For specific information on BSDArchitect functionality or commands, refer to the [BSDArchitect Reference Manual](#).

Understanding BSDArchitect Internal Flow

For the Internal flow, BSDArchitect adds the boundary scan logic to the design internally. That is, the TAP controller instance is placed within the original top-level of the design and the boundary scan cells are inserted between the I/O pads and the core logic. Designs that already include I/O pads should use the Internal flow. [Figure 3-1](#) shows the inputs and outputs for a design using the Internal flow.

Figure 3-1. BSDArchitect Internal Flow Inputs and Outputs



Inputs

There are two required inputs: the design netlist and the I/O pad descriptions. The design netlist must be formatted in Verilog and the I/O pad descriptions must use Verilog specparams. The I/O pad descriptions may be contained within the netlist. For more information, see “[Building I/O Pad Definitions](#)” on page 44. Refer to “[Recommended Practices](#)” on page 38 for more information on formatting your design.

Optional input files include the pin mapping file and user dofile. For more information on pin mapping files, see “[Loading Pin Maps](#)” on page 55. You can execute a dofile using the Dofile

command or the -Dofile switch at invocation. Refer to “[Running Batch Mode Using Dofiles](#)” on page 20 for more information on dofiles.

 **Note** VHDL netlists are not supported in the Internal flow.

Outputs

BSDArchitect formats the output HDL models and test bench files in the Internal flow in Verilog. For example, the <prefix>_bscan.<suffix> output file will be <prefix>_bscan.v. The remaining file formats are as described in “[BSDArchitect External Flow](#)” on page 23.

Building I/O Pad Definitions

To insert boundary scan cells, you must define the I/O pad cells in your design. These pads may be located in any level of hierarchy in the netlist. You create the I/O pad cell descriptions using Verilog specparams in one of two methods:

- In a separate Verilog file — Direct the tool to read this file by including the -Library switch at invocation. Design updates may be simplified with this method because updates for the I/O pad definitions do not require changes to the original netlist. This is the recommended method.
- In the design netlist — When you place Verilog specparams within the design netlist, the netlist must contain the Verilog descriptions of the I/O pad cells and you must put the specparams within these descriptions. The -Library switch is not needed at invocation for this method.

The important parameters in both of these methods are the name of the design and the mapping information. This information, along with the Verilog descriptions of the I/O pads, is enough for the tool to search for existing I/O pads in the design. For more information on using Verilog specparams, refer to “[Defining I/O Pads](#)” on page 99.

Invoking BSDArchitect

Use the following command to invoke BSDArchitect:

```
shell> <Tessent_Tree_Path>/bin/bsdarchitect design_name.v \  
-top top_design_name -library pad_definitions.v
```

Include the -Library switch along with the name of the appropriate I/O pad descriptions file. You do not need to use the -Library switch if your design netlist already contains Verilog

specparams to define the I/O pads. Refer to “[Shell Command Dictionary](#)” in the *BSDArchitect Reference Manual* for more information on the invocation and available switches.

Figure 3-2. BSDArchitect Tool Flow

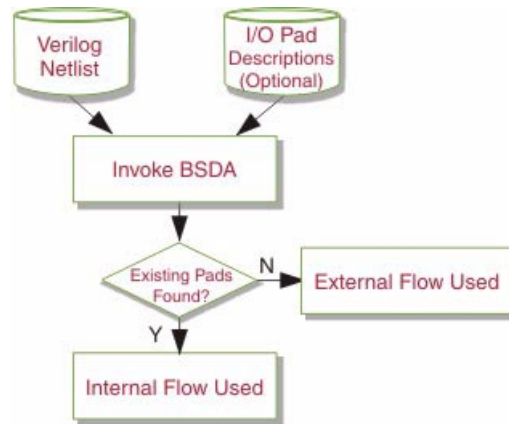


Figure 3-2 illustrates the tool flow for BSDArchitect. Once invoked, BSDArchitect searches the specified netlist for I/O pads. If existing pads are found, the tool uses the Internal flow and inserts the necessary boundary scan logic. If there are no I/O pads in the design or the existing I/O pads are not recognized, the tool reports that no pad instance was found and uses the External flow, creating a wrapper over the core module. If BSDArchitect is not detecting your existing pads, see “[Defining I/O Pads](#)” on page 99 to verify that your pads are defined correctly.

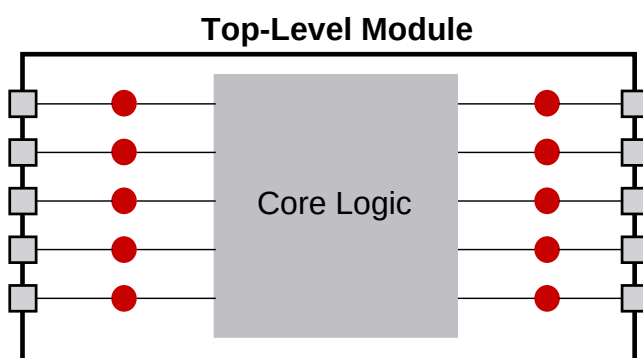
Note

Once BSDArchitect is running, you can force a core wrapper to be created by issuing the [Set Bscan Flow](#) command with the Wrapper option.

Example on Boundary Scan Insertion

Consider a simple design where all I/O pad cells are defined and all pads exist in the same hierarchy. [Example 3-1](#) shows this simple example where the small grey squares are the instances of I/O pads and the red circles are where BSDArchitect will insert boundary scan cells. This is an example for the Internal flow.

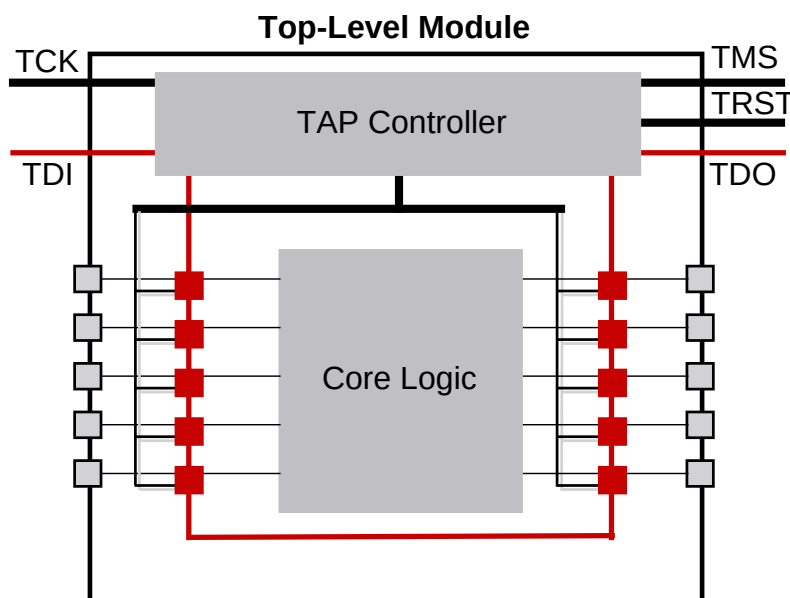
Example 3-1. Internal Boundary Scan Input



The tool searches for the pad cells and locates the ports on each pad. When the Run command is issued, the tool traces the pad connections checking for errors. When no errors are found, the tool inserts the boundary scan circuitry.

[Example 3-2](#) shows the resulting boundary scan register (BSR) inserted by the tool highlighted in red. The red squares represent the newly inserted boundary scan cells.

Example 3-2. Internal Boundary Scan Output



Reporting Circuit Components

The [Report Circuit Components](#) command can help in debugging by providing a hierarchical list of all the instances in a design.

Use the following command to report all the instances in the design:

report circuit components

Or include the optional *design_name* argument to display the hierarchical tree structure of all the instances within a specific instance. For example:

```
report circuit components my_design
```

Adding I/O Pads to Ports Without Pads

BSDArchitect allows you to add I/O pads only to ports without pads; the ports must be defined in the netlist. The pads specified in the netlist cannot be changed. You can use custom-defined pad cells of any type as long as their corresponding ports are configured for that type. If a port does not have an existing pad cell and you do not specify one, BSDArchitect inserts a default I/O pad cell. The tool places this default pad cell, as well as any custom-defined pad cells, inside of a pad block.

Pads are added using the following commands:

- [Add Technology Cell](#) — Defines cell information for technology mapping of boundary scan cells, I/O pad cells, and custom TAP controllers.
- [Set Technology Mapping](#) — Sets a target technology when creating technology-specific boundary scan logic.
- [Add Pad Cell](#) — Associates a specified I/O pad cell with specified core ports.
- [Set Pad Cell](#) — Associates specified I/O pad cells with specified port types.

The tool adds JTAG ports and pads after creating the boundary scan logic. For new JTAG ports, the tool creates five new pins and adds default pads to those pins. A tri-state pad is added for TDO. For more information on optional TDO connections, see “[Sharing Functional Pins With TDO](#)” on page 58. Custom input and output pads can be added for JTAG ports.

Existing JTAG ports are recognized in one of two ways:

- The ports have standard names (tdi, tms, tck, trst, tdo). The tool automatically recognizes the ports and adds the default or user-specified I/O pads.
- The ports have user-specified names given by the [Set Controller Naming](#) command. This command is either entered in the command line or executed within a dofile.

BSDArchitect places all new pads in the same module at the top level of the design. This includes pads added on both ports without pads and on JTAG ports. You cannot use the Set Pad Instances command in the Internal flow to change this behavior.

The following example defines and associates an output pad pad_out_1 with output port out1:

```
add technology cell pad_out_1 output -inputs in -outputs out -connect in/dout out/pin
add pad cell pad_out_1 out1
```

Reporting Pad Cells

When you issue the [Report Pad Cell](#) command in the Internal flow, the tool displays the pad cell information sorted into two sections. The sections vary depending on whether the report command is issued before or after the [Run](#) command. The possible sections are:

- Ports on which pads were FOUND

This section is always included in the report regardless of the Run command. It lists the information for all pad cells detected in the original netlist.

- Ports on which new pads are to be ADDED

This section is included only when you issue the Report Pad Cell command before the Run command. It lists the pads intended for the ports. These may be default pads from the tool or user-specified pads. The instance name column is not included because the pads have not yet been added to the design. The pads are added to the design during the Run command.

- Ports on which new pads are ADDED

This section is included only when you issue the Report Pad Cell command after the Run command. It lists the pads that were added to the design. The pads listed in this section are the same pads listed in “Ports on which new pads are to be ADDED.” The instance names are included because the pads were added.

The following example shows a report issued before the Run command:

BSDA> report pad cell

```
-----
          BSDArchitect Pad Report
-----
Design: test_chip
Date: Tue Oct 5 13:25:41 2004
-----
Ports on which pads were FOUND
-----
Core-Port      Top_Port      Type      Cell Name Instance
-----
bit_bidi       bit_bidi      Bidi...   bidi_pad     test_chip/test_pad_inst/b...
bus_bidi[1]    bus_bidi[1]   Bidi...   bidi_pad     test_chip/test_pad_inst/b...
bus_bidi[0]    bus_bidi[0]   Bidi...   bidi_pad     test_chip/test_pad_inst/b...
bit_o          bit_o         Output    out_pad      test_chip/test_pad_inst/o...
bit_tri_o      bit_tri_o     Tristate  tri_pad      test_chip/test_pad_inst/t...
bus_o[0]       bus_o[0]      Output    out_pad      test_chip/test_pad_inst/o...
bus_o[1]       bus_o[1]      Output    out_pad      test_chip/test_pad_inst/o...
bus_tri_o[0]   bus_tri_o[0]  Tristate  tri_pad      test_chip/test_pad_inst/t...
bus_tri_o[1]   bus_tri_o[1]  Tristate  tri_pad      test_chip/test_pad_inst/t...
bus_i[0]       bus_i[0]      Input     in_pad       test_chip/test_pad_inst/i...
bus_i[0]       bus_i[0]      Input     in_pad       test_chip/test_pad_inst/i...
```

Ports on which pads are to be ADDED

Core-Port	Top-Port	Type	Cell Name
bit_i	bit_i	Input	io_pad_in
tdi	tdi	Input	io_pad_in
tms	my_tms	Input	io_pad_in
tck	my_tck	Input	io_pad_in
tdo	tdo	Tristate	tri_enable_high
trst	my_trst	Input	io_pad_in

Adding Boundary Scan Cells

To specify boundary scan cells for your ports, use one of the following:

- To specify a boundary scan cell for one or more specific ports, use the Add Bscan Cell command. The following example places a BC_1 type cell on the output ports out1 and out2:

```
add bscan cell bc_1 out1 out2
```

- If you do not want a boundary scan cell placed on a particular port, use the Add Bscan Cell command and specify 'none' for the boundary scan cell type. The following example specifies that no boundary scan cell is placed on output port out3:

```
add bscan cell none out3
```

- To specify boundary scan cells for all ports or ports of a given type, use the Set Bscan Cell command. For example you can specify a cell for placement on all input ports. Any cells specified with the Add Bscan Cell command will override the Set Bscan Cell command regardless of the command order. The following example places a BC_1 type cell on all output ports:

```
set bscan cell -out bc_1
```

- To specify boundary scan cells for each port and specify the port order, use a pin mapping file. For more information on using a pin mapping file, see [“Loading Pin Maps”](#) on page 55.

After specifying boundary scan cells, verify your changes with the [Report Bscan Cell](#) command. BSDArchitect inserts the specified or default boundary scan cells in your design when you issue the Run command.

Boundary Scan Cell Placement

The following sections describe the placement of boundary scan cells for data ports. For information on bidirectional and tri-state control ports, see [“Boundary Scan Cell Placement for Control Ports”](#) on page 52.

Ports Without I/O Pads in the Netlist

Boundary scan cells are added to the top level of the design. For more information, see “[Adding I/O Pads to Ports Without Pads](#)” on page 47.

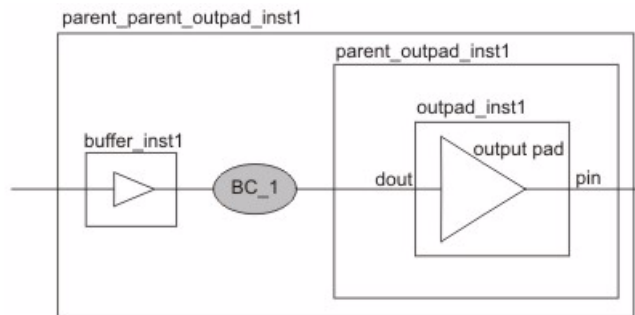
Ports With I/O Pads in the Netlist

Boundary scan cells are inserted in the parent module of the I/O pad if there is only one instance of the parent module in the design. If there is more than one instance of the I/O pad parent module, the placement is as follows:

- Boundary Scan Cells on a Single Data Port

The boundary scan cell is placed at the highest hierarchy between the I/O pad port and any logic encountered along the data path. In [Example 3-3](#), the BC_1 cell is placed between the dout port and the buffer. The highest hierarchy between them is *parent_parent_outpad_inst1*.

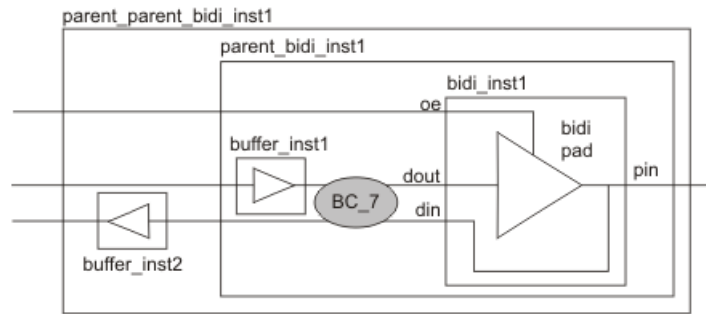
Example 3-3. Boundary Scan Cell Placement on Single Data Port



- Boundary Scan Cells on Multiple Data Ports

For the bidirectional 2-cell model, the two data ports, din and dout, share a single boundary scan cell. Each port is traced back to any logic and the boundary scan cell is placed at the highest hierarchy common to both ports. In [Example 3-4](#), the dout port can be traced back to the buffer in *parent_bidi_inst1*, and the din port can be traced back to the buffer in *parent_parent_bidi_inst1*. The highest hierarchy common to both ports is *parent_bidi_inst1*, which is where the BC_7 cell is placed.

Example 3-4. Boundary Scan Cell Placement on Multiple Data Ports



Note



Your design may be modified, if necessary, to unify modules. For more information, see “[Uniquification](#)” on page 59.

Adding Boundary Scan Cells to Control Ports

BSDArchitect adds the following cells for bidirectional and tri-state ports:

- Tri-state ports — Two boundary scan cells are added: one cell for the output port and one for the control port. BSDArchitect uses a BC_1 cell for both ports by default.
- Bidirectional 3-cell model — Three boundary scan cells are added: one cell is added for each of the input, output, and control ports.
- Bidirectional 2-cell model — Two boundary scan cells are added: one combined cell for the input and output data ports and one cell for the control port. BSDArchitect uses the bidirectional 2-cell model by default with a BC_7 combined cell for the data ports and a BC_2_A cell for the control port.

In the Internal flow, BSDArchitect inserts all boundary scan circuitry into the existing netlist. The control ports for tri-state and bidirectional pads may not be visible at the top level of the design. To access these ports, use the notation in [Table 3-1](#) for the [Add Bscan Cell](#) and [Delete Bscan Cell](#) commands.

Table 3-1. Control Port Notation

Port	Notation
Output (tri-state)	<port_name>
Inout Interface (bidirectional)	<port_name>
Control	<port_name>/oe
Input (bidirectional)	<port_name>/din

Use parentheses when specifying a control port of a bus signal. For example:

```
BSDA> add bscan cell my_bc_2 bus_tri_o(0)/oe
```

The following example adds a BC_1 type cell to the port named “tri” and its control port:

```
add bscan cell BC_1 tri tri/oe
```

The following example adds the user-defined cell my_BC_7 to the bidirectional port named “bidi_X”:

```
add bscan cell my_BC_7 bidi_X
```

The following example adds the boundary scan cell my_BC_2_A to the control ports of the bidirectional ports named “bidi_X” and “bidi_Y”:

```
add bscan cell my_BC_2_A bidi_X/oe bidi_Y/oe
```

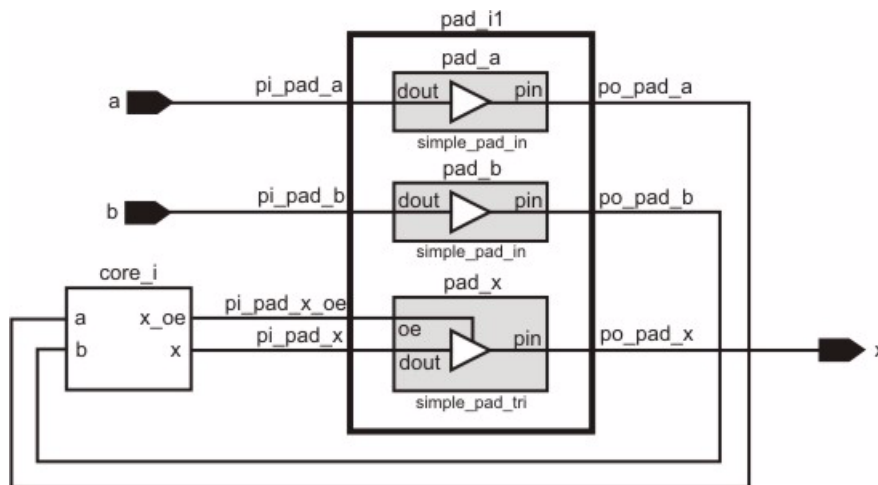
The following example of a bidirectional 3-cell model adds a BC_1 boundary scan cell to the output, control, and input of the bidirectional port named “bidi_K”:

```
add bscan cell BC_1 bidi_K bidi_K/oe bidi_K/din
```

[Example 3-5](#) illustrates where there is no top-level signal associated with the control port of the tri-state pad. The following commands add boundary scan cells to the tri-state port x using the notation in [Table 3-1](#):

```
add bscan cell bc_1 x  
add bscan cell bc_1 x/oe
```

Example 3-5. No Top-Level Signal for a Tri-State Control Port



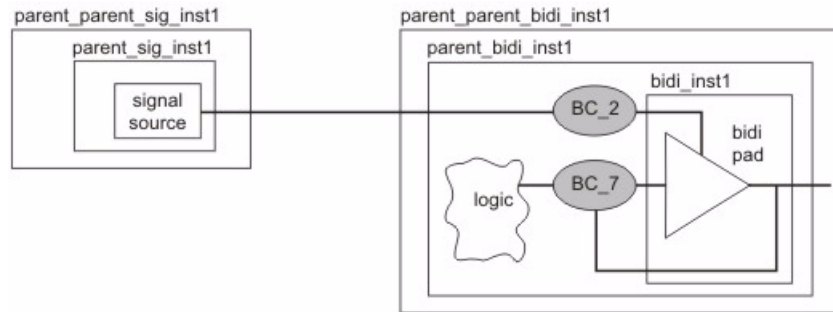
Boundary Scan Cell Placement for Control Ports

The placement of bidirectional or tri-state control boundary scan cells depends on the control signal configuration. Each of the following configurations apply to both internal and external control signal sources.

- The control signal supports a single pad and does not pass through an inverter or buffer.

BSDArchitect places the control boundary scan cell in the parent module of the bidirectional or tri-state pad. [Example 3-6](#) shows the BC_2 cell placed within the parent module at the same hierarchy as the bidirectional pad instance.

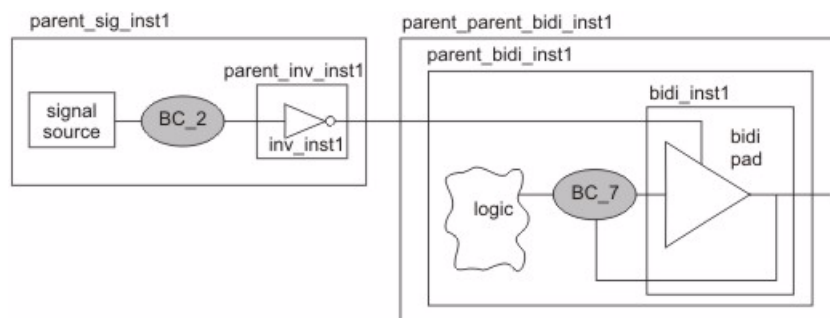
Example 3-6. Single Control Signal Without Inverter/Buffer



- The control signal supports a single pad and passes through one or more buffers or inverters.

BSDArchitect places the control boundary scan cell behind all buffers in the control signal path at the same hierarchy as the parent module of the first buffer or inverter. If multiple instances of the parent module exist, the control boundary scan cell is placed at the highest hierarchy between the signal source and the buffer or inverter. [Example 3-7](#) shows the BC_2 cell placed at the same hierarchy as the inverter parent instance *parent_inv_inst1*. If multiple instances of *parent_sig_inst* exist, the BC_2 cell would be in the same position because this location is also the highest hierarchy between the signal and the inverter parent instance.

Example 3-7. Single Control Signal With Inverter

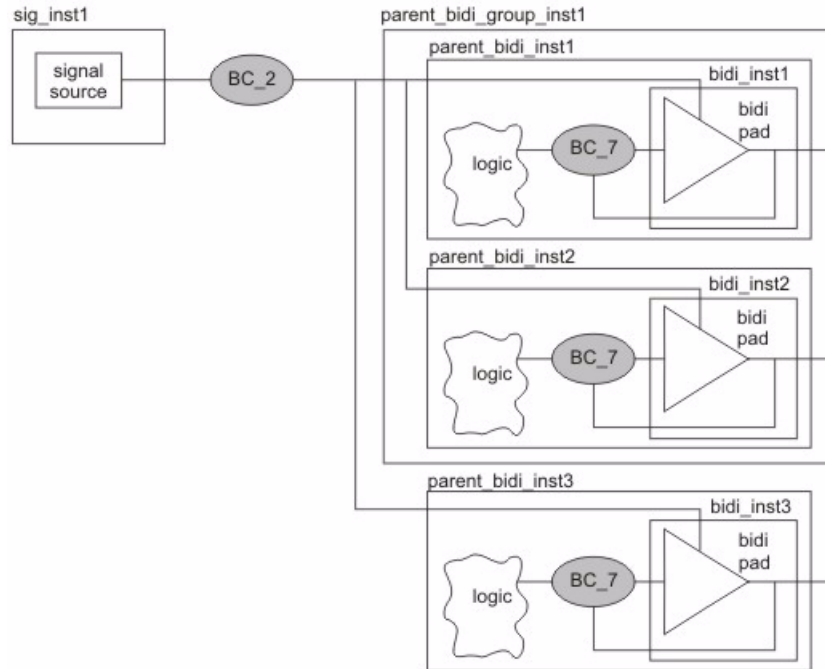


- The control signal supports multiple pads and does not pass through an inverter or buffer.

BSDArchitect places the control boundary scan cell immediately before the common point within the same module as the common point. [Example 3-8](#) shows the BC_2 cell placed in the top module just before the control signal splits to the separate pads. If this example only contained the first two bidirectional pads and the common point was

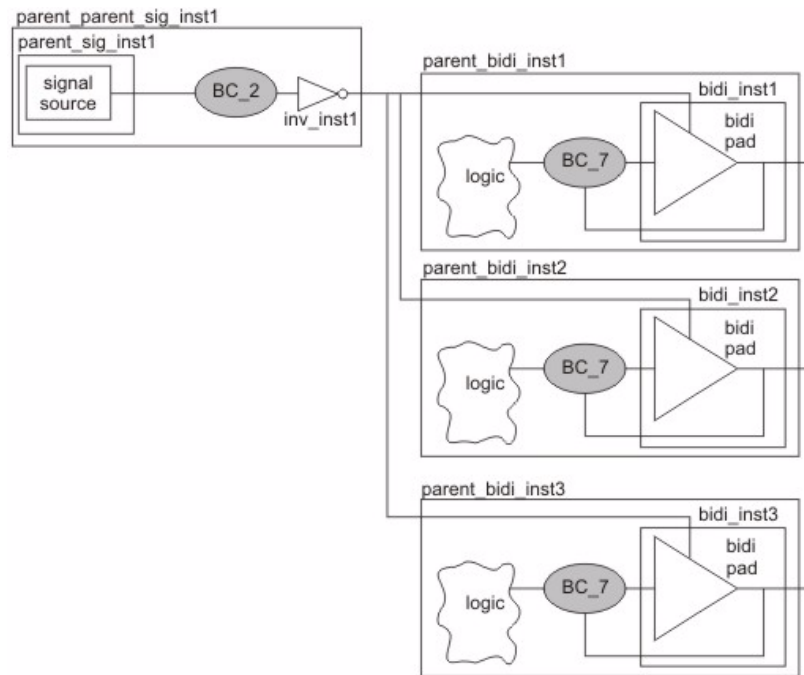
contained in *parent_bidi_group_inst1*, then the BC_2 cell would also be placed in *parent_bidi_group_inst1*.

Example 3-8. Multiple Control Signals Without Inverter/Buffer



- The control signal supports multiple pads and passes through one or more inverters or buffers.

BSDArchitect places the control boundary scan cell before the common point behind all inverters or buffers in the control signal path within the parent module of the buffer or inverter closest to the signal source. If multiple instances of that parent module exist in the design, the control boundary scan cell is placed at the highest hierarchy between the signal source and the inverter or buffer. [Example 3-9](#) shows the BC_2 cell placed within the parent module of the inverter. If there were multiple instances of *parent_parent_sig_inst*, the BC_2 cell would be at the same position because this location is also the highest hierarchy between the inverter and the signal source.

Example 3-9. Multiple Control Signals With Inverter

Loading Pin Maps

Use the [Load Pin Map](#) command to add boundary scan cells and specify port ordering for your design in a single file. Refer to “[Using a Pin Mapping File](#)” on page 111 for more information on formatting this file.

When mapping tri-state or bidirectional ports in the Internal flow, use the syntax outlined in [Table 3-1](#) on page 51. Use the keyword “void” for the pinID of the control (oe) and input (din) ports because they are not mapped directly to a packaging pin.

When power or ground ports are declared in the pin mapping file by the -pwr or -gnd switch, the tool checks the netlist for the pad cells and makes appropriate connections. Consider the following four cases:

1. The port exists at the top level, and there is no pad cell specified in the pin map file.
If a pad is found in the netlist, the tool issues an error. If no pad is found in the netlist, the tool removes the specified port from the top level, adds linkage in the BSDL file, and adds a generic pad cell to the port.
2. The port exists at the top level, and a pad cell exists in the netlist and is specified in the pin map file.

The pad cell specified in the file must be predefined by the Add Technology Cell command or the tool issues an error. The tool removes the port from the top level and adds linkage in the BSDL file.

3. The port does not exist at the top level, and there is no pad cell specified in the pin map file.

In this case, it is not clear that a power pad is desired. The tool adds linkage in the BSDL file but does not add a pad cell.

4. The port does not exist at the top level, and a pad cell exists in the netlist and is specified in the pin map file.

The pad cell specified in the file must be predefined by the Add Technology Cell command or the tool will issue an error. Linkage will be added in the BSDL file.

The following is an example pin map file for use with the Internal flow:

```
//-----
//Port      Pin   Power/ Cell      Pad Cell      Safe      Inv
//Name      ID    Ground Type      Name          Value
//-----
bit_bidi/oe  void                -cell bc_2_a
bit_bidi     0                -cell bc_7_low      -safe X
bit_diff_o   3 4                -cell bc_1          -safe X
bit_int_o    7                -cell bc_1          -safe X
bit_o        9                -cell bc_1          -safe X
bit_oc_o     10               -cell bc_1          -safe X
bit_tri/oe   void                -cell bc_1
bit_tri      12                -cell bc_1          -safe X
bit_bidi2/oe void                -cell bc_1
bit_bidi2    13                -cell bc_1          -safe X
bit_bidi2/din void                -cell bc_1
bit_diff_i   1 2                -cell bc_1          -safe X
bit_gnd      5      -gnd                -p ground_cell
bit_i        6                -cell bc_1          -safe X
bit_inv_i    8                -cell bc_1          -safe X
bit_pwr      11     -pwr                -p power_cell
tdi          14
tms          15
tck          16
tdo          17
trst         18
```

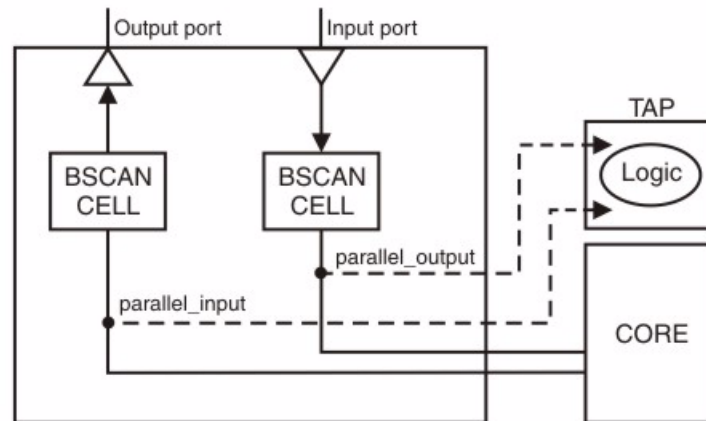
Making Top-Level Port Connections

This section describes the connections made for the following commands:

- Add Port Connection
- Connect Core Chains
- Connect Iscan Chains

For a port connection to a primary output port, the connection is made from the `parallel_input` of the boundary scan cell for the output port as shown in [Figure 3-3](#). Similarly, for a connection to a primary input port, the connection is made from the `parallel_output` of the boundary scan cell for the input port.

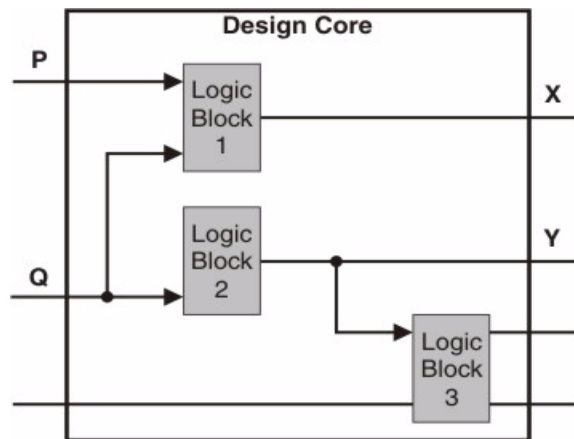
Figure 3-3. Top-Level Port Connections



Placing and Connecting External Registers

The [Add External Register](#) command is unique in that it adds a new module and instance to the top-level design. Consider the design core in [Example 3-10](#):

Example 3-10. Design Core Example



The following commands add an external register to the design and remove the ports connected to the capture and update ports of external register from the top level:

```
BSDA> add external register my_reg 2
// Added external register "my_reg" of length "2".
BSDA> add bscan instruction my_inst -reg_name my_reg
```

```
// Added instruction "my_inst" with opcode "0010".
```

```
BSDA> set external_register interface my_reg -capture X Y -update P Q
```

```
// Note: External register interface for register "my_reg" done.
```

To make the ports visible at the top level, use the [Delete Nontop Port](#) command. For example:

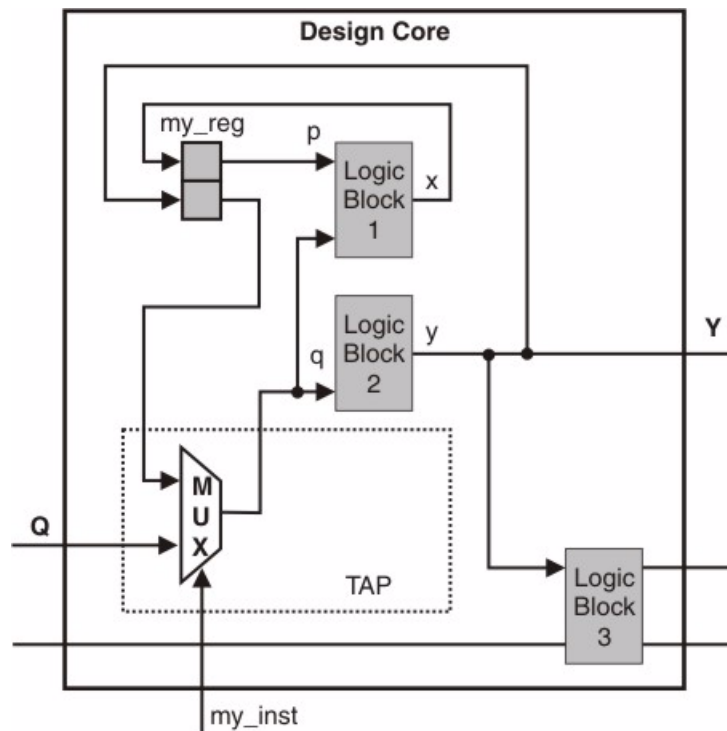
```
BSDA> delete nontop port Q Y
```

```
// Port "Q" will be available at top.
```

```
// Port "Y" will be available at top.
```

The resulting core is illustrated in [Example 3-11](#). Port Q has a multiplexer controlled by the instruction signal, my_inst. This mux is placed within the TAP module. The input port P and the output port X are no longer available at the top level.

Example 3-11. Resulting Design Core



Sharing Functional Pins With TDO

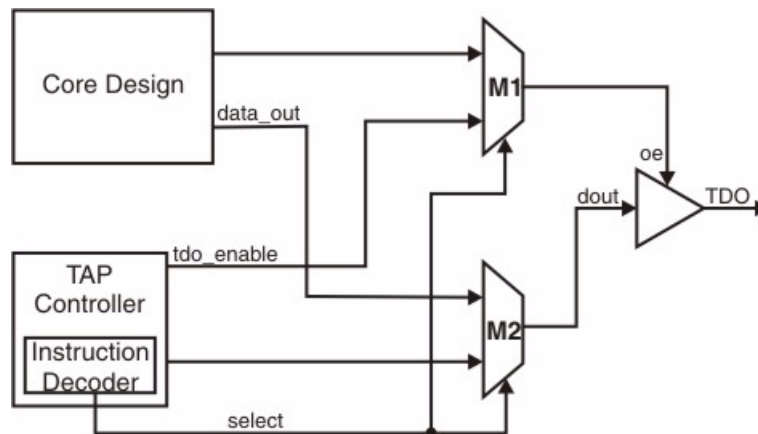
In your design, you may want to avoid adding an additional pin for the TDO port. It is possible to share a functional pin with TDO. The pin functions as designed during normal chip operation and functions as TDO during chip testing.

To set up your design for TDO pin sharing, the netlist must include a top-level output port named TDO with a tristate pad. The ports on the tristate pad must be connected to core signals. BSDArchitect recognizes this as intention for pin sharing and adds the necessary circuitry.

For TDO pin sharing, the tool adds two multiplexers as illustrated in [Example 3-12](#). Multiplexer M1 controls the signal that operates the output enable (oe) of the tri-state or bidirectional pad. Multiplexer M2 controls the signal for the dout port of the pad. The select signal for both M1 and M2 comes from the instruction decoder in the TAP controller. When the instruction register contains an unused opcode (not defined as a JTAG or user instruction), the tool assumes that the chip is in functional mode and the TDO pin is controlled by the core. When the instruction register contains a defined opcode, the tool assumes the chip is in testing mode, and the TDO pin is used accordingly.

There is one problem with TDO pin sharing. According to the IEEE 1149.1 - 2001 standard, the presence of an unused opcode in the instruction register should trigger a bypass of the chip. Because TDO pin sharing uses the unused opcodes to define when the chip is in functional mode, the unused opcodes are not used for bypass. You must define opcodes for bypass use.

Example 3-12. Pin Sharing Example



Uniquification

When a design has I/O pads defined in the netlist, BSDArchitect inserts boundary scan cells at the same hierarchy as the pads. When boundary scan cells are added to an existing module, the module definition is modified. Any change to the module definition will change all instances of that module unless the modified instance is uniquified.

The tool uniquifies the module instance by creating a new unique module to replace the modified module. The instance name remains the same. The process is repeated for parent modules if there are multiple instances of the parent module.

When I/O pads are at the top level, and boundary scan cells are also at the top level, there is no need for Uniquification. This is the case for the External flow and for I/O pads and boundary scan cells added on ports without pads in the Internal flow.

Unsupported Commands

The following commands are not available in the Internal flow:

Add Datacontrol Port	Report Datacontrol Port
Add Port Mapping	Report Port Mapping
Delete Datacontrol Port	Set Configuration Specification
Delete Port Mapping	Set Output Format

Refer to the [Command Dictionary](#) in the *BSDArchitect Reference Manual* for more information on specific commands.

Chapter 4

Boundary Scan Customizations

You can run boundary scan insertion after customizing the setup. Examples of customizations include changing the width of the instruction register, adding optional instructions, setting up for technology-mapped output, and connecting boundary scan circuitry to internal scan circuitry.

Creating Customizations

Customizations can be entered as individual commands in the command line window or command prompt, or they can be entered into a dofile. To issue commands in a dofile, enter the following command at the “BSDA>” prompt:

dofile cust_bscan.do

In this example, the dofile, *cust_bscan.do*, contains the customized setup commands. The following example shows contents of this dofile:

```
//
// Specify the pin ordering using a pin_map file
//
// There are two methods that can be used to specify the port ordering.
// The following displays an example of each.
//
// Here we display the use of the set port order command in a dofile.
//
// * add bscan cell BC_4 CLK
// * add bscan cell BC_3 CLR
//
// define the order of the boundary scan cells
// * set port order CLK CLR D EN1 IN_IO ...
//
// Here we display the use of a pin map file.
//
load pin_map pin_map
// if the load pin map command is used after a dofile that defines the
// port ordering, then the definitions in the pin_map replace those
// made in the dofile
//
// Do not connect a boundary scan cell to pin D
add bscan cell none D

// Change the instruction register length from default (4 bits)
// to 3 bits and add an idcode instruction
set bscan IR_size 3
add bscan instruction idcode -reg_name bscan_id_reg \
    -code 010
```

```
set bscan idcode -manufacturer_name vlsi \  
-manufacturer_id 00010101011 \  
-part_number 0000111100001111 -version 0001  
//  
// Define the directory and file names to use for saving the  
// new boundary scan files.  
setup file naming -root temp_files/my_mod -testbench my_testbench.v  
run  
report bscan status -file bscan_report -replace -all  
save bscan -replace  
exit
```

Technology Mapping

Technology mapping provides support for replacing generic cells by equivalent technology-specific cells. Technology-mapped cells can be boundary scan cells, I/O pad cells, and TAP controllers.

Technology mapping support is cell specific rather than instance specific. Consequently, the attributes specified for a particular technology cell affect all instances of that cell.

The technology-mapped cells may be defined using library files or using the [Add Technology Cell](#) command.

In some ASIC libraries, the boundary scan logic is included in PAD cells. In other cases, ASIC libraries are separate from pad cells. In both cases, BSDArchitect instantiates boundary scan cells within the BSR instance entity.

When using technology mapping in your design, refer to “[Understanding BSDArchitect Output Files](#)” on page 125 to write out proper files.

Libraries

Generic library components corresponding to clock_gating options are presented in the following directory within the Mentor Graphics Tessent software tree:

<Tessent_Tree_Path>/lib/tools/jtag_libs/src directory

You use these components to create a generic RTL level description of the boundary scan circuitry. Once you define a cell in a library file, you can add it to the design using the [Set Technology Mapping](#) command.

Technology-specific library files contain information about the cells and their interconnections. BSDArchitect accepts libraries in two formats. If you are using the Internal flow, your library must be a Verilog file. The following formats define the cells:

- VHDL — You map the vendor components to generic components using the ‘attribute’ statement.
- Verilog — You map the vendor components to generic components using the ‘specparam’ statement.

[Table 4-1](#) shows how the library attributes translate for use with the Add Technology Cell command.

Table 4-1. Library Attributes vs. Command Line Arguments

Library User-Defined Attributes	Add Technology Cell Arguments
mgc_dft_cell_type	cell_type
mgc_dft_pin_type	-Connect
mgc_dft_connect	-Connect
mgc_dft_bscell_mode	-BScell_mode
mgc_dft_disable_result	-Disable_result

Technology Mapping Syntax

The following sections describe BSDArchitect syntax requirements for technology mapping using both library files and the Add Technology Cell command.

Specifying Cell Type Information

To specify a boundary scan cell type, use the ‘mgc_bs_bsr_type’ or ‘mgc_dft_cell_type’ attribute. For all other types of cells, use the ‘mgc_dft_cell_type’ attribute. The cell type attribute is used inside the description of the vendor cell.

If you are defining the cell using the Add Technology Cell command, specify the cell type as the second command argument following the new cell name. For more information, refer to the [Add Technology Cell](#) command in the *BSDArchitect Reference Manual*.

Specifying Port Mapping Information

To specify the pin mapping information for boundary scan cells, use the ‘mgc_bs_pin_map’ or ‘mgc_dft_pin_type’ attribute. To specify port mapping for all other cells, use the ‘mgc_dft_pin_type’ attribute. The port mapping attribute is used inside the description of the vendor cell.

If you are defining the cell using the Add Technology Cell command, specify the port mapping information following the -Connect switch. Refer to the [Generic Port Names](#) table for the Add Technology Cell command for specific port names for all cell types.

You must provide appropriate pin mapping information for all pins in the generic cell and the vendor cell, otherwise BSDArchitect generates an error message. The [Generic Port Names](#) table shows which ports you must define and which ports are optional.

The following three sections show the pin mapping information for possible cases using different methods:

Using the ‘mgc_bs_pin_map’ Attribute

The following list illustrates the six different mappings for a vendor pin ‘vpin’ specified using the ‘mgc_bs_pin_map’ attribute:

1. **vpin/lpin** — Maps vpin on the vendor cell to lpin on the generic cell.
2. **vpin/ground** — Maps vpin on the vendor cell to the ground line.
3. **vpin/power** — Maps vpin on the vendor cell to the power line.
4. **delete/lpin** — Indicates that lpin on the generic cell does not exist on the vendor cell.
5. **vpin/open** — Indicates that vpin on the vendor cell is left unconnected.
6. **vpin/vpin_n** — Maps vpin on the vendor cell to the signal vpin_n in the synthesized circuitry. If the signal vpin_n is not already available in the instance containing the mapped cell, BSDArchitect creates a local signal with name vpin_n.

The following snippet specifies the port mapping information for a BC_1 cell vendor_bc1 using the ‘mgc_bs_pin_map’ attribute:

```
attribute mgc_bs_pin_map of vendor_bc1 : entity is
    "pi/parallel_input, si/serial_input, po/parallel_output,
    so/serial_output, mode_1/mode_1_v, mode_2/mode_2_v, delete/mode,
    shdr/shiftdr, ckdr/clockdr, updr/updatedr, vcc/power, gnd/ground,
    opn_s/open";
```

Using the ‘mgc_dft_cell_type’ Attributes

The following list illustrates the same six different mappings for a vendor pin ‘vpin’ specified using the ‘mgc_dft_cell_type’ attributes:

1. attribute mgc_dft_pin_type of vpin: signal is “lpin”;
2. attribute mgc_dft_pin_type of vpin: signal is “gnd”;
3. attribute mgc_dft_pin_type of vpin: signal is “vdd”;
4. attribute mgc_dft_nonexistent_signals of vendor_cell: entity is “lpin”;
5. attribute mgc_dft_pin_type of vpin: signal is “open”;
6. attribute mgc_dft_connect of vpin: signal is “vpin_n”;

The following snippet specifies the port mapping information for a BC_1 cell vendor_bc1 using the 'mgc_dft_cell_type' attributes:

```
attribute mgc_dft_pin_type of pi : signal is "parallel_input";
attribute mgc_dft_pin_type of si : signal is "serial_input";
attribute mgc_dft_pin_type of po : signal is "parallel_output";
attribute mgc_dft_pin_type of so : signal is "serial_output";
attribute mgc_dft_pin_type of shdr : signal is "shiftdr";
attribute mgc_dft_pin_type of ckdr : signal is "clockdr";
attribute mgc_dft_pin_type of updr : signal is "updatedr";
attribute mgc_dft_pin_type of vcc : signal is "vdd";
attribute mgc_dft_pin_type of gnd : signal is "gnd";
attribute mgc_dft_pin_type of opn_s : signal is "open";
attribute mgc_dft_nonexistent_signals of vendor_bc1 : entity is "mode";
attribute mgc_dft_connect of mode_1 : signal is "mode_1_v";
attribute mgc_dft_connect of mode_2 : signal is "mode_2_v";
```

Using the Add Technology Cell Command

The following list illustrates the same six different mappings, and one additional mapping for a vendor pin 'vpin' specified using the switches for the Add Technology Cell command:

1. **-connect vpin/lpin** — Maps vpin on the vendor cell to lpin on the generic cell.
2. **-connect vpin/gnd** — Maps vpin on the vendor cell to the ground line.
3. **-connect vpin/pwr** — Maps vpin on the vendor cell to the power line.
4. **-nonexistent lpin** — Indicates that lpin on the generic cell does not exist on the vendor cell.
5. **-connect vpin/open** — Indicates that vpin on the vendor cell is left unconnected.
6. **-connect vpin/vpin_n** — Maps vpin on the vendor cell to the signal vpin_n in the synthesized circuitry.
7. **-connect vpin/"path/my_net"** — (for Internal flow only) Maps vpin on the vendor cell to the net my_net. Net names must be enclosed in double quotes.

The following snippet specifies the port mapping information for a BC_1 cell vendor_bc1 using the Add Technology Cell command:

```
-connect pi/parallel_input si/serial_input po/parallel_output so/serial_output \
    mode_1/mode_1_v mode_2/mode_2_v shdr/shiftdr ckdr/clockdr updr/updatedr \
    vcc/power gnd/ground opn_s/open \
-nonexistent mode
```

Design Rules Checking for Technology Mapping

BSDArchitect conducts internal design rules checking to verify that the technology mapping inputs are consistent and the output generated conforms to the IEEE 1149.1 standard. The tools

reports failure of these checks as errors and does not add boundary scan circuitry. In some libraries, however, it may not be possible to map the technology dependent component types to the generic cell types used in BSDArchitect. In such cases, use the Set Drc Handling command to override the BSDArchitect design checks. This command uses the following syntax:

SET DRc Handling **Error** | **Warning** | **Ignore**

For more information, refer to the [Set Drc Handling](#) command in the *BSDArchitect Reference Manual*.

Defining Boundary Scan Cells

The following are valid boundary scan cell types available for technology mapping in BSDArchitect:

BC_1 Cell	BC_3 Cell	BC_8 Cell
BC_2 Cell	BC_4 Cell	BC_9 Cell
BC_2_A Cell	BC_5 Cell	BC_10 Cell
BC_2_A_EXT Cell	BC_7 Cell	BC_X Cell
BC_2_B Cell	BC_7_LOW Cell	
AC_1 Cell	AC_7_LOW Cell	AC_10 Cell
AC_2 Cell	AC_8 Cell	AC_SelX Cell
AC_7 Cell	AC_9 Cell	AC_SelU Cell

After defining a boundary scan cell, the cell may be added using the Add Bscan Cell or Set Bscan Cell command.

Boundary Scan Rules Checking

When you map a technology-specific boundary scan cell to a generic cell type, BSDArchitect applies the rules checking for the generic cell to the technology-specific cell as well. These checks include the following:

- BC_2, BC_3 and BC_4 cells can be used only with input ports.
- BC_7/BC_2_A can be used only for bidi/bidi control (BC_7 is a combined input/output cell).
- When a BC_7 is used for a bidi port, the input port cannot have a bscell associated with it.

- When a BC_7 is used for a bidi port, the enable port must have a BC_2_A cell.
- External control signals must have a BC_5 for tri-state control and a BC_2_A_EXT for bidirectional control ports.
- BC_8 is used only for an open-collector bidirectional port.
- BC_9 and BC_10 cells can be used only on output, tri-state, and bidirectional output ports. These cells can only be used with simple output ports in the Internal flow.
- AC_1, AC_2, AC_7, AC_8, AC_9, and AC_10 cells can be used only for output or output-type ports (bidirectional output and tri-state output).
- When an AC_7 cell is used for a bidi port, the input port cannot have a bscell associated with it.
- When an AC_7 cell is used for a bidi port, the controls still will have BC_2 cells.
- AC_SelX and AC_SelU are internal cells and are not related to specific ports.

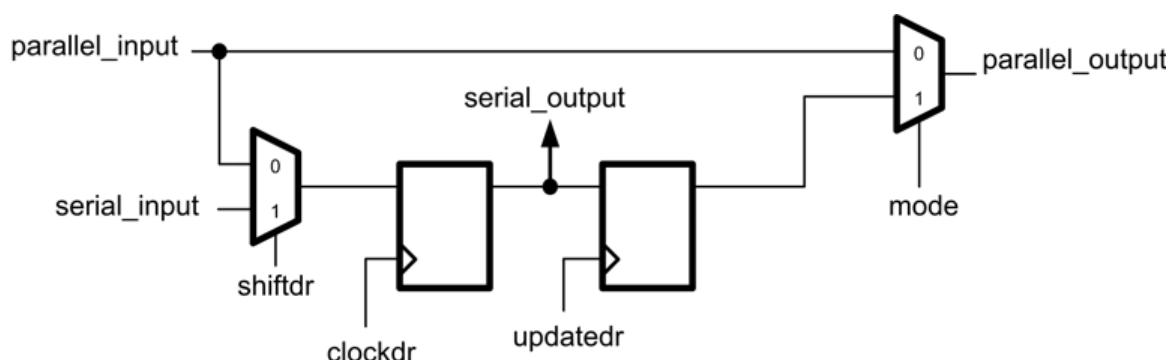
Specifying the Boundary Scan Cell Mode

The IEEE 1149.1 standard allows two different modes for boundary scan cells: **observe** and **both**. Boundary scan cells of observe-only mode can only be used with input ports. Of the standard boundary scan cell types, BC_4 is of mode **observe** and all others are of mode **both**.

When you map a technology boundary scan cell to a standard cell type, the mode information is inherited from the standard cell type. For BC_X cells, however, you can specify a cell mode by using the 'mgc_dft_bscell_mode' attribute, whose values can be **observe** or **both**. The default is **both**.

BC_1 Cell

Figure 4-1. Generic BC_1 Cell



The following example is a Verilog BC_1 definition using the 'mgc_dft_cell_type' attribute:

```
module ctillike_bc_1 (v_shiftdr, v_clockdr, v_updatedr,
    v_parallel_input, v_extra_pin,v_serial_input, v_mode,
    v_serial_output, v_parallel_output);

input    v_shiftdr, v_clockdr, v_updatedr, v_parallel_input,
    v_serial_input, v_mode, v_extra_pin;

output v_serial_output, v_parallel_output;

specify
    specparam mgc_dft_cell_type = "bc_1";
    specparam mgc_dft_connect$v_extra_pin = "extra_pin_forbc1";
    specparam mgc_dft_pin_type$v_clockdr = "clockdr";
    specparam mgc_dft_pin_type$v_shiftdr = "shiftdr";
    specparam mgc_dft_pin_type$v_updatedr = "updatedr";
    specparam mgc_dft_pin_type$v_mode = "mode";
    specparam mgc_dft_pin_type$v_parallel_input = "parallel_input";
    specparam mgc_dft_pin_type$v_parallel_output = "parallel_output";
    specparam mgc_dft_pin_type$v_serial_input = "serial_input";
    specparam mgc_dft_pin_type$v_serial_output = "serial_output";
endspecify
endmodule
```

The following example is a Verilog definition using the ‘mgc_bs_bsr_type’ attribute:

```
module old_syntax_bc_1 ( v_shiftdr, v_clockdr, v_updatedr,
    v_parallel_input, v_extra_pin, v_serial_input, v_mode,
    v_serial_output, v_parallel_output );

input v_shiftdr, v_clockdr, v_updatedr, v_parallel_input,
    v_serial_input, v_mode, v_extra_pin;

output v_serial_output, v_parallel_output;

specify
    specparam mgc_bs_bsr_type = "bc_1";
    specparam mgc_bs_pin_map = "
        v_extra_pin/extra_pin_forbc1, v_clockdr/clockdr,
        v_shiftdr/shiftdr, v_updatedr/updatedr, v_mode/mode,
        v_parallel_input/parallel_input,
        v_parallel_output/parallel_output, v_serial_input/serial_input ,
        v_serial_output/serial_output";
endspecify
endmodule
```

The following VHDL example maps the generic cell bc_1 to the vendor cell vendor_bc1:

```
-- description of generic bc_1 (clock gating on, asynchronous -- version)
entity bc_1 is
    port(shiftdr,clockdr,updatedr,parallel_input,serial_input,
        mode : in std_ulogic;
        serial_output,parallel_output : out std_ulogic);
end bc_1;

-- description of vendor cell
```

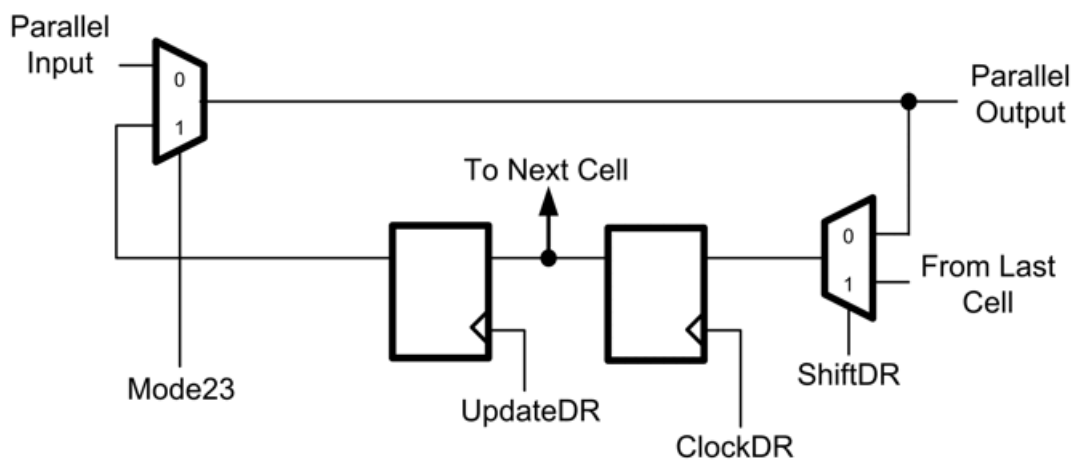
```

entity vendor_bcl is
  port(pi, si : in std_ulogic;
        po, so : out std_ulogic;
        mode_1, mode_2 : in std_ulogic;
        shdr, updr, ckdr : in std_ulogic;
        vcc, gnd : in std_ulogic;
        opn_s : in std_ulogic);
  attribute mgc_bs_bsr_type : string;
  attribute mgc_bs_pin_map : string;
  attribute mgc_bs_bsr_type of vendor_bcl : entity is "bc_1";
  attribute mgc_bs_pin_map of vendor_bcl : entity is
    "pi/parallel_input, si/serial_input, po/parallel_output,
    so/serial_output, mode_1/mode_1_v, mode_2/mode_2_v, delete/mode,
    shdr/shiftdr, ckdr/clockdr, updr/updatedr, vcc/power, gnd/ground,
    opn_s/open";
end vendor_bcl;

```

BC_2 Cell

Figure 4-2. Generic BC_2 Cell



The following example is a Verilog BC_2 definition using the 'mgc_dft_cell_type' attribute:

```

module my_bc_2 (my_shiftdr, my_clockdr, my_updatedr,
    my_parallel_input, my_serial_input, my_mode,
    my_serial_output, my_parallel_output);

input    my_shiftdr, my_clockdr, my_updatedr, my_parallel_input,
    my_serial_input, my_mode;

output my_serial_output, my_parallel_output;

specify
    specparam mgc_dft_cell_type = "bc_2";
    specparam mgc_dft_pin_type$my_clockdr = "clockdr";
    specparam mgc_dft_pin_type$my_shiftdr = "shiftdr";
    specparam mgc_dft_pin_type$my_updatedr = "updatedr";
    specparam mgc_dft_pin_type$my_mode = "mode23";
    specparam mgc_dft_pin_type$my_parallel_input = "parallel_input";
    specparam mgc_dft_pin_type$my_parallel_output = "parallel_output";
    specparam mgc_dft_pin_type$my_serial_input = "serial_input";
    specparam mgc_dft_pin_type$my_serial_output = "serial_output";
endspecify
endmodule

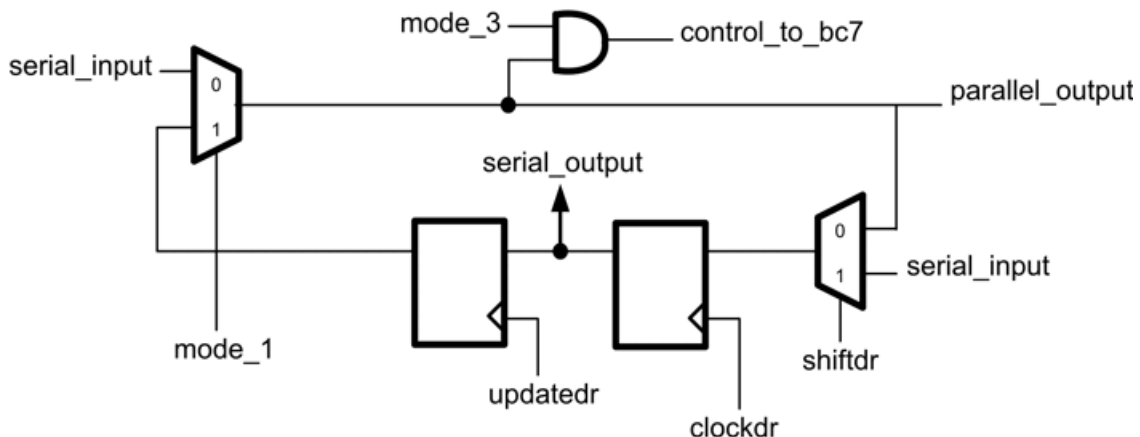
```

The following example is a BC_2 cell definition using the Add Technology Cell command:

```
add technology cell my_bc_2 bc_2
    -inputs my_shiftdr my_clockdr my_updatedr my_parallel_input my_serial_input \
    my_mode \
    -outputs my_serial_output my_parallel_output \
    -connect my_shiftdr/shiftdr my_clockdr/clockdr my_updatedr/updatedr \
    my_parallel_input/parallel_input my_serial_input/serial_input my_mode/mode23 \
    my_serial_output/serial_output my_parallel_output/parallel_output
```

BC_2_A Cell

Figure 4-3. Generic BC_2_A Cell



The following example is a Verilog BC_2_A definition using the ‘mgc_dft_cell_type’ attribute:

```
module my_bc_2_a (my_shiftdr, my_clockdr, my_updatedr, my_parallel_in,
    my_serial_in, my_model, my_mode2, my_serial_out, my_parallel_out,
    my_control_signal);

input my_shiftdr, my_clockdr, my_updatedr, my_parallel_in, my_serial_in,
    my_model, my_mode2;

output my_serial_out, my_parallel_out, my_control_signal;

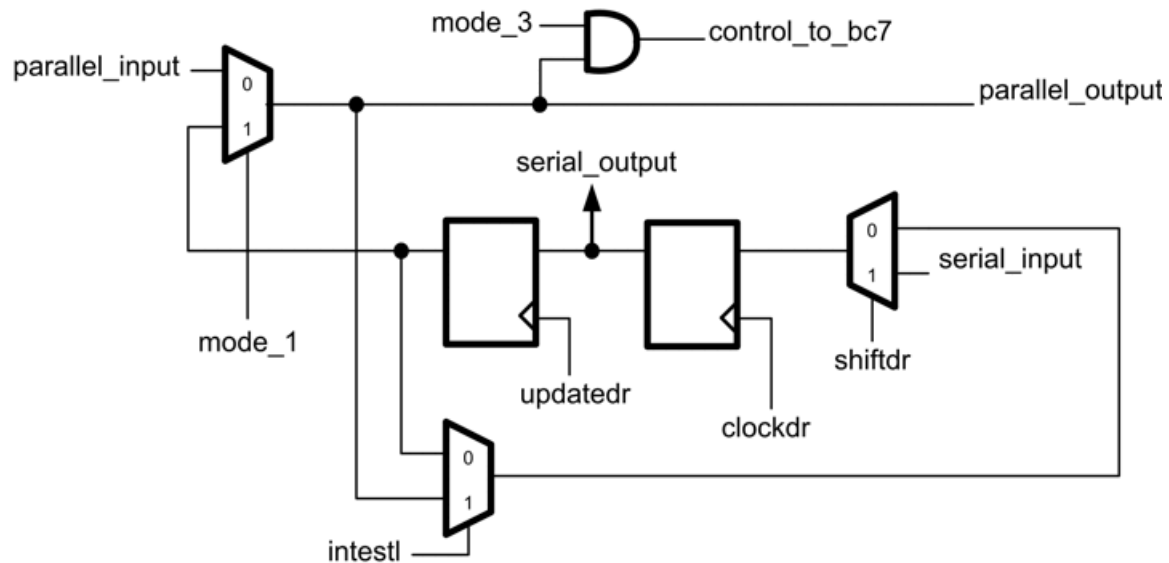
specify
    specparam mgc_dft_cell_type = "bc_2_a";
    specparam mgc_dft_pin_type$my_clockdr = "clockdr";
    specparam mgc_dft_pin_type$my_shiftdr = "shiftdr";
    specparam mgc_dft_pin_type$my_updatedr = "updatedr";
    specparam mgc_dft_pin_type$my_model = "mode_1";
    specparam mgc_dft_pin_type$my_mode2 = "mode_3";
    specparam mgc_dft_pin_type$my_parallel_in = "parallel_input";
    specparam mgc_dft_pin_type$my_parallel_out = "parallel_output";
    specparam mgc_dft_pin_type$my_serial_in = "serial_input";
    specparam mgc_dft_pin_type$my_serial_out = "serial_output";
    specparam mgc_dft_pin_type$my_control_signal = "control_to_bc7";
endspecify
endmodule
```

The following example is a BC_2_A cell definition using the Add Technology Cell command:

```
add technology cell my_bc_2_a bc_2_a
-inputs my_shiftdr my_clockdr my_updatedr my_parallel_in my_serial_in \
my_model my_mode2 \
-outputs my_serial_out my_parallel_out \
-connect my_shiftdr/shiftdr my_clockdr/clockdr my_updatedr/updatedr \
my_parallel_in/parallel_input my_serial_in/serial_input my_model/mode_1 \
my_mode2/mode_3 my_serial_out/serial_output my_parallel_out/parallel_output \
my_control_signal/control_to_bc7
```

BC_2_A_EXT Cell

Figure 4-4. Generic BC_2_A_EXT Cell



The following example is a Verilog BC_2_A_EXT definition using the ‘mgc_dft_cell_type’ attribute:

```

module my_bc_2_a_ext (my_shiftdr, my_clockdr, my_updatedr, my_intest1,
    my_parallel_in, my_serial_in, my_model1, my_mode2, my_serial_out,
    my_parallel_out, my_control_signal);

input my_shiftdr, my_clockdr, my_updatedr, my_intest1, my_parallel_in,
    my_serial_in, my_model1, my_mode2;

output my_serial_out, my_parallel_out, my_control_signal;

specify
    specparam mgc_dft_cell_type = "bc_2_a_ext";
    specparam mgc_dft_pin_type$my_clockdr = "clockdr";
    specparam mgc_dft_pin_type$my_shiftdr = "shiftdr";
    specparam mgc_dft_pin_type$my_updatedr = "updatedr";
    specparam mgc_dft_pin_type$my_intest1 = "intest1";
    specparam mgc_dft_pin_type$my_model1 = "mode_1";
    specparam mgc_dft_pin_type$my_mode2 = "mode_3";
    specparam mgc_dft_pin_type$my_parallel_in = "parallel_input";
    specparam mgc_dft_pin_type$my_parallel_out = "parallel_output";
    specparam mgc_dft_pin_type$my_serial_in = "serial_input";
    specparam mgc_dft_pin_type$my_serial_out = "serial_output";
    specparam mgc_dft_pin_type$my_control_signal = "control_to_bc7";
endspecify
endmodule

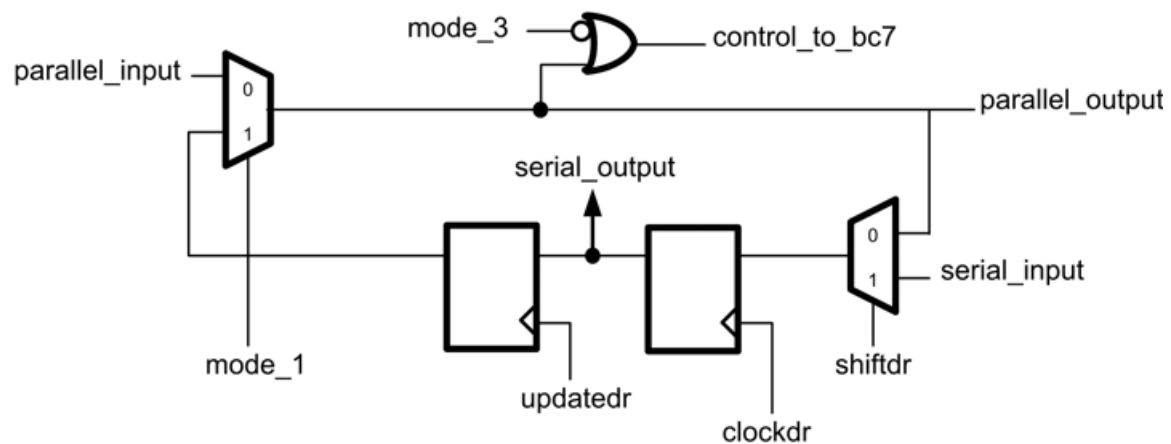
```

The following example is a BC_2_A_EXT cell definition using the Add Technology Cell command:

```
add technology cell my_bc_2_a_ext bc_2_a_ext
-inputs my_shiftdr my_clockdr my_updatedr my_intestl my_parallel_in my_serial_in \
my_mode1 my_mode2 \
-outputs my_serial_out my_parallel_out my_control_signal \
-connect my_shiftdr/shiftdr my_clockdr/clockdr my_updatedr/updatedr \
my_intestl/intestl my_parallel_in/parallel_input my_serial_in/serial_input \
my_mode1/mode_1 my_mode2/mode_3 my_serial_out/serial_output \
my_parallel_out/parallel_output my_control_signal/control_to_bc7
```

BC_2_B Cell

Figure 4-5. Generic BC_2_B Cell



The following example is a Verilog BC_2_B definition using the 'mgc_dft_cell_type' attribute:

```
module my_bc_2_b (my_shiftdr, my_clockdr, my_updatedr, my_parallel_in,
    my_serial_in, my_model, my_mode2, my_serial_out, my_parallel_out,
    my_control_signal);

input my_shiftdr, my_clockdr, my_updatedr, my_parallel_in, my_serial_in,
    my_model, my_mode2;

output my_serial_out, my_parallel_out, my_control_signal;

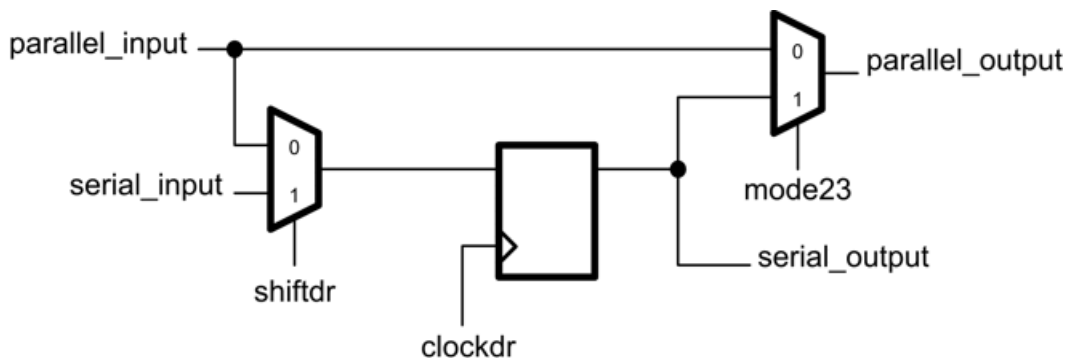
specify
    specparam mgc_dft_cell_type = "bc_2_b";
    specparam mgc_dft_pin_type$my_clockdr = "clockdr";
    specparam mgc_dft_pin_type$my_shiftdr = "shiftdr";
    specparam mgc_dft_pin_type$my_updatedr = "updatedr";
    specparam mgc_dft_pin_type$my_model = "mode_1";
    specparam mgc_dft_pin_type$my_mode2 = "mode_3";
    specparam mgc_dft_pin_type$my_parallel_in = "parallel_input";
    specparam mgc_dft_pin_type$my_parallel_out = "parallel_output";
    specparam mgc_dft_pin_type$my_serial_in = "serial_input";
    specparam mgc_dft_pin_type$my_serial_out = "serial_output";
    specparam mgc_dft_pin_type$my_control_signal = "control_to_bc7";
endspecify
endmodule
```

The following example is a BC_2_B cell definition using the Add Technology Cell command:

```
add technology cell my_bc_2_b bc_2_b
-inputs my_shiftdr my_clockdr my_updatedr my_parallel_in my_serial_in \
my_model1 my_mode2 \
-outputs my_serial_out my_parallel_out \
-connect my_shiftdr/shiftdr my_clockdr/clockdr my_updatedr/updatedr \
my_parallel_in/parallel_input my_serial_in/serial_input my_model1/mode_1 \
my_mode2/mode_3 my_serial_out/serial_output my_parallel_out/parallel_output \
my_control_signal/control_to_bc7
```

BC_3 Cell

Figure 4-6. Generic BC_3 Cell



The following example is a Verilog BC_3 definition using the 'mgc_dft_cell_type' attribute:

```

module my_bc_3 (my_shiftdr, my_clockdr, my_parallel_in, my_serial_in,
               my_mode, my_serial_out, my_parallel_out);

input my_shiftdr, my_clockdr, my_parallel_in, my_serial_in, my_mode;

output my_serial_out, my_parallel_out;

specify
  specparam mgc_dft_cell_type = "bc_3";
  specparam mgc_dft_pin_type$my_clockdr = "clockdr";
  specparam mgc_dft_pin_type$my_shiftdr = "shiftdr";
  specparam mgc_dft_pin_type$my_model = "mode23";
  specparam mgc_dft_pin_type$my_parallel_in = "parallel_input";
  specparam mgc_dft_pin_type$my_parallel_out = "parallel_output";
  specparam mgc_dft_pin_type$my_serial_in = "serial_input";
  specparam mgc_dft_pin_type$my_serial_out = "serial_output";
endspecify
endmodule

```

The following example is a BC_3 cell definition using the Add Technology Cell command:

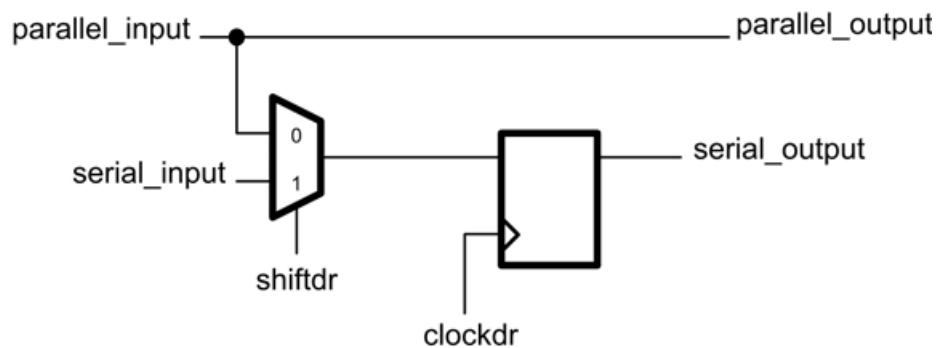
```

add technology cell my_bc_3 bc_3
-inputs my_shiftdr my_clockdr my_parallel_in my_serial_in my_mode \
-outputs my_serial_out my_parallel_out \
-connect my_shiftdr/shiftdr my_clockdr/clockdr my_parallel_in/parallel_input \
my_serial_in/serial_input my_mode/mode23 my_serial_out/serial_output \
my_parallel_out/parallel_output

```

BC_4 Cell

Figure 4-7. Generic BC_4 Cell



The following example is a Verilog BC_4 definition using the ‘mgc_dft_cell_type’ attribute:

```

module my_bc_4 (my_shiftdr, my_clockdr, my_parallel_in, my_serial_in,
    my_serial_out, my_parallel_out);

input my_shiftdr, my_clockdr, my_parallel_in, my_serial_in;

output my_serial_out, my_parallel_out;

specify
    specparam mgc_dft_cell_type = "bc_4";
    specparam mgc_dft_pin_type$my_clockdr = "clockdr";
    specparam mgc_dft_pin_type$my_shiftdr = "shiftdr";
    specparam mgc_dft_pin_type$my_parallel_in = "parallel_input";
    specparam mgc_dft_pin_type$my_parallel_out = "parallel_output";
    specparam mgc_dft_pin_type$my_serial_in = "serial_input";
    specparam mgc_dft_pin_type$my_serial_out = "serial_output";
endspecify
endmodule

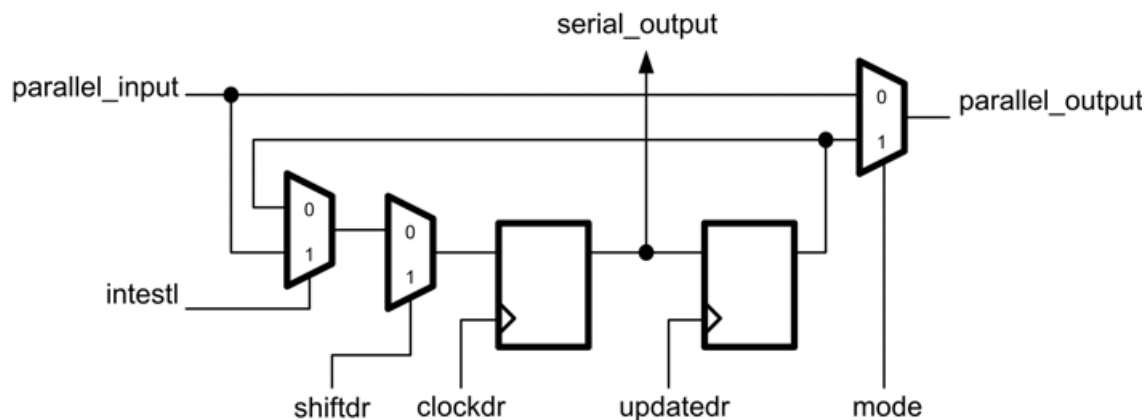
```

The following example is a BC_4 cell definition using the Add Technology Cell command:

```
add technology cell my_bc_4 bc_4
  -inputs my_shiftdr my_clockdr my_parallel_in my_serial_in \
  -outputs my_serial_out my_parallel_out \
  -connect my_shiftdr/shiftdr my_clockdr/clockdr my_parallel_in/parallel_input \
  my_serial_in/serial_input my_serial_out/serial_output \
  my_parallel_out/parallel_output
```

BC_5 Cell

Figure 4-8. Generic BC_5 Cell



The following example is a Verilog BC_5 definition using the 'mgc_dft_cell_type' attribute:

```
module my_bc_5 (my_shiftdr, my_clockdr, my_updatedr, my_parallel_in,
               my_serial_in, my_mode, my_intestl, my_serial_out, my_parallel_out);

input my_shiftdr, my_clockdr, my_updatedr, my_parallel_in, my_serial_in,
      my_mode, my_intestl;

output my_serial_out, my_parallel_out;

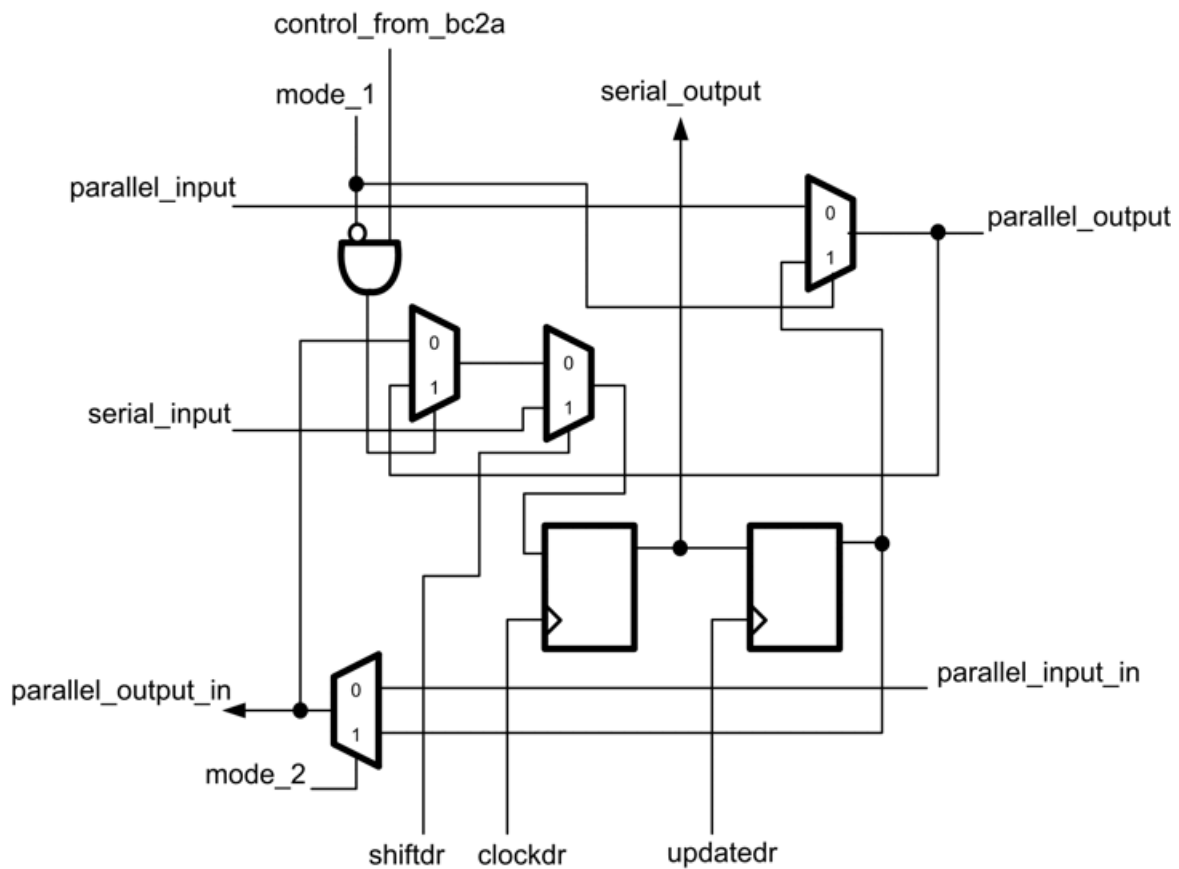
specify
  specparam mgc_dft_cell_type = "bc_5";
  specparam mgc_dft_pin_type$my_clockdr = "clockdr";
  specparam mgc_dft_pin_type$my_shiftdr = "shiftdr";
  specparam mgc_dft_pin_type$my_updatedr = "updatedr";
  specparam mgc_dft_pin_type$my_intestl = "intestl";
  specparam mgc_dft_pin_type$my_mode = "mode";
  specparam mgc_dft_pin_type$my_parallel_in = "parallel_input";
  specparam mgc_dft_pin_type$my_parallel_out = "parallel_output";
  specparam mgc_dft_pin_type$my_serial_in = "serial_input";
  specparam mgc_dft_pin_type$my_serial_out = "serial_output";
endspecify
endmodule
```

The following example is a BC_5 cell definition using the Add Technology Cell command:

```
add technology cell my_bc_5 bc_5
-inputs my_shiftdr my_clockdr my_updatedr my_intestl my_parallel_in my_serial_in \
my_mode \
-outputs my_serial_out my_parallel_out \
-connect my_shiftdr/shiftdr my_clockdr/clockdr my_updatedr/updatedr \
my_parallel_in/parallel_input my_serial_in/serial_input my_intestl/intestl \
my_mode/mode my_serial_out/serial_output my_parallel_out/parallel_output
```

BC_7 Cell

Figure 4-9. Generic BC_7 Cell



The following example is a Verilog BC_7 definition using the ‘mgc_dft_cell_type’ attribute:

```

module my_bc_7 (my_shiftdr, my_clockdr, my_updatedr, my_parallel_in,
    my_parallel_in_in, my_serial_in, my_model, my_mode2,
    my_control_signal, my_serial_out, my_parallel_out,
    my_parallel_out_in);

input my_shiftdr, my_clockdr, my_updatedr, my_parallel_in, my_serial_in,
    my_parallel_in_in, my_model, my_mode2, my_control_signal;

output my_serial_out, my_parallel_out, my_parallel_out_in;

specify
    specparam mgc_dft_cell_type = "bc_7";
    specparam mgc_dft_pin_type$my_clockdr = "clockdr";
    specparam mgc_dft_pin_type$my_shiftdr = "shiftdr";
    specparam mgc_dft_pin_type$my_updatedr = "updatedr";
    specparam mgc_dft_pin_type$my_model = "mode_1";
    specparam mgc_dft_pin_type$my_mode2 = "mode_2";
    specparam mgc_dft_pin_type$my_parallel_in = "parallel_input";
    specparam mgc_dft_pin_type$my_parallel_out = "parallel_output";
    specparam mgc_dft_pin_type$my_parallel_in_in = "parallel_input_in";
    specparam mgc_dft_pin_type$my_parallel_out_in = "parallel_output_in";
    specparam mgc_dft_pin_type$my_serial_in = "serial_input";
    specparam mgc_dft_pin_type$my_serial_out = "serial_output";
    specparam mgc_dft_pin_type$my_control_signal = "control_from_bc2a";
endspecify
endmodule

```

The following example is a BC_7 cell definition using the Add Technology Cell command:

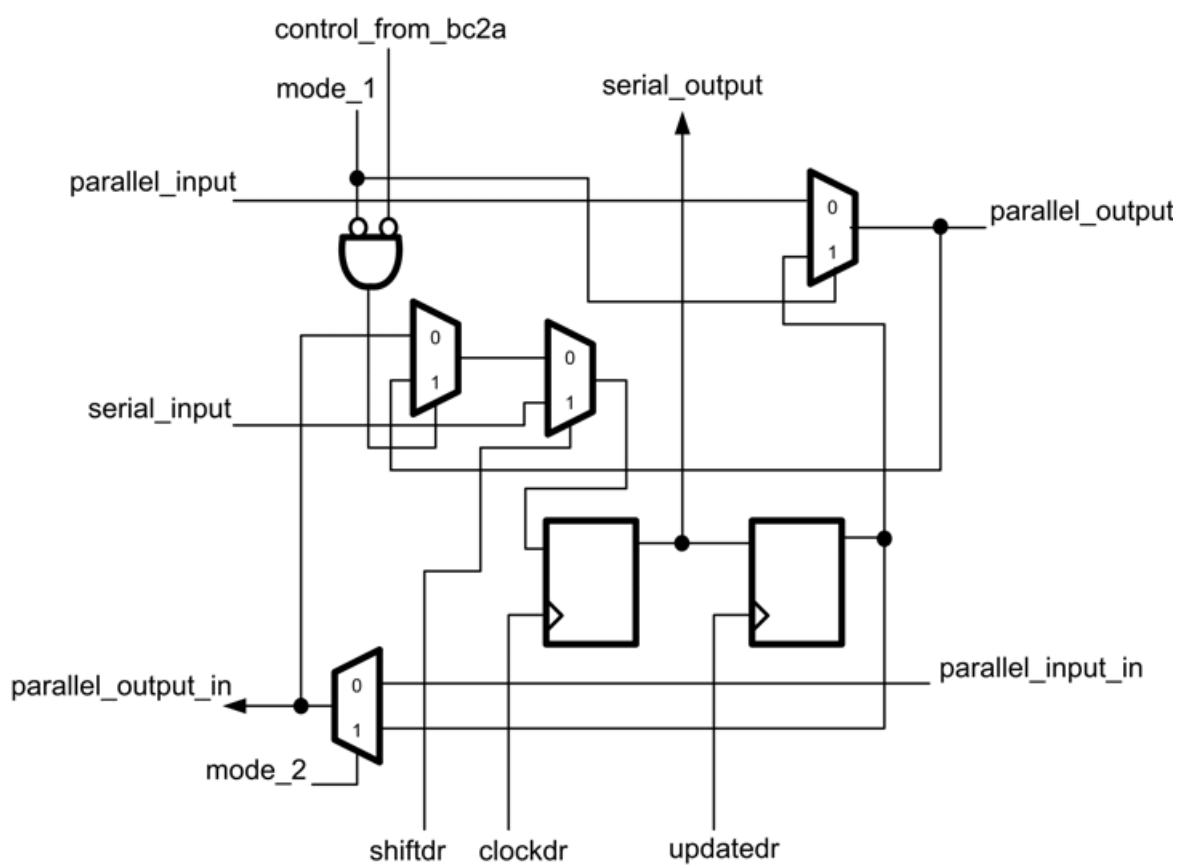
```

add technology cell my_bc_7 bc_7
    -inputs my_shiftdr my_clockdr my_updatedr my_parallel_in my_serial_in \
    my_parallel_in_in my_model my_mode2 my_control_signal\
    -outputs my_serial_out my_parallel_out my_parallel_out_in\
    -connect my_shiftdr/shiftdr my_clockdr/clockdr my_updatedr/updatedr \
    my_parallel_in/parallel_input my_parallel_in_in/parallel_input_in \
    my_serial_in/serial_input my_model/mode_1 my_mode2/mode_2 \
    my_serial_out/serial_output my_parallel_out/parallel_output \
    my_parallel_out_in/parallel_output_in my_control_signal/control_from_bc2a

```

BC_7_LOW Cell

Figure 4-10. Generic BC_7_LOW Cell



The following example is a Verilog BC_7_LOW definition using the ‘mgc_dft_cell_type’ attribute:

```
module my_bc_7_low (my_shiftdr, my_clockdr, my_updatedr, my_parallel_in,
    my_parallel_in_in, my_serial_in, my_model, my_mode2,
    my_control_signal, my_serial_out, my_parallel_out,
    my_parallel_out_in);

input my_shiftdr, my_clockdr, my_updatedr, my_parallel_in, my_serial_in,
    my_parallel_in_in, my_model, my_mode2, my_control_signal;

output my_serial_out, my_parallel_out, my_parallel_out_in;

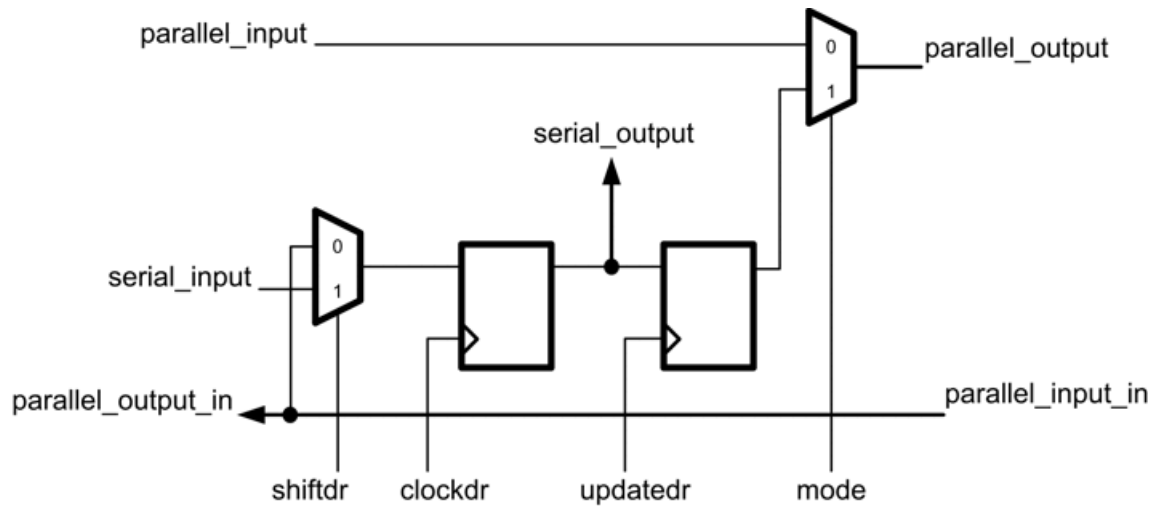
specify
    specparam mgc_dft_cell_type = "bc_7_low";
    specparam mgc_dft_pin_type$my_clockdr = "clockdr";
    specparam mgc_dft_pin_type$my_shiftdr = "shiftdr";
    specparam mgc_dft_pin_type$my_updatedr = "updatedr";
    specparam mgc_dft_pin_type$my_model = "mode_1";
    specparam mgc_dft_pin_type$my_mode2 = "mode_2";
    specparam mgc_dft_pin_type$my_parallel_in = "parallel_input";
    specparam mgc_dft_pin_type$my_parallel_out = "parallel_output";
    specparam mgc_dft_pin_type$my_parallel_in_in = "parallel_input_in";
    specparam mgc_dft_pin_type$my_parallel_out_in = "parallel_output_in";
    specparam mgc_dft_pin_type$my_serial_in = "serial_input";
    specparam mgc_dft_pin_type$my_serial_out = "serial_output";
    specparam mgc_dft_pin_type$my_control_signal = "control_from_bc2a";
endspecify
endmodule
```

The following example is a BC_7_LOW cell definition using the Add Technology Cell command:

```
add technology cell my_bc_7_low bc_7_low
    -inputs my_shiftdr my_clockdr my_updatedr my_parallel_in my_serial_in \
    my_parallel_in_in my_mode1 my_mode2 my_control_signal \
    -outputs my_serial_out my_parallel_out my_parallel_out_in \
    -connect my_shiftdr/shiftdr my_clockdr/clockdr my_updatedr/updatedr \
    my_parallel_in/parallel_input my_parallel_in_in/parallel_input_in \
    my_serial_in/serial_input my_mode1/mode_1 my_mode2/mode_2 \
    my_serial_out/serial_output my_parallel_out/parallel_output \
    my_parallel_out_in/parallel_output_in my_control_signal/control_from_bc2a
```

BC_8 Cell

Figure 4-11. Generic BC_8 Cell



The following example is a Verilog BC_8 definition using the ‘mgc_dft_cell_type’ attribute:

```

module my_bc_8 (my_shiftdr, my_clockdr, my_updatedr, my_parallel_in,
    my_parallel_in_in, my_serial_in, my_mode, my_serial_out,
    my_parallel_out, my_parallel_out_in);

input my_shiftdr, my_clockdr, my_updatedr, my_parallel_in, my_serial_in,
    my_parallel_in_in, my_mode;

output my_serial_out, my_parallel_out, my_parallel_out_in;

specify
    specparam mgc_dft_cell_type = "bc_8";
    specparam mgc_dft_pin_type$my_clockdr = "clockdr";
    specparam mgc_dft_pin_type$my_shiftdr = "shiftdr";
    specparam mgc_dft_pin_type$my_updatedr = "updatedr";
    specparam mgc_dft_pin_type$my_mode = "mode";
    specparam mgc_dft_pin_type$my_parallel_in = "parallel_input";
    specparam mgc_dft_pin_type$my_parallel_out = "parallel_output";
    specparam mgc_dft_pin_type$my_parallel_in_in = "parallel_input_in";
    specparam mgc_dft_pin_type$my_parallel_out_in = "parallel_output_in";
    specparam mgc_dft_pin_type$my_serial_in = "serial_input";
    specparam mgc_dft_pin_type$my_serial_out = "serial_output";
endspecify
endmodule

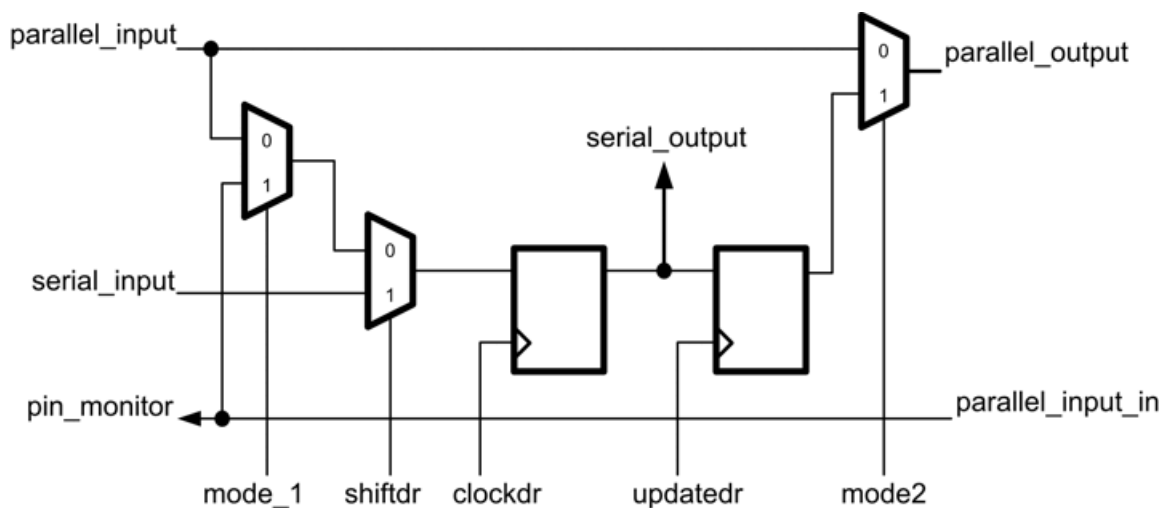
```

The following example is a BC_8 cell definition using the Add Technology Cell command:

```
add technology cell my_bc_8 bc_8
-inputs my_shiftdr my_clockdr my_updatedr my_parallel_in my_serial_in \
my_parallel_in_in my_mode \
-outputs my_serial_out my_parallel_out my_parallel_out_in \
-connect my_shiftdr/shiftdr my_clockdr/clockdr my_updatedr/updatedr \
my_parallel_in/parallel_input my_parallel_in_in/parallel_input_in \
my_serial_in/serial_input my_mode/mode my_serial_out/serial_output \
my_parallel_out/parallel_output my_parallel_out_in/parallel_output_in
```

BC_9 Cell

Figure 4-12. Generic BC_9 Cell



The following example is a Verilog BC_9 definition using the 'mgc_dft_cell_type' attribute:

```
module my_bc_9 (my_shiftdr, my_clockdr, my_updatedr, my_parallel_in,
    my_parallel_in_in, my_serial_in, my_model, my_mode2,
    my_serial_out, my_parallel_out, my_pin_monitor);

input my_shiftdr, my_clockdr, my_updatedr, my_parallel_in, my_serial_in,
    my_parallel_in_in, my_model, my_mode2;

output my_serial_out, my_parallel_out, my_pin_monitor;

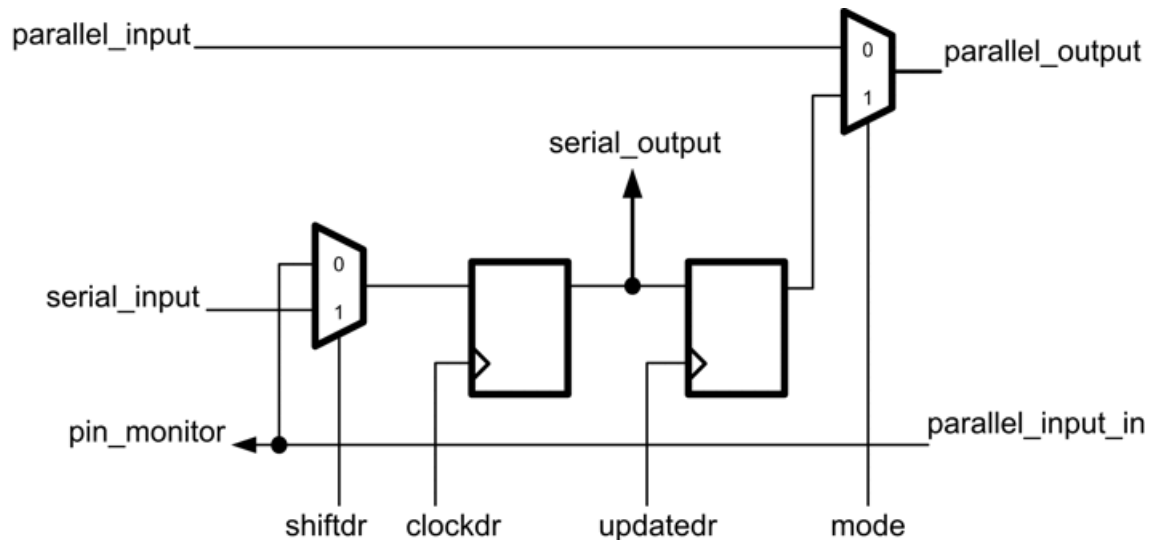
specify
    specparam mgc_dft_cell_type = "bc_9";
    specparam mgc_dft_pin_type$my_clockdr = "clockdr";
    specparam mgc_dft_pin_type$my_shiftdr = "shiftdr";
    specparam mgc_dft_pin_type$my_updatedr = "updatedr";
    specparam mgc_dft_pin_type$my_model = "mode_1";
    specparam mgc_dft_pin_type$my_mode2 = "mode2";
    specparam mgc_dft_pin_type$my_parallel_in = "parallel_input";
    specparam mgc_dft_pin_type$my_parallel_out = "parallel_output";
    specparam mgc_dft_pin_type$my_parallel_in_in = "parallel_input_in";
    specparam mgc_dft_pin_type$my_pin_monitor = "pin_monitor";
    specparam mgc_dft_pin_type$my_serial_in = "serial_input";
    specparam mgc_dft_pin_type$my_serial_out = "serial_output";
endspecify
endmodule
```

The following example is a BC_9 cell definition using the Add Technology Cell command:

```
add technology cell my_bc_9 bc_9
-inputs my_shiftdr my_clockdr my_updatedr my_parallel_in my_serial_in \
my_parallel_in_in my_mode1 my_mode2 \
-outputs my_serial_out my_parallel_out my_pin_monitor\
-connect my_shiftdr/shiftdr my_clockdr/clockdr my_updatedr/updatedr \
my_parallel_in/parallel_input my_parallel_in_in/parallel_input_in \
my_serial_in/serial_input my_mode1/mode_1 my_mode2/mode2 \
my_serial_out/serial_output my_parallel_out/parallel_output \
my_pin_monitor/pin_monitor
```

BC_10 Cell

Figure 4-13. Generic BC_10 Cell



The following example is a Verilog BC_10 definition using the 'mgc_dft_cell_type' attribute:

```
module my_bc_10 (my_shiftdr, my_clockdr, my_updatedr, my_parallel_in,
  my_parallel_in_in, my_serial_in, my_mode, my_serial_out,
  my_parallel_out, my_pin_monitor);

input my_shiftdr, my_clockdr, my_updatedr, my_parallel_in, my_serial_in,
  my_parallel_in_in, my_mode;

output my_serial_out, my_parallel_out, my_pin_monitor;

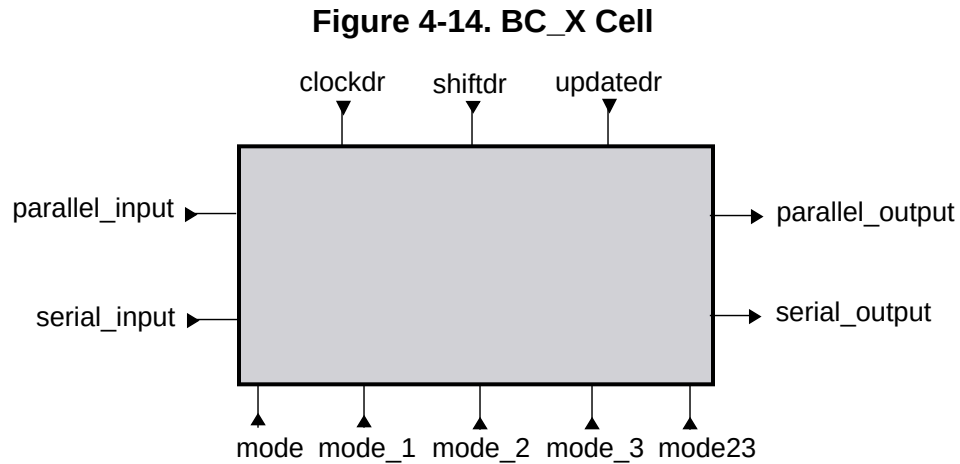
specify
  specparam mgc_dft_cell_type = "bc_10";
  specparam mgc_dft_pin_type$my_clockdr = "clockdr";
  specparam mgc_dft_pin_type$my_shiftdr = "shiftdr";
  specparam mgc_dft_pin_type$my_updatedr = "updatedr";
  specparam mgc_dft_pin_type$my_mode = "mode";
  specparam mgc_dft_pin_type$my_parallel_in = "parallel_input";
  specparam mgc_dft_pin_type$my_parallel_out = "parallel_output";
  specparam mgc_dft_pin_type$my_parallel_in_in = "parallel_input_in";
  specparam mgc_dft_pin_type$my_pin_monitor = "pin_monitor";
  specparam mgc_dft_pin_type$my_serial_in = "serial_input";
  specparam mgc_dft_pin_type$my_serial_out = "serial_output";
endspecify
endmodule
```

The following example is a BC_10 cell definition using the Add Technology Cell command:

```
add technology cell my_bc_10 bc_10
-inputs my_shiftdr my_clockdr my_updatedr my_parallel_in my_serial_in \
my_parallel_in_in my_mode \
-outputs my_serial_out my_parallel_out my_pin_monitor\
-connect my_shiftdr/shiftdr my_clockdr/clockdr my_updatedr/updatedr \
my_parallel_in/parallel_input my_parallel_in_in/parallel_input_in \
my_serial_in/serial_input my_mode/mode my_serial_out/serial_output \
my_parallel_out/parallel_output my_pin_monitor/pin_monitor
```

BC_X Cell

It may be difficult to classify certain technology boundary scan cells into any of the standard boundary scan cell types used in BSDArchitect. In this case, you use the special cell type 'BC_X'. If you specify this cell type, the design rules checks specified in [Boundary Scan Rules Checking](#) do not prevent the insertion of boundary scan.



The following example is a Verilog BC_X definition using the 'mgc_dft_cell_type' attribute:

```
module my_bc_x (my_shiftdr, my_clockdr, my_updatedr, my_parallel_in,
               my_serial_in, my_mode, my_model, my_mode2, my_mode3, my_mode4,
               my_serial_out, my_parallel_out);

input my_shiftdr, my_clockdr, my_updatedr, my_parallel_in, my_serial_in,
      my_mode, my_model, my_mode2, my_mode3, my_mode4;

output my_serial_out, my_parallel_out;

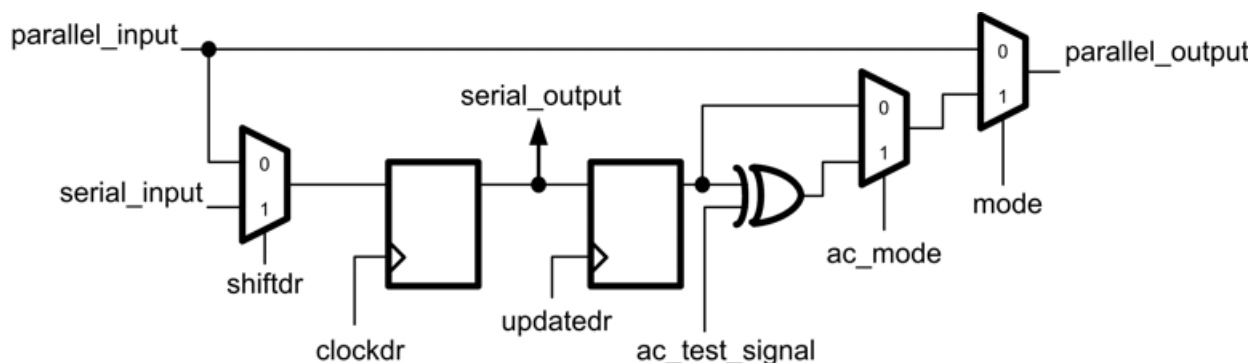
specify
  specparam mgc_dft_cell_type = "bc_x";
  specparam mgc_dft_pin_type$my_clockdr = "clockdr";
  specparam mgc_dft_pin_type$my_shiftdr = "shiftdr";
  specparam mgc_dft_pin_type$my_updatedr = "updatedr";
  specparam mgc_dft_pin_type$my_mode = "mode";
  specparam mgc_dft_pin_type$my_model = "mode_1";
  specparam mgc_dft_pin_type$my_mode2 = "mode_2";
  specparam mgc_dft_pin_type$my_mode3 = "mode_3";
  specparam mgc_dft_pin_type$my_mode4 = "mode23";
  specparam mgc_dft_pin_type$my_parallel_in = "parallel_input";
  specparam mgc_dft_pin_type$my_parallel_out = "parallel_output";
  specparam mgc_dft_pin_type$my_serial_in = "serial_input";
  specparam mgc_dft_pin_type$my_serial_out = "serial_output";
endspecify
endmodule
```

The following example is a BC_X cell definition using the Add Technology Cell command:

```
add technology cell my_bc_x bc_x
-inputs my_shiftdr my_clockdr my_updatedr my_parallel_in my_serial_in \
my_mode my_model my_mode2 my_mode3 my_mode4 \
-outputs my_serial_out my_parallel_out \
-connect my_shiftdr/shiftdr my_clockdr/clockdr my_updatedr/updatedr \
my_parallel_in/parallel_input my_serial_in/serial_input my_mode/mode \
my_model/mode_1 my_mode2/mode_2 my_mode3/mode_3 my_mode4/mode23 \
my_serial_out/serial_output my_parallel_out/parallel_output
```

AC_1 Cell

Figure 4-15. Generic AC_1 Cell



The following example is a Verilog AC_1 definition using the ‘mgc_dft_cell_type’ attribute:

```

module my_ac_1 (my_shiftdr, my_clockdr, my_updatedr,
    my_parallel_input, my_serial_input, my_mode, my_ac_mode,
    my_ac_test_signal, my_serial_output, my_parallel_output);

input my_shiftdr, my_clockdr, my_updatedr, my_parallel_input,
    my_serial_input, my_mode, my_ac_mode, my_ac_test_signal;

output my_serial_output, my_parallel_output;

specify
    specparam mgc_dft_cell_type = "ac_1";
    specparam mgc_dft_pin_type$my_clockdr = "clockdr";
    specparam mgc_dft_pin_type$my_shiftdr = "shiftdr";
    specparam mgc_dft_pin_type$my_updatedr = "updatedr";
    specparam mgc_dft_pin_type$my_mode = "mode";
    specparam mgc_dft_pin_type$my_parallel_input = "parallel_input";
    specparam mgc_dft_pin_type$my_parallel_output = "parallel_output";
    specparam mgc_dft_pin_type$my_serial_input = "serial_input";
    specparam mgc_dft_pin_type$my_serial_output = "serial_output";
    specparam mgc_dft_pin_type$my_ac_mode = "ac_mode";
    specparam mgc_dft_pin_type$my_ac_test_signal = "ac_test_signal";
endspecify
endmodule

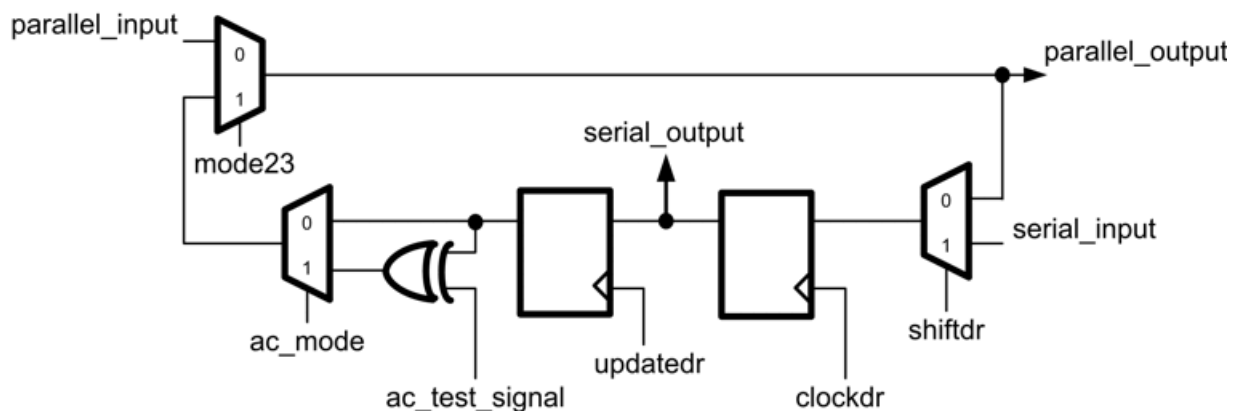
```

The following example is an AC_1 cell definition using the Add Technology Cell command:

```
add tech cell my_ac_1 ac_1
    -inputs my_shiftdr my_clockdr my_updatedr my_parallel_input my_serial_input \
    my_mode my_ac_test_signal my_ac_mode \
    -outputs my_serial_output my_parallel_output \
    -connect my_shiftdr/shiftdr my_clockdr/clockdr \
    my_updatedr/updatedr my_parallel_input/parallel_input \
    my_serial_input/serial_input my_mode/mode my_ac_test_signal/ac_test_signal \
    my_ac_mode/ac_mode my_serial_output/serial_output \
    my_parallel_output/parallel_output
```

AC 2 Cell

Figure 4-16. Generic AC_2 Cell



The following example is a Verilog AC_2 definition using the 'mgc_dft_cell_type' attribute:

```
module my_ac_2 (my_shiftdr, my_clockdr, my_updatedr,
               my_parallel_input, my_serial_input, my_mode23, my_ac_mode,
               my_ac_test_signal my_serial_output, my_parallel_output);

input my_shiftdr, my_clockdr, my_updatedr, my_parallel_input,
      my_serial_input, my_mode23, my_ac_mode, my_ac_test_signal;

output my_serial_output, my_parallel_output;

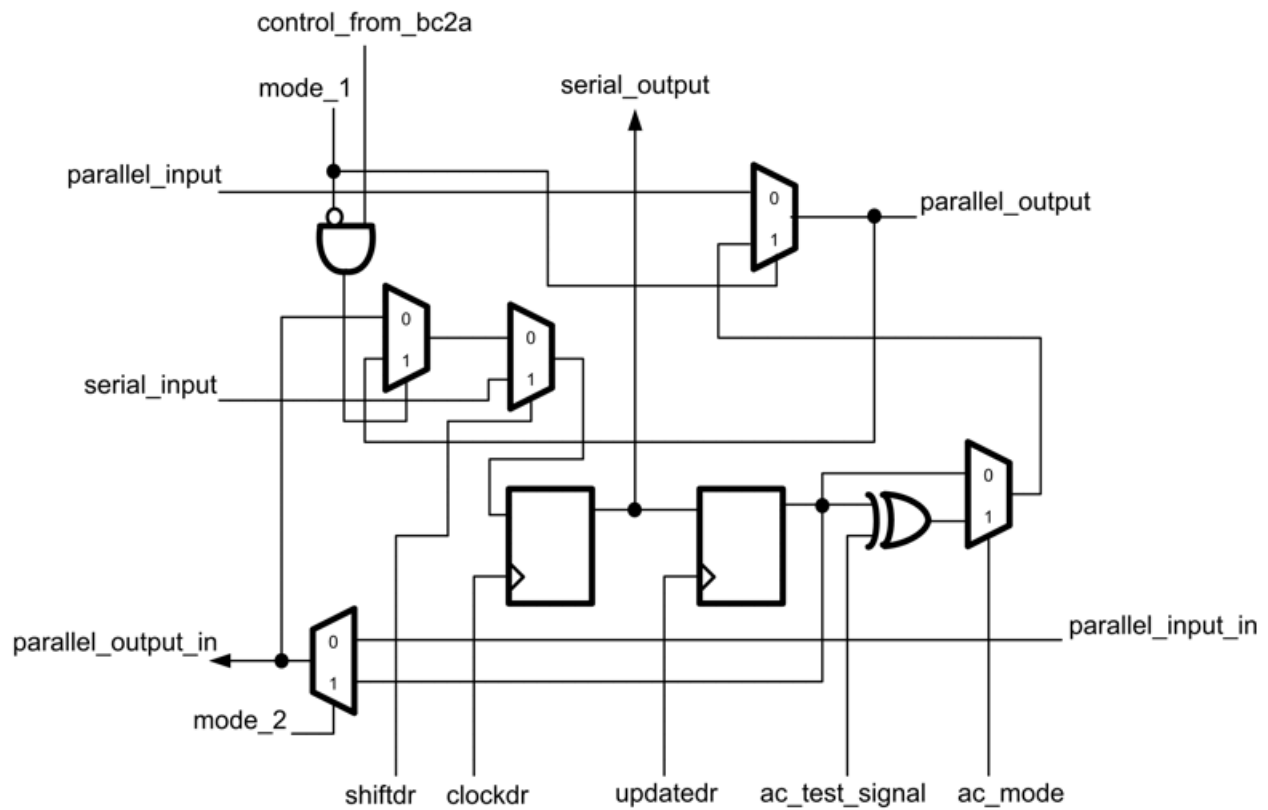
specify
  specparam mgc_dft_cell_type = "ac_2";
  specparam mgc_dft_pin_type$my_clockdr = "clockdr";
  specparam mgc_dft_pin_type$my_shiftdr = "shiftdr";
  specparam mgc_dft_pin_type$my_updatedr = "updatedr";
  specparam mgc_dft_pin_type$my_mode23 = "mode23";
  specparam mgc_dft_pin_type$my_parallel_input = "parallel_input";
  specparam mgc_dft_pin_type$my_parallel_output = "parallel_output";
  specparam mgc_dft_pin_type$my_serial_input = "serial_input";
  specparam mgc_dft_pin_type$my_serial_output = "serial_output";
  specparam mgc_dft_pin_type$my_ac_mode = "ac_mode";
  specparam mgc_dft_pin_type$my_ac_test_signal = "ac_test_signal";
endspecify
endmodule
```

The following example is an AC_2 cell definition using the Add Technology Cell command:

```
add tech cell my_ac_2 ac_2
-inputs my_shiftdr my_clockdr my_updatedr my_parallel_input my_serial_input \
my_mode23 my_ac_mode my_ac_test_signal \
-outputs my_serial_output my_parallel_output \
-connect my_shiftdr/shiftdr my_clockdr/clockdr my_updatedr/updatedr \
my_parallel_input/parallel_input my_serial_input/serial_input my_mode23/mode23 \
my_ac_test_signal/ac_test_signal my_ac_mode/ac_mode \
my_serial_output/serial_output my_parallel_output/parallel_output
```

AC_7 Cell

Figure 4-17. Generic AC_7 Cell



The following example is a Verilog AC_7 definition using the 'mgc_dft_cell_type' attribute:

```

module my_ac_7 (my_shiftldr, my_clockdr, my_updatedr, my_parallel_in,
    my_parallel_in_in, my_serial_in, my_mode_1, my_mode_2,
    my_control_from_bc2a, my_ac_mode, my_ac_test_signal, my_serial_out,
    my_parallel_out, my_parallel_out_in);

input my_shiftldr, my_clockdr, my_updatedr, my_parallel_in, my_serial_in,
    my_parallel_in_in, my_mode_1, my_mode_2, my_control_from_bc2a,
    my_ac_mode, my_ac_test_signal;

output my_serial_out, my_parallel_out, my_parallel_out_in;

specify
    specparam mgc_dft_cell_type = "ac_7";
    specparam mgc_dft_pin_type$my_clockdr = "clockdr";
    specparam mgc_dft_pin_type$my_shiftldr = "shiftldr";
    specparam mgc_dft_pin_type$my_updatedr = "updatedr";
    specparam mgc_dft_pin_type$my_mode_1 = "mode_1";
    specparam mgc_dft_pin_type$my_mode_2 = "mode_2";
    specparam mgc_dft_pin_type$my_parallel_in = "parallel_input";
    specparam mgc_dft_pin_type$my_parallel_out = "parallel_output";
    specparam mgc_dft_pin_type$my_parallel_in_in = "parallel_input_in";
    specparam mgc_dft_pin_type$my_parallel_out_in = "parallel_output_in";
    specparam mgc_dft_pin_type$my_serial_in = "serial_input";
    specparam mgc_dft_pin_type$my_serial_out = "serial_output";
    specparam mgc_dft_pin_type$my_control_from_bc2a = "control_from_bc2a";
    specparam mgc_dft_pin_type$my_ac_mode = "ac_mode";
    specparam mgc_dft_pin_type$my_ac_test_signal = "ac_test_signal";
endspecify
endmodule

```

The following example is an AC_7 cell definition using the Add Technology Cell command:

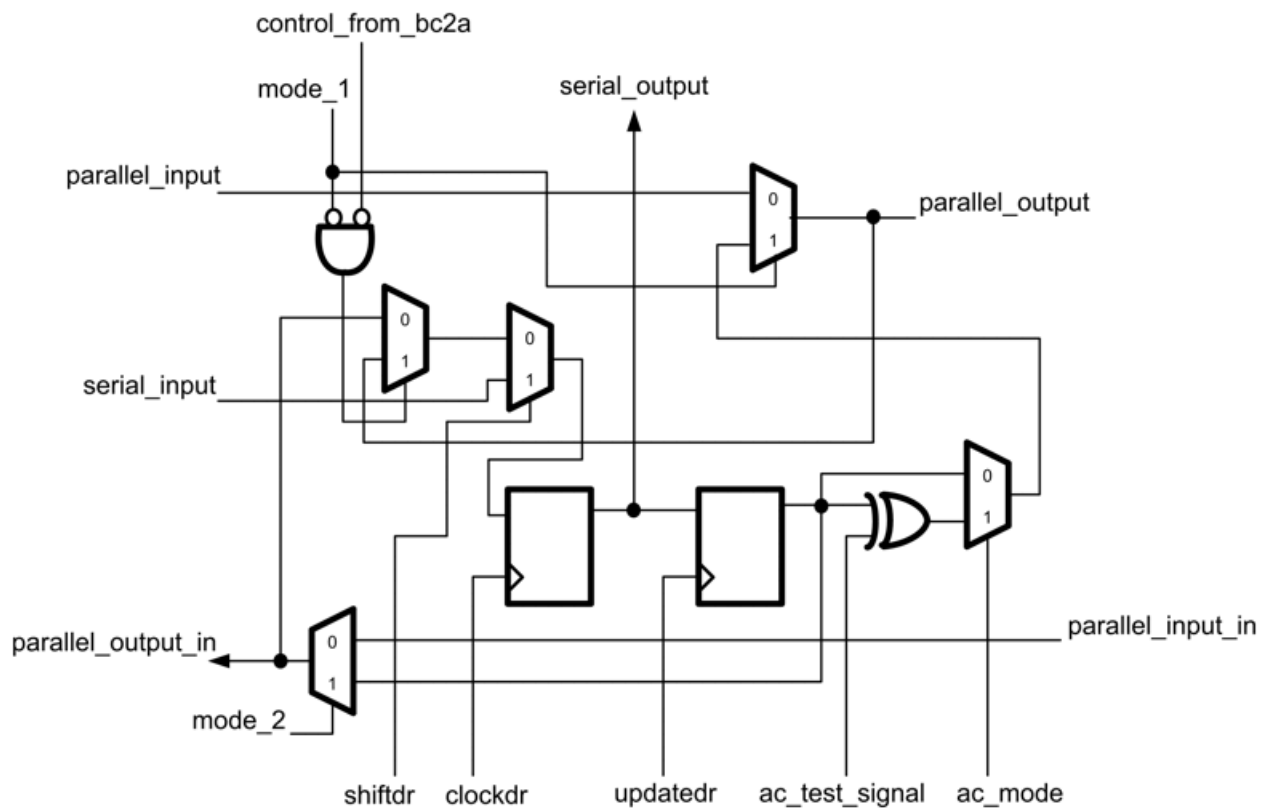
```

add tech cell my_ac_7 ac_7
    -inputs my_control_from_bc2a my_shiftldr my_clockdr my_updatedr \
    my_parallel_in my_parallel_in_in my_serial_in my_mode_1 my_mode_2 \
    my_ac_test_signal my_ac_mode \
    -outputs my_serial_out my_parallel_out my_parallel_out_in \
    -connect my_control_from_bc2a/control_from_bc2a my_shiftldr/shiftldr \
    my_clockdr/clockdr my_updatedr/updatedr my_parallel_in/parallel_input \
    my_parallel_in_in/parallel_input_in my_serial_in/serial_input my_mode_1/mode_1 \
    my_mode_2/mode_2 my_ac_test_signal/ac_test_signal ac_mode/ac_mode \
    my_serial_out/serial_output my_parallel_out/parallel_output \
    my_parallel_out_in/parallel_output_in

```

AC_7_LOW Cell

Figure 4-18. Generic AC_7_LOW Cell



The following example is a Verilog AC_7_LOW definition using the 'mgc_dft_cell_type' attribute:

```

module my_ac_7_low (my_shiftdr, my_clockdr, my_updatedr, my_mode_1,
    my_mode_2, my_ac_mode, my_parallel_in, my_parallel_in_in,
    my_serial_in, my_control_from_bc2a, my_ac_test_signal, my_serial_out,
    my_parallel_out, my_parallel_out_in);

input my_shiftdr, my_clockdr, my_updatedr, my_parallel_in, my_serial_in,
    my_parallel_in_in, my_mode_1, my_mode_2, my_control_from_bc2a,
    my_ac_mode, my_ac_test_signal;

output my_serial_out, my_parallel_out, my_parallel_out_in;

specify
    specparam mgc_dft_cell_type = "ac_7_low";
    specparam mgc_dft_pin_type$my_clockdr = "clockdr";
    specparam mgc_dft_pin_type$my_shiftdr = "shiftdr";
    specparam mgc_dft_pin_type$my_updatedr = "updatedr";
    specparam mgc_dft_pin_type$my_mode_1 = "mode_1";
    specparam mgc_dft_pin_type$my_mode_2 = "mode_2";
    specparam mgc_dft_pin_type$my_parallel_in = "parallel_input";
    specparam mgc_dft_pin_type$my_parallel_out = "parallel_output";
    specparam mgc_dft_pin_type$my_parallel_in_in = "parallel_input_in";
    specparam mgc_dft_pin_type$my_parallel_out_in = "parallel_output_in";
    specparam mgc_dft_pin_type$my_serial_in = "serial_input";
    specparam mgc_dft_pin_type$my_serial_out = "serial_output";
    specparam mgc_dft_pin_type$my_control_from_bc2a = "control_from_bc2a";
    specparam mgc_dft_pin_type$my_ac_mode = "ac_mode";
    specparam mgc_dft_pin_type$my_ac_test_signal = "ac_test_signal";
endspecify
endmodule

```

The following example is an AC_7_LOW cell definition using the Add Technology Cell command:

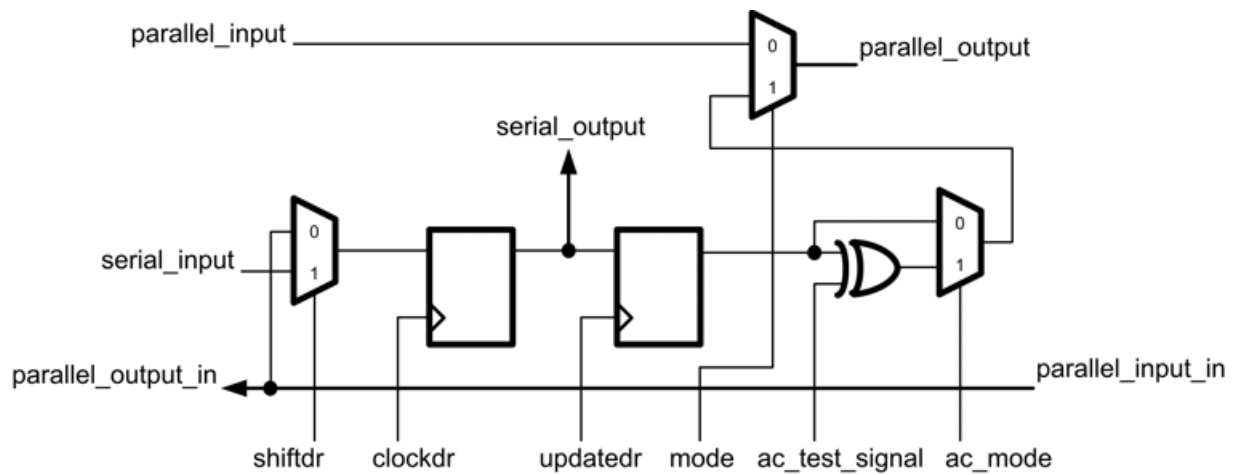
```

add tech cell my_ac_7_low ac_7_low
    -inputs my_control_from_bc2a my_shiftdr my_clockdr my_updatedr \
    my_parallel_in my_parallel_in_in my_serial_in my_mode_1 my_mode_2 \
    my_ac_test_signal my_ac_mode \
    -outputs my_serial_out my_parallel_out my_parallel_out_in \
    -connect my_control_from_bc2a/control_from_bc2a my_shiftdr/shiftdr \
    my_clockdr/clockdr my_updatedr/updatedr my_parallel_in/parallel_input \
    my_parallel_in_in/parallel_input_in my_serial_in/serial_input my_mode_1/mode_1 \
    my_mode_2/mode_2 my_ac_test_signal/ac_test_signal ac_mode/ac_mode \
    my_serial_out/serial_output my_parallel_out/parallel_output \
    my_parallel_out_in/parallel_output_in

```

AC_8 Cell

Figure 4-19. Generic AC_8 Cell



The following example is a Verilog AC_8 definition using the 'mgc_dft_cell_type' attribute:

```
module my_ac_8 (my_shiftdr, my_clockdr, my_updatedr, my_parallel_in,
  my_parallel_in_in, my_serial_in, my_mode, my_ac_mode,
  my_ac_test_signal, my_serial_out, my_parallel_out,
  my_parallel_out_in);

input my_shiftdr, my_clockdr, my_updatedr, my_parallel_in, my_serial_in,
  my_parallel_in_in, my_mode, my_ac_mode, my_ac_test_signal;

output my_serial_out, my_parallel_out, my_parallel_out_in;

specify
  specparam mgc_dft_cell_type = "ac_8";
  specparam mgc_dft_pin_type$my_clockdr = "clockdr";
  specparam mgc_dft_pin_type$my_shiftdr = "shiftdr";
  specparam mgc_dft_pin_type$my_updatedr = "updatedr";
  specparam mgc_dft_pin_type$my_mode = "mode";
  specparam mgc_dft_pin_type$my_parallel_in = "parallel_input";
  specparam mgc_dft_pin_type$my_parallel_out = "parallel_output";
  specparam mgc_dft_pin_type$my_parallel_in_in = "parallel_input_in";
  specparam mgc_dft_pin_type$my_parallel_out_in = "parallel_output_in";
  specparam mgc_dft_pin_type$my_serial_in = "serial_input";
  specparam mgc_dft_pin_type$my_serial_out = "serial_output";
  specparam mgc_dft_pin_type$my_ac_mode = "ac_mode";
  specparam mgc_dft_pin_type$my_ac_test_signal = "ac_test_signal";
endspecify
endmodule
```

The following example is an AC_8 cell definition using the Add Technology Cell command:

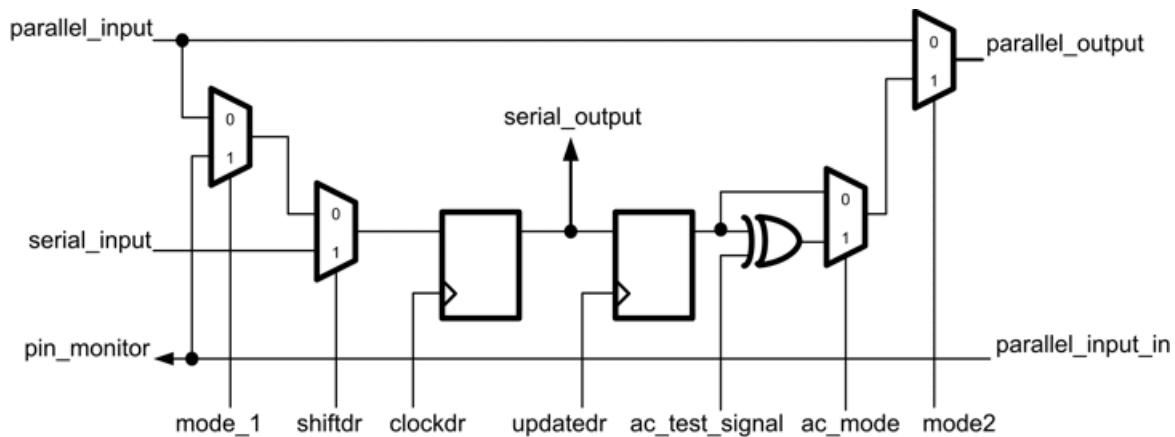
```

add tech cell my_ac_8 ac_8
-inputs my_shiftdr my_clockdr my_updatedr my_parallel_in my_serial_in my_mode \
my_parallel_in_in my_ac_mode my_ac_test_signal \
-outputs my_serial_out my_parallel_out my_parallel_out_in \
-connect my_shiftdr/shiftdr my_clockdr/clockdr my_updatedr/updatedr \
my_parallel_in/parallel_input my_ac_mode/ac_mode \
my_ac_test_signal/ac_test_signal my_serial_in/serial_input my_mode/mode \
my_serial_out/serial_output my_parallel_in_in/parallel_input_in \
my_parallel_out_in/parallel_output_in my_parallel_out/parallel_output

```

AC_9 Cell

Figure 4-20. Generic AC_9 Cell



The following example is a Verilog AC_9 definition using the 'mgc_dft_cell_type' attribute:

```
module my_ac_9 (my_shiftdr, my_clockdr, my_updatedr, my_parallel_in,
    my_parallel_in_in, my_serial_in, my_model, my_mode2, my_ac_test_signal
    my_ac_mode, my_serial_out, my_parallel_out, my_pin_monitor);

input my_shiftdr, my_clockdr, my_updatedr, my_parallel_in, my_serial_in,
    my_parallel_in_in, my_model, my_mode2, my_ac_mode, my_ac_test_signal;

output my_serial_out, my_parallel_out, my_pin_monitor;

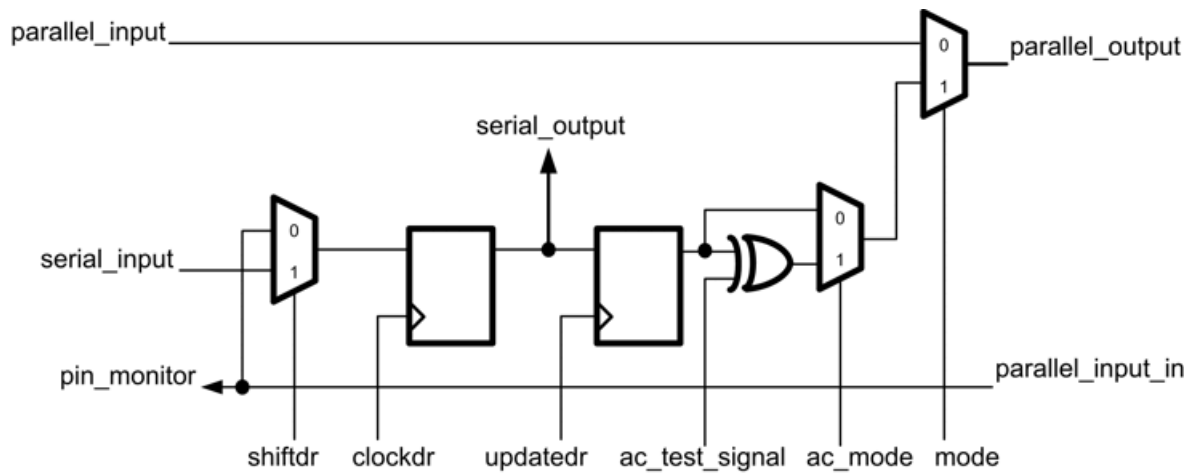
specify
    specparam mgc_dft_cell_type = "ac_9";
    specparam mgc_dft_pin_type$my_clockdr = "clockdr";
    specparam mgc_dft_pin_type$my_shiftdr = "shiftdr";
    specparam mgc_dft_pin_type$my_updatedr = "updatedr";
    specparam mgc_dft_pin_type$my_model = "mode_1";
    specparam mgc_dft_pin_type$my_mode2 = "mode2";
    specparam mgc_dft_pin_type$my_parallel_in = "parallel_input";
    specparam mgc_dft_pin_type$my_parallel_out = "parallel_output";
    specparam mgc_dft_pin_type$my_parallel_in_in = "parallel_input_in";
    specparam mgc_dft_pin_type$my_pin_monitor = "pin_monitor";
    specparam mgc_dft_pin_type$my_serial_in = "serial_input";
    specparam mgc_dft_pin_type$my_serial_out = "serial_output";
    specparam mgc_dft_pin_type$my_ac_mode = "ac_mode";
    specparam mgc_dft_pin_type$my_ac_test_signal = "ac_test_signal";
endspecify
endmodule
```

The following example is an AC_9 cell definition using the Add Technology Cell command:

```
add tech cell my_ac_9 ac_9
    -inputs my_shiftdr my_clockdr my_updatedr my_parallel_in my_serial_in \
    my_model my_mode2 my_parallel_in_in my_ac_mode my_ac_test_signal \
    -outputs my_serial_out my_pin_monitor my_parallel_out \
    -connect my_shiftdr/shiftdr my_clockdr/clockdr my_updatedr/updatedr \
    my_parallel_in/parallel_input my_ac_mode/ac_mode \
    my_ac_test_signal/ac_test_signal my_serial_in/serial_input my_model/mode1 \
    my_mode2/mode2 my_pin_monitor/pin_monitor my_serial_out/serial_output \
    my_parallel_in_in/parallel_input_in my_parallel_out/parallel_output
```

AC_10 Cell

Figure 4-21. Generic AC_10 Cell



The following example is a Verilog AC_10 definition using the 'mgc_dft_cell_type' attribute:

```
module my_ac_10 (my_shiftdr, my_clockdr, my_updatedr, my_parallel_in,
  my_parallel_in_in, my_serial_in, my_ac_test_signal, my_ac_mode,
  my_mode, my_serial_out, my_parallel_out, my_pin_monitor);

input my_shiftdr, my_clockdr, my_updatedr, my_parallel_in, my_serial_in,
  my_parallel_in_in, my_mode, my_ac_mode, my_ac_test_signal;

output my_serial_out, my_parallel_out, my_pin_monitor;

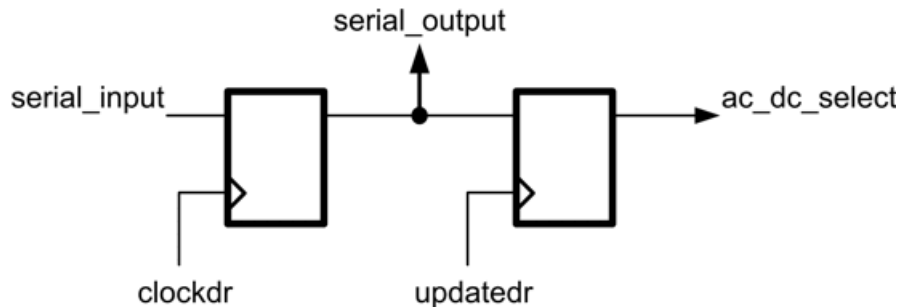
specify
  specparam mgc_dft_cell_type = "ac_10";
  specparam mgc_dft_pin_type$my_clockdr = "clockdr";
  specparam mgc_dft_pin_type$my_shiftdr = "shiftdr";
  specparam mgc_dft_pin_type$my_updatedr = "updatedr";
  specparam mgc_dft_pin_type$my_mode = "mode";
  specparam mgc_dft_pin_type$my_parallel_in = "parallel_input";
  specparam mgc_dft_pin_type$my_parallel_out = "parallel_output";
  specparam mgc_dft_pin_type$my_parallel_in_in = "parallel_input_in";
  specparam mgc_dft_pin_type$my_pin_monitor = "pin_monitor";
  specparam mgc_dft_pin_type$my_serial_in = "serial_input";
  specparam mgc_dft_pin_type$my_serial_out = "serial_output";
  specparam mgc_dft_pin_type$my_ac_mode = "ac_mode";
  specparam mgc_dft_pin_type$my_ac_test_signal = "ac_test_signal";
endspecify
endmodule
```

The following example is an AC_10 cell definition using the Add Technology Cell command:

```
add tech cell my_ac_10 ac_10
-inputs my_shiftdr my_clockdr my_updatedr my_parallel_in my_serial_in my_mode \
my_parallel_in_in my_ac_mode my_ac_test_signal \
-outputs my_serial_out my_pin_monitor my_parallel_out \
-connect my_shiftdr/shiftdr my_clockdr/clockdr my_updatedr/updatedr \
my_parallel_in/parallel_input my_ac_mode/ac_mode \
my_ac_test_signal/ac_test_signal my_serial_input/serial_input my_mode2/mode \
my_pin_monitor/pin_monitor my_serial_out/serial_output \
my_parallel_in_in/parallel_input_in my_parallel_out/parallel_output
```

AC_SelX Cell

Figure 4-22. Generic AC_SelX Cell



The following example is a Verilog AC_SelX definition using the ‘mgc_dft_cell_type’ attribute:

```
module my_ac_selx ( my_clockdr, my_updatedr,
    my_serial_input, my_serial_output);

input my_clockdr, my_updatedr, my_serial_input;

output my_serial_output, my_ac_dc_select;

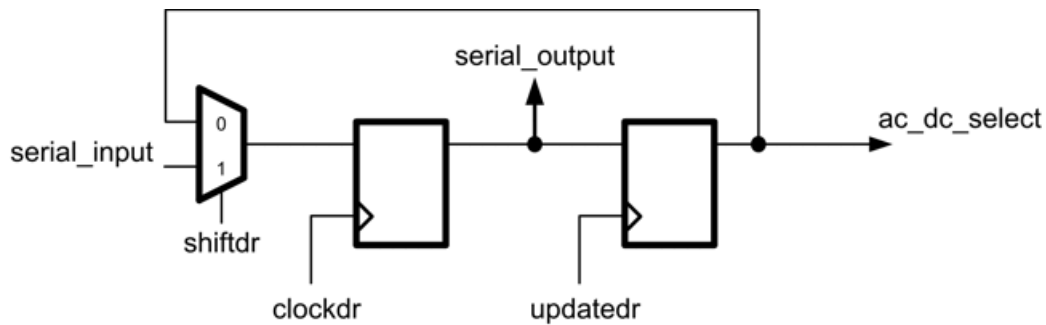
specify
    specparam mgc_dft_cell_type = "ac_selx";
    specparam mgc_dft_pin_type$my_clockdr = "clockdr";
    specparam mgc_dft_pin_type$my_updatedr = "updatedr";
    specparam mgc_dft_pin_type$my_serial_input = "serial_input";
    specparam mgc_dft_pin_type$my_serial_output = "serial_output";
    specparam mgc_dft_pin_type$my_ac_dc_select = "ac_dc_select";
endspecify
endmodule
```

The following example is an AC_SelX cell definition using the Add Technology Cell command:

```
add tech cell my_ac_selx ac_selx
-inputs my_clockdr my_updatedr my_serial_input \
-outputs my_serial_output my_ac_dc_select \
-connect my_clockdr/clockdr my_updatedr/updatedr \
my_serial_input/serial_input my_serial_output/serial_output \
my_ac_dc_select/ac_dc_select
```

AC_SelU Cell

Figure 4-23. Generic AC_SelU Cell



The following example is a Verilog AC_SelU definition using the 'mgc_dft_cell_type' attribute:

```
module my_ac_selu (my_shiftdr, my_clockdr, my_updatedr,
                  my_serial_input, my_serial_output);

input my_shiftdr, my_clockdr, my_updatedr, my_serial_input;

output my_serial_output, my_ac_dc_select;

specify
  specparam mgc_dft_cell_type = "ac_selu";
  specparam mgc_dft_pin_type$my_clockdr = "clockdr";
  specparam mgc_dft_pin_type$my_shiftdr = "shiftdr";
  specparam mgc_dft_pin_type$my_updatedr = "updatedr";
  specparam mgc_dft_pin_type$my_serial_input = "serial_input";
  specparam mgc_dft_pin_type$my_serial_output = "serial_output";
  specparam mgc_dft_pin_type$my_ac_dc_select = "ac_dc_select";
endspecify
endmodule
```

The following example is an AC_SelU cell definition using the Add Technology Cell command:

```
add tech cell my_ac_selu ac_selu
-inputs my_clockdr my_shiftdr my_updatedr my_serial_input
-outputs my_serial_output my_ac_dc_select \
-connect my_clockdr/clockdr my_shiftdr/shiftdr my_updatedr/updatedr \
my_serial_input/serial_input my_serial_output/serial_output \
my_ac_dc_select/ac_dc_select
```

Defining I/O Pads

BSDArchitect provides the ability to map I/O pads with particular technologies. Using the Add Pad Cell command, you can map technology cells to the I/O pads you may have previously

added. However, before you associate a particular pad cell with a technology-specific cell, you must first define the technology cell information with the [Add Technology Cell](#) command.

ADD TEchnology Cell *cell_name cell_type -INputs input_port... -Outputs output_port... [-INOuts inout_port...] -Connect connect_pairs... [-Nonexistent generic_ports...]*

Once you've defined a technology cell name, you can use the technology cell name as input to the Add Pad Cell command.

ADD PAd Cell *technology_cell_name core_port_name...*

BSDArchitect supports the following types of I/O pads:

- [Input Pad](#)
- [Output Pad](#)
- [Tri-state Pad](#)
- [Bidirectional Pad](#)
- [Open Collector Output Pad](#)
- [Differential Input and Output Pads](#)
- [AC Pads](#)

You add either generic I/O pads or technology-specific I/O pads. The following sections discuss adding generic I/O pads of these types using the tool commands. You can also define certain attributes in VHDL or create specparam settings in Verilog to add the same port information.

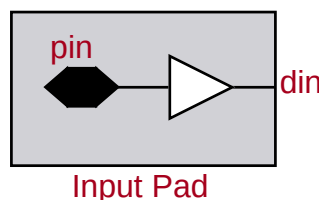
To add technology-mapped pads to core ports, issue the following command:

ADD PAd Cell {*technology_cell_name* | None} *core_port_name...*

For more information on this command, refer to [Add Pad Cell](#) in the *BSDArchitect Reference Manual*.

Input Pad

Figure 4-24. Basic Input Pad



The following examples create the input pad `my_iopad_in` using the Add Technology Cell command and using Verilog specparams:

```

add technology cell my_iopad_in input -input my_pin -output my_din \
-connect my_pin/pin my_din/din

module my_iopad_in (my_pin, my_din);
    input my_pin;
    output my_din;

    // Define input pad with specparams
    specify
        // PAD cell type
        specparam mgc_dft_cell_type = "input_pad";

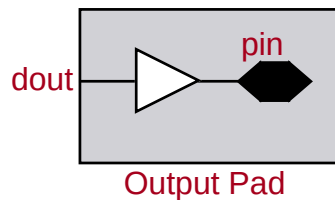
        // PAD port mapping information
        specparam mgc_dft_pin_type$my_pin = "pin";
        specparam mgc_dft_pin_type$my_din = "data_input";

    endspecify
endmodule

```

Output Pad

Figure 4-25. Basic Output Pad



The following examples create the output pad `my_iopad_out` using the Add Technology Cell command and using Verilog specparams:

```

add technology cell my_iopad_out output -input my_dout -output my_pin \
-connect my_pin/pin my_dout/dout

module my_iopad_out (my_pin, my_dout);
    input my_dout;
    output my_pin;

    // Define output pad with specparams
    specify
        // PAD cell type
        specparam mgc_dft_cell_type = "output_pad";

        // PAD port mapping information
        specparam mgc_dft_pin_type$my_dout = "data_output";
        specparam mgc_dft_pin_type$my_pin = "pin";

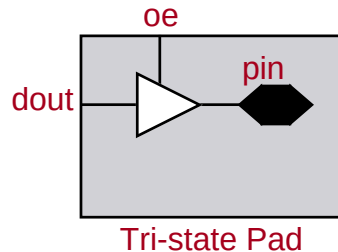
    endspecify
endmodule

```

Tri-state Pad

To add tri-state pads, use the Add Tristate Port command. For more information, refer to the [Add Tristate Port](#) command in the *BSDArchitect Reference Manual*.

Figure 4-26. Basic Tri-State Pad



The following examples create the tri-state pad `my_tri_l` using the Add Technology Cell command and using Verilog specparams:

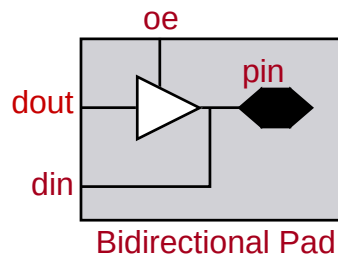
```
add technology cell my_tri_l tristate \  
    -input my_dout my_oe \  
    -output my_pin \  
    -connect my_din/din my_dout/dout my_oe/oe  
  
module my_tri_l (my_dout, my_oe, my_din, PI);  
    input my_dout, my_oe, PI;  
    output my_din;  
  
    // Define tri-state pad with specparams  
    specify  
        // PAD cell type  
        specparam mgc_dft_cell_type = "tristate_pad";  
  
        // PAD port mapping information  
        specparam mgc_dft_pin_type$my_dout = "data_output";  
        specparam mgc_dft_pin_type$my_oe = "output_enable_h";  
        specparam mgc_dft_pin_type$my_din = "pin";  
    endspecify  
  
endmodule
```

This Verilog model specifies mapping of the output enable, `my_oe`, to `output_enable_h`. Alternatively, it could map to `output_enable_l`. These different mappings provide a way to specify the enable signal of the pad as active high or active low.

Bidirectional Pad

You can add bidirectional pads using the Add Tristate Port command. [Figure 4-27](#) illustrates a generic bidirectional pad.

Figure 4-27. Basic Bidirectional Pad



The following examples create the bidirectional pad `my_bidi_l` using the Add Technology Cell command and using Verilog specparams:

```
add technology cell my_bidi_l bidirectional -input my_dout my_oe -output my_din \
    -inout my_pin -connect my_din/din my_dout/dout my_oe/oe
```

```
module my_bidi_l (my_pin, my_dout, my_oe, my_din);
    inout my_pin;
    input my_dout, my_oe;
    output my_din;

    // Define bidirectional pad with specparams
    specify
        // PAD cell type
        specparam mgc_dft_cell_type = "bidirectional_pad";

        // PAD port mapping information
        specparam mgc_dft_pin_type$my_dout = "data_output";
        specparam mgc_dft_pin_type$my_oe = "output_enable_l";
        specparam mgc_dft_pin_type$my_din = "data_input";
        specparam mgc_dft_pin_type$my_pin = "pin";
    endspecify
endmodule
```

This Verilog model specifies mapping of the output enable, `my_oe`, to `output_enable_l`. Alternatively, it could map to `output_enable_h`. These different mappings provide a way to specify the enable signal of the pad as active high or active low.

Note



If you connect the bidirectional pad to a TAP input, BSDArchitect connects *pin* to the top-level pin and *din* to the TAP input and sets the pad to the inactive state; *dout* is left floating.

Open Collector Output Pad

To associate open collector output pads with core output ports, use the Add Opencollector Port command. This command generates a generic open collector pad based on the options you set.

ADD OPencollector Port **-Out *out_port_name*** -Disable_result [WEAK0 | WEAK1 | PULL0 | PULL1]

The only difference between the WEAK and PULL values is the placement of the pull-up (or pull-down) resistor. For situations when a pull-up or pull-down is internal to the component, use the PULL values. Otherwise, for external pull-up and pull-down resistors, use the WEAK values.

For more information, refer to the [Add Opencollector Port](#) command in the *BSDArchitect Reference Manual*.

There are two types of open collector driver pins: the H-type shown in [Figure 4-28](#), and the L-type shown in [Figure 4-29](#). The H-type provides strong H (high) data and weak L (low) data and the L-type provides strong L data and weak H data.

Note

 The “open-collector” term is a commonly used name in the industry, although the driver circuit can also be constructed based on an open-emitter (ECL), open-drain (MOSFET), or open-source (MOSFET) configuration.

[Figure 4-28](#) shows the H-type of open collector pad, and [Table 4-2](#) shows the associated truth table. You use this type of pad with the following configurations: PNP open collector, P-channel open drain, NPN open emitter, and N-channel open source.

Figure 4-28. H-Type Open Collector Output Pad

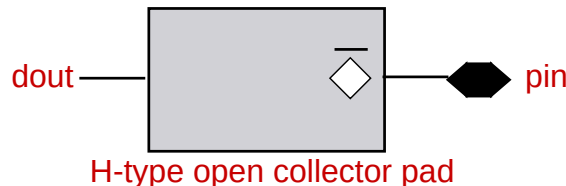


Table 4-2. H-Type Open Collector Truth Table

dout	pin
0	weak0
1	1

To specify an H-type open collector output pad, issue the following command where *oc_port* is the name of the port in your design that you are configuring:

```
add opencollector port -out oc_port -disable_result weak0
```

Figure 4-29 shows the L-type of open collector pad, and Table 4-3 shows the associated truth table. You use this type of pad with the following configurations: NPN open collector, N-channel open drain, PNP open emitter, and P-channel open source.

Figure 4-29. L-Type Open Collector Output Pad

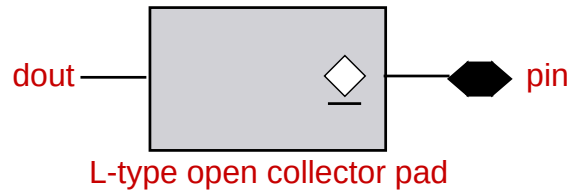


Table 4-3. L-Type Open Collector Truth Table

dout	pin
0	0
1	weak1

To specify an L-type open collector output pad, issue the following command where *oc_port* is the name of the port in your design that you are configuring:

```
add opencollector port -out oc_port -disable_result weak1
```

Mapping information is required for pins *dout* and *pin*. The mapping information is the same for both types of open collector pads.

The following examples create the open collector pad *my_iopad_oc* using the Add Technology Cell command and using Verilog specparams:

```
add technology cell my_iopad_oc opencollector -input my_dout -output my_pin \  
-connect my_pin/pin my_dout/dout
```

```
module my_iopad_oc (my_pin, my_dout);  
    input my_dout;  
    output my_pin;  
  
    // Define open collector pad with specparams  
    specify  
        // PAD cell type  
        specparam mgc_dft_cell_type = "opencollector_pad";  
  
        // PAD port mapping information  
        specparam mgc_dft_pin_type$my_dout = "data_output_h";  
        specparam mgc_dft_pin_type$my_pin = "pin_1";  
    endspecify  
  
endmodule
```

Dout can be mapped to either data_output_h or data_output_l. Similarly, my_pin can be mapped to pin_l or pin_h. The various combinations of these mappings determine which of the open collector pads is used.

Differential Input and Output Pads

To associate differential pad information with specified ports, use the Add Differential Port command.

ADD Differential Port **core_port_name...** [-Powerdown {None | High | Low}]
[-Type {Voltage | Current}] [-Diff_n_name *diff_n_name...*]

For more information, refer to the [Add Differential Port](#) command in the *BSDArchitect Reference Manual*.

[Figure 4-30](#) illustrates the differential input and output pads, while [Table 4-3](#) shows the truth tables associated with both.

Figure 4-30. Generic Differential Input and Output Pads

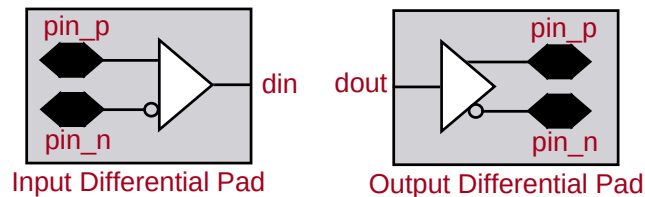


Table 4-4. Differential Pad Truth Tables

Input Differential Pad			Output Differential Pad		
pin_p	pin_n	din	dout	pin_p	pin_n
0	0	X	0	0	1
0	1	0	1	1	0
1	0	1			
1	1	X			

The following examples create the differential input pad my_iopad_diff_in using the Add Technology Cell command and using Verilog specparams:

```
add technology cell my_iopad_diff_in diff_in -input my_pin -output my_din \
-connect my_pin_n/pin_n my_din/din my_pin_p/pin_p
```

```

module my_iopad_diff_in (my_pin_p, my_pin_n, my_din);
    input my_pin_p, my_pin_n;
    output my_din;

    // Define differential input pad with specparams
    specify
    // PAD cell type
        specparam mgc_dft_cell_type = "differential_input_pad";

    // PAD port mapping information
        specparam mgc_dft_pin_type$my_pin_n = "pin_n";
        specparam mgc_dft_pin_type$my_pin_p = "pin_p";
        specparam mgc_dft_pin_type$my_din = "data_input";
    endspecify

endmodule

```

The following examples create the differential output pad `my_iopad_diff_out` using the Add Technology Cell command and using Verilog specparams:

```

add technology cell my_iopad_diff_out diff_out -input my_dout -output my_pin_p my_pin_n \
-connect my_pin_n/pin_n my_pin_n/pin_n my_dout/dout

module my_iopad_diff_out (my_pin_n, my_pin_p, my_dout);
    input my_dout;
    output my_pin_n, my_pin_p;

    // Define differential output pad with specparams
    specify
    // PAD cell type
        specparam mgc_dft_cell_type = "differential_output_pad";

    // PAD port mapping information
        specparam mgc_dft_pin_type$my_dout = "data_output";
        specparam mgc_dft_pin_type$my_pin_p = "pin_p";
        specparam mgc_dft_pin_type$my_pin_n = "pin_n";
    endspecify

endmodule

```

The specparam, `mgc_dft_cell_type`, can also be specified as “`diff_out_pad`.”

AC Pads

BSDArchitect provides the following extended generic AC versions of the input, bidirectional, and differential input pads. These pads are identical to the corresponding DC versions, but contain additional pins.

AC Input Pad

Figure 4-31. AC Input Pad



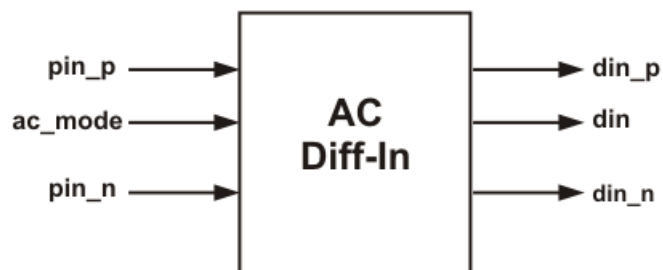
The following example creates the input pad PAD_AC_IN using the Add Technology Cell command and using Verilog specparams:

```
add technology cell PAD_AC_IN input -inputs I acmode -output O \  
-connect I/pin O/din acmode/ac_mode
```

```
module PAD_AC_IN (  
  // Outputs  
  O,  
  // Inputs  
  I,  
  // ac mode  
  acmode  
);  
// Input ports  
input I, acmode ;  
// Output ports  
output O ;  
// Input Type  
specify  
  specparam mgc_dft_cell_type= "ac_input";  
  // Vendor pin to generic pin mapping  
  specparam mgc_dft_pin_type$I = "pin";  
  specparam mgc_dft_pin_type$O = "din";  
  specparam mgc_dft_pin_type$acmode = "ac_mode";  
endspecify  
endmodule
```

AC Differential Input Pad

Figure 4-32. AC Differential Input Pad



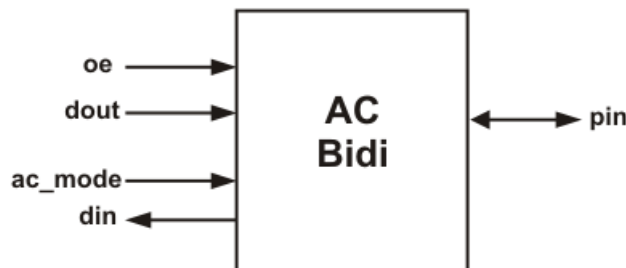
The following example creates the differential input pad PAD_AC_DIFFI using the Add Technology Cell command and using Verilog specparams:

```
add technology cell PAD_AC_DIFFI diff_in -inputs P N acmode -outputs O dP dN \  
-connect P/pin_p N/pin_n O/din dP/din_p dN/din_n acmode/ac_mode
```

```
module PAD_AC_DIFFI (  
  // Outputs  
  O,  
  // Inputs  
  P,  
  N,  
  // ac mode  
  acmode,  
  dP,  
  dN,  
);  
  // Input ports  
  input P, N, acmode;  
  // Output ports  
  output O , dP, dN;  
  // Differential input Type  
  specify  
    specparam mgc_dft_cell_type="ac_differential_input_pad";  
  // Vendor pin to generic pin mapping  
  specparam mgc_dft_pin_type$P = "pin_p";  
  specparam mgc_dft_pin_type$N = "pin_n";  
  specparam mgc_dft_pin_type$O = "data_input";  
  specparam mgc_dft_pin_type$dP = "din_p";  
  specparam mgc_dft_pin_type$dN = "din_n";  
  specparam mgc_dft_pin_type$acmode = "ac_mode";  
  endspecify  
endmodule
```

AC Bidirectional Pad

Figure 4-33. AC Bidi Pad



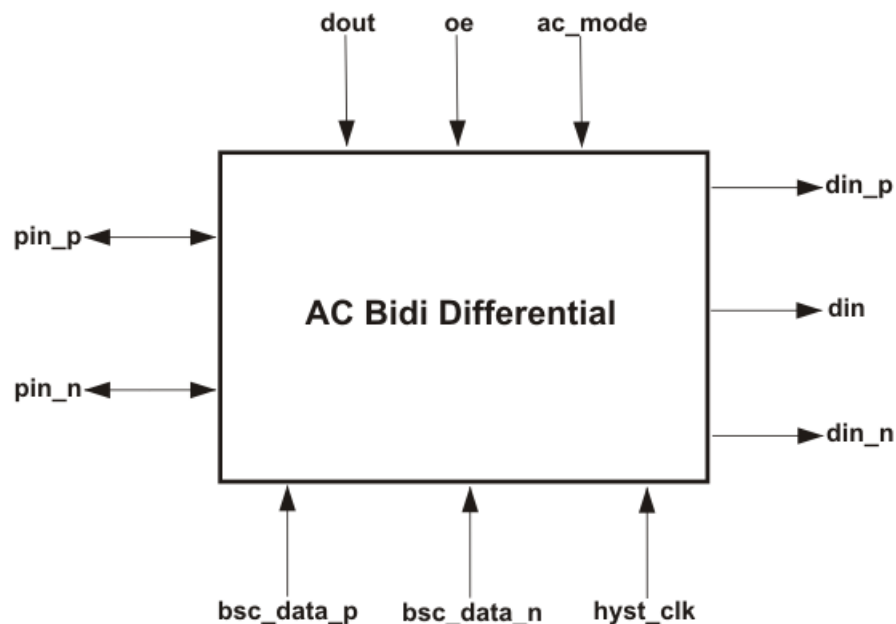
The following example creates the bidirectional pad PAD_AC_BIDI using the Add Technology Cell command and using Verilog specparams:

```
add technology cell PAD_AC_BIDI bidirectional -inputs I OE acmode -output O -inout IO \  
-connect IO/pin I/dout OE/oe O/din acmode/ac_mode
```

```
module PAD_AC_BIDI (  
  // Outputs  
  O,  
  // Inputs  
  I,  
  // ac mode  
  acmode ,  
  OE,  
  //Inouts  
  IO  
);  
// Input ports  
input I, OE, acmode ;  
// Output ports  
output O ;  
// Inout ports  
inout IO;  
// Bi-di Type  
specify  
specparam mgc_dft_cell_type="ac_bidirectional_pad";  
// Vendor pin to generic pin mapping  
specparam mgc_dft_pin_type$I = "pin";  
specparam mgc_dft_pin_type$OE = "dout";  
specparam mgc_dft_pin_type$OE = "output_enable_1";  
specparam mgc_dft_pin_type$I = "din";  
specparam mgc_dft_pin_type$acmode = "ac_mode";  
endspecify  
endmodule
```

AC Bidi Differential Pad

Figure 4-34. AC Bidi Differential Pad



The following example creates the AC bidi differential pad `my_ac_bidi_diff_enable_low` using the Add Technology Cell command and using Verilog specparams:

```
add technology cell my_ac_bidi_diff_enable_low \
  -inputs my_dout my_oe my_ac my_bsc_p my_bsc_n my_clk \
  -outputs my_din my_din_p my_din_n \
  -inouts my_pin_p my_pin_n \
  -connect my_dout/dout my_oe/oe my_ac/ac_mode \
    my_bsc_p/bsc_data_p my_bsc_n/bsc_data_n my_clk/hyst_clk my_din/din \
    my_din_p/din_p my_din_n/din_n my_pin_p/pin_p my_pin_n/pin_n

module my_ac_bidi_diff_enable_low (
  my_pin_p,
  my_pin_n,
  my_dout,
  my_oe,
  my_din,
  my_din_p,
  my_din_n,
  my_ac,
  my_bsc_p,
  my_bsc_n,
  my_clk
);
  inout my_pin_p, my_pin_n;
  input my_dout, my_oe, my_ac;
  input my_bsc_p, my_bsc_n, my_clk;
  output my_din, my_din_p, my_din_n;
  specify
    specparam mgc_dft_cell_type = "ac_bidi_differential_pad";
    specparam mgc_dft_pin_type$my_pin_p = "pin_p";
    specparam mgc_dft_pin_type$my_pin_n = "pin_n";
    specparam mgc_dft_pin_type$my_dout = "dout";
    specparam mgc_dft_pin_type$my_oe = "oe";
    specparam mgc_dft_pin_type$my_din = "din";
    specparam mgc_dft_pin_type$my_din_p = "din_p";
    specparam mgc_dft_pin_type$my_din_n = "din_n";
    specparam mgc_dft_pin_type$my_ac = "ac_mode";
    specparam mgc_dft_pin_type$my_bsc_p = "bsc_data_p";
    specparam mgc_dft_pin_type$my_bsc_n = "bsc_data_n";
    specparam mgc_dft_pin_type$my_clk = "hyst_clk";
  endspecify
endmodule
```

Using a Pin Mapping File

A pin mapping file provides pin information and port ordering information that BSDArchitect can use instead of specifying certain application commands. You create this file external to the tool and read it into BSDArchitect using the [Load Pin Map](#) command. Executing a Run command after loading the pin mapping file causes BSDArchitect to consider both the loaded pin mapping descriptions and subsequently entered BSDArchitect commands when generating boundary scan circuitry.

Loading a pin mapping file can modify information you may have already supplied with BSDArchitect commands. Specifically, information from the pin map file overrides any port order defined with the [Set Port Order](#) command or any boundary scan cell types defined with the [Add Bscan Cell](#) command. If loading a pin file overrides previously specified information, BSDArchitect generates a warning message informing you that your previous definitions have been modified.

Pin Map File Contents

The pin mapping file contains the following information:

- **Pin order information**

This entry specifies boundary scan cell connection order. You can use top-level pin names in the pin mapping file. The pin specified on the first line of the pin mapping file connects to TDO, and each successive pin specified connects in the next highest position. Thus, BSDArchitect orders the boundary scan cells sequentially from TDO to TDI based on the pin order you specify in the pin map file.

If you do not specify all the pins in the pin mapping file, unspecified pins follow the specified pins in the boundary scan cell order. If you choose not to use a pin mapping file, you can instead specify this information with the Set Port Order command.

- **Device package pin mapping**

This entry specifies the mapping of logical signals to physical pins of a particular package. This information creates a PIN_MAP_STRING in the BSDL file.

- **Pad cell types**

This entry specifies the pad cell types you want to use with the pins in your design. If you choose not to use a pin mapping file, you can instead specify this information with the Add Pad Cell, Add Tristate Port, Add Differential Port, and Add Opencollector Port commands.

- **Boundary scan cell types**

This entry specifies the boundary scan cell types you want to use with the pins in your design. If you choose not to use a pin mapping file, you can instead specify this information with the Add Bscan Cell command.

- **Pad Inversion**

The pin map file also specifies if the pad used for a particular signal is an inverting pad or not. This is specified by using the -inv argument when specifying the *logical_port_name*.

Pin Map File Syntax

The following general information applies to the pin mapping file:

- Spaces, tabs, and blank lines are equivalent
- Comments begin with either `--` or `//`
- Information for each pin ends with a semicolon (`;`)

Each entry uses the following syntax:

logical_port_name ***PINIDs*** [-pwr | -gnd] [-p padcell_type] [-c cell_names] [-INV]
[-s X | 0 | 1] [-INTernal];

- ***logical_port_name***

A required string specifying the design signal name to which you want to assign the PINIDs.

For tri-state and bidirectional ports, you must specify the *logical_port_name* of the output port. Specify the information for the control port by using a separate entry with a value of “VOID” for the PINID field.

If the *logical_port_name* is TDI, TDO, TMS, TCK, or TRST then only the PINID and -p options are valid.

Signal names VDD and VSS are valid syntax for power and ground in the pin mapping file, and you can specify them multiple times. The tool also supports user-defined names for power and ground pins using the -pwr and -gnd switches.

- ***PINIDs***

A required repeatable integer or alphanumeric identifier specifying the actual pin name or set of pin names that you want to assign to the *logical_port_name*. Separate multiple PINIDs with a space.

You can use multiple PINIDs fields per line each separated by a space. This allows you to specify a vector using individual bits. For example, the following PINID's are valid, and both represent the same information:

```
a(0) A0 -p my_pad;  
a(1) A1 -p my_pad;  
a(2) A2 -p my_pad;
```

or

```
a A0 A1 A2 -p my_pad my_pad my_pad
```

For tri-state and bidirectional ports, only one entry should have a PINID field (unless specified bit by bit in the map file). If an entry contains more than one PINID field, the BSDL file will contain multiple entries for the same port in the PIN_MAP attribute.

- **-p *padcell***

An optional switch and string pair that specifies the exact pad cell you want to place on the specified port.

- **-c *cell_names***

An optional switch and repeatable string pair that lists the boundary scan cell (two cells, if a bidirectional pin) you want to connect to the specified pin.

This switch is not valid when using JTAG ports (TDI, TDO, TMS, TCK, or TRST).

The boundary scan *cell_names* is an optional field. It may be an ASIC library cell name or BC_1, BC_2, BC_2_A, BC_2_A_EXT, BC_2_B, BC_3, BC_4, BC_5, BC_7 or BC_7_LOW, BC_8, BC_9, or BC_10. You must use a *logical_port_name* whenever you specify a boundary scan cell name.

- **-INV**

An optional switch to specify the input data that the pad used with the specific port is inverted at the output. Note that you must issue the “Set Pad Inversion ON” command in order for the BSDArchitect to consider the inversion when generating a test bench.

- **-s X | 0 | 1**

An optional switch that specifies the safe value associated with the *logical_port_name*. By default, for tri-state and bidirectional boundary scan cells, the safe value is the inactive value of the corresponding control cell. For boundary scan control cells, the safe value is the inactive value of that control cell. For all other types of boundary scan cells, the safe value is X.

This switch is not valid when using JTAG ports (TDI, TDO, TMS, TCK, or TRST).

- **-INternal**

An optional switch that specifies that a particular port is an internal port.

Internal ports are those core ports that contain a boundary scan cell on them, but those ports do not come to the top.

- **-pwr | -gnd**

An optional switch that declares the value of the *logical_port_name* argument as either power or ground. In the resulting BSDL file, these ports will be listed as linkage type. This switch is not valid when using JTAG ports (TDI, TDO, TMS, TCK, or TRST). BSDArchitect does not place a boundary scan cell on power and ground pins.

Existing Power or Ground Pins:

If the power or ground pin exists on the core, then the tool places a generic I/O pad cell, or a specific technology I/O pad cell that you specified with the [Add Technology Cell](#) command using the POWER option.

The following example shows usage of the user-defined names for power and ground (Note that the technology cell named *power_cell* had to have been previously defined with the Add Technology Cell command using the POWER option):

```
my_pwr      71  -pwr
my_gnd      72  -gnd
custom_pwr  73  -pwr  -p power_cell
VDD         74
VDD         75
VSS         76
VSS         77
```

BSDArchitect generates the following BSDL file when using the preceding declarations:

```
port (
    .
    .
    .
    vss : linkage bit_vector(1 downto 0);
    my_gnd : linkage bit ;
    vdd : linkage bit_vector(1 downto 0);
    my_pwr : linkage bit;
    custom_pwr : linkage bit;
    .
    .
    .
);

constant DW: PIN_MAP_STRING:=
    "vss: (76, 77)," &
    "my_gnd: (72)," &
    "vdd: (74, 75)," &
    "my_pwr: (71)," &
    "custom_pwr: (73)";
```

Non-existent Power or Ground Pins:

If you specify *-pwr* or *-gnd*, as well as *-p padcell_type* for a pin that does not exist on the core, the tool adds a dummy I/O pad in the pad ring block.

Note



If you specify *-pwr* or *-gnd* without “*-p padcell_type*”, the tool will not add a pad unless the power or ground pin already exists on the core.

The following is an excerpt from a pin mapping file specifying “*-p pwr_pad -pwr*” for a nonexistent pin:

```
my_vdd      3  -p pwr_pad -pwr ;
```

BSDArchitect generates the following output files when using the preceding declaration:

RTL generated:

```

module pad_instance
...
pwr_pad pwrSIG ( .my_vdd ( my_vdd ));
...
The following shows the resultant BSDL file:
port (
...
my_vdd : linkage bit;
...
);

constant DW: PIN_MAP_STRING:=
...
"my_vdd: (3)," &
...

```

Pin Mapping Example

The following is the default BSDArchitect pinmap file (*my_pinmap*) that the dofile for this example generated:

```

-----
--          BSDArchitect Pin Map  Report
-----
-- Design : test
-----
-- Port Name      PinID    Cell Type      Pad Cell Name    Safe Value      Inv
-----
bit_bidi_ctr      void      -cell bc_2_a          ;
bit_bidi_o         0      -cell bc_7_low      -safe X          ;
bit_diff_o        3 4      -cell bc_1          -safe X          ;
bit_int_o         7      -cell bc_1          -safe X          ;
bit_o             9      -cell bc_1          -safe            ;
bit_oc_o          10      -cell bc_1          -safe X          ;
bit_tri_ctrl      void      -cell bc_1          ;
bit_tri_o         12      -cell bc_1          -safe X          ;
bit_diff_i        1 2      -cell bc_1          -safe X          ;
bit_gnd           5      -cell bc_1          -safe X          ;
bit_i             6      -cell bc_1          -safe X          ;
bit_inv_i         8      -cell bc_1          -safe X          ;
bit_pwr           11      -cell bc_1          -safe X          ;
tdi               13                        ;
tms               14                        ;
tck               15                        ;
tdo               16                        ;
trst              17                        ;
-----

```

The following pinmap file is a manually edited version of the default file generated by BSDArchitect (*my_pinmap_modified*) that the dofile for this example generated:

```

-----
--BSDArchitect Pin Map Report
-----
--Design : test
-----
--Port Name      PinID      Cell Type      Pad Cell Name      Safe Value      Inv
-----
bit_bidi_ctrl void      -cell bc_2_a      ;
bit_bidi_o       0        -cell bc_7_low    -safe X           ;
bit_diff_o       3 4      -cell bc_1        -safe X           ;
bit_int_o        7        -cell bc_1        -safe X           -internal;
bit_o            10       -cell bc_1        -pad outpad       -safe 0           ;
bit_oc_o         9        -cell bc_1        -safe X           ;
bit_tri_ctrl void      -cell bc_1        ;
bit_tri_o        12       -cell bc_1        -safe X           ;
bit_diff_i       1 2      -cell bc_4        -safe X           ;
bit_gnd          5        -gnd              ;
bit_i            6        -cell bc_2        -pad inpad        -safe X           ;
bit_inv_i        8        -cell bc_1        -pad inpad_inv    -safe X           -inv  ;
bit_pwr          11       -pwr              ;
tdi             13              ;
tms             14              ;
tck             15              ;
tdo             16              ;
trst            17              ;
-----

```

Using User-Defined Instructions

BSDArchitect allows you to use user-defined instructions. To do so, you must define a register for a given user-defined instruction before using the Add Bscan Instruction command to define the user-defined instruction.

To define a register use the following command:

ADD COre Register **register_name** **input_port_name** **output_port_name** [-Top | -Notop]
[-Length integer] [-Inv]

The *input_port_name* and *output_port_name* arguments are core logic ports. If you define a port as a register for a user-defined instruction, BSDArchitect makes the port available at the top-level of the design by default. To remove these ports from the top level, use the -notop argument. Boundary scan cells will not be used for these input and output ports if you issue either the -notop argument or the Add Bscan Cell none command for these ports.

After defining a register, you can add an instruction for that register by using the following command:

ADD BScan Instruction **instr_name** {-Reg_name **reg_name**} [-Type *instruction_type*]
[-Code *opcode*] [-Outputs {Hi_impedance | Cell_value}]

The *instr_name* argument is the instruction and the *reg_name* is the name of the register you defined with the Add Core Register command.

Once you have defined a register and its associated user-defined instruction, BSDArchitect generates a signal with the same name as the user-defined instruction. This signal becomes active when you load the instruction in the instruction register.

BSDArchitect generally does not have information about the clock and enable signals associated with a user-defined register (the internal scan and multiple scan instructions are the exception). BSDArchitect normally connects the register between TDI and TDO when you load the instruction in the instruction register. So, you need to manually connect the clock and enable signals for the register.

The user-defined internal scan and multiple scan instructions are used to denote a particular category of instructions (internal scan and internal scan + bsr, respectively). The names of these instructions are not required to be INT_SCAN and MULT_SCAN. For these instructions, BSDArchitect automatically connects the clock and enable signals.

Additionally, you can use the Add Port Connection command to connect a core-design port either to a top-level boundary scan port, to internal boundary scan signals, or both.

The following is a simple example of how to use a user-defined instruction. In this example we'll show how to define a register named, `pcnt_reg`, add an associated instruction called, `lbp cnt`, and then connect the BSDArchitect-generated `lbp cnt` signal to a core logic port named, `lbp cnt_b`:

```
add core register pcnt_reg patc_sin patc_sout -length 7  
add bscan instruction lbp cnt -reg pcnt_reg  
add port connection lbp cnt_b buf lbp cnt
```

The `lbp cnt` signal connects to a core logic port named `lbp cnt_b` when the `lbp cnt` instruction is loaded in the instruction register.

You can issue additional Add Port Connection commands to connect the clock and any other signals. Valid *signal_names* include all TAP controller signals, instruction enables, core input ports, core output ports, and available intermediate signals at the TAP level. The *runbist* signal name is valid if the RUNBIST instruction has been added via the Add Bscan Instruction command. For a list of valid signal names, issue the Report Port Connection command with the `-Usable_signals` switch.

For a full description, refer to the [Add Port Connection](#) command in the *BSDArchitect Reference Manual*.

Connecting Internal Scan Circuitry

Board-level interconnect testing does not require access to internal scan chains. It simply tests the interconnection circuitry between chips on the board. However, chip-level testing and chip-testing at the board level both require internal scan access. Thus, if your test strategy includes chip-test at the board level, and you are not using BIST circuitry, you should connect the internal scan circuitry to the boundary scan circuitry.

BSDArchitect can connect the internal scan circuitry to the boundary scan circuitry, and gives you several options to choose from as you decide how to connect the circuitry together. The following subsections describe these options and show an example of connecting the boundary scan circuitry to internal scan.

Adding Dummy Scan Pins

An optional flow is to use BSDArchitect at the RTL-level. At this point, your design most likely does not contain internal scan circuitry. However, if you know the internal scan strategy you are going to use, you can add dummy scan ports for the scan ports that will eventually be part of the design.

The following example contains the entity of the example design “aha” before the addition of scan insertion. If this design is to contain scan, you would need to add scan ports before using this entity with BSDArchitect.

```
library ieee;
use ieee.std_logic_1164.all;
entity aha is
    port( s:out std_ulogic;
          co:out std_ulogic;
          a:in std_ulogic;
          b:in std_ulogic;
          c:in std_ulogic);
end aha;
```

For a strategy requiring two scan chains, the entity would look like the following example. This example assumes that *aha_scan2.vhd* is the name of the design entity you are using and that the design contains two internal scan chains.

```
library ieee;
use ieee.std_logic_1164.all;

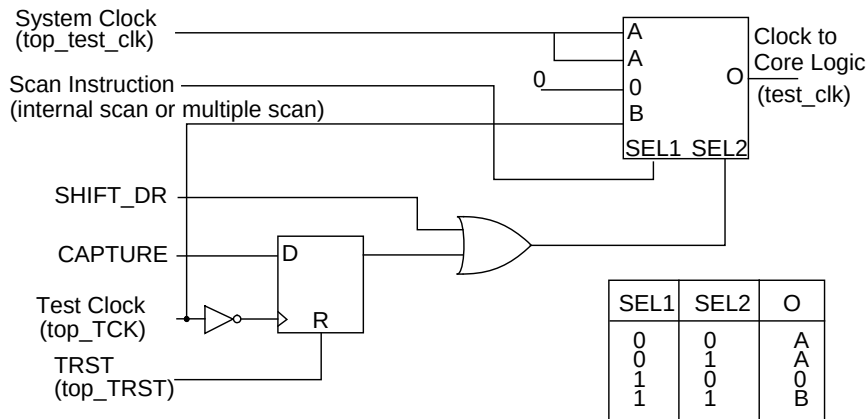
entity aha_scan2 is
    port( s:out std_ulogic;
          co:out std_ulogic;
          a:in std_ulogic;
          b:in std_ulogic;
          c:in std_ulogic;
          sclk:in std_ulogic; -- Scan Clock
          sc_en:in std_ulogic; -- Scan Enable
          sc1_i:in std_ulogic; -- Scan Input Pins
          sc2_i:in std_ulogic;
          sc1_o:out std_ulogic; -- Scan Output Pins
          sc2_o:out std_ulogic);
end aha_scan2;
```

Specifying the Scan Architecture and Testing Mode

Mux-DFF and clocked-scan are the two supported internal scan architectures. BSDArchitect does not support LSSD.

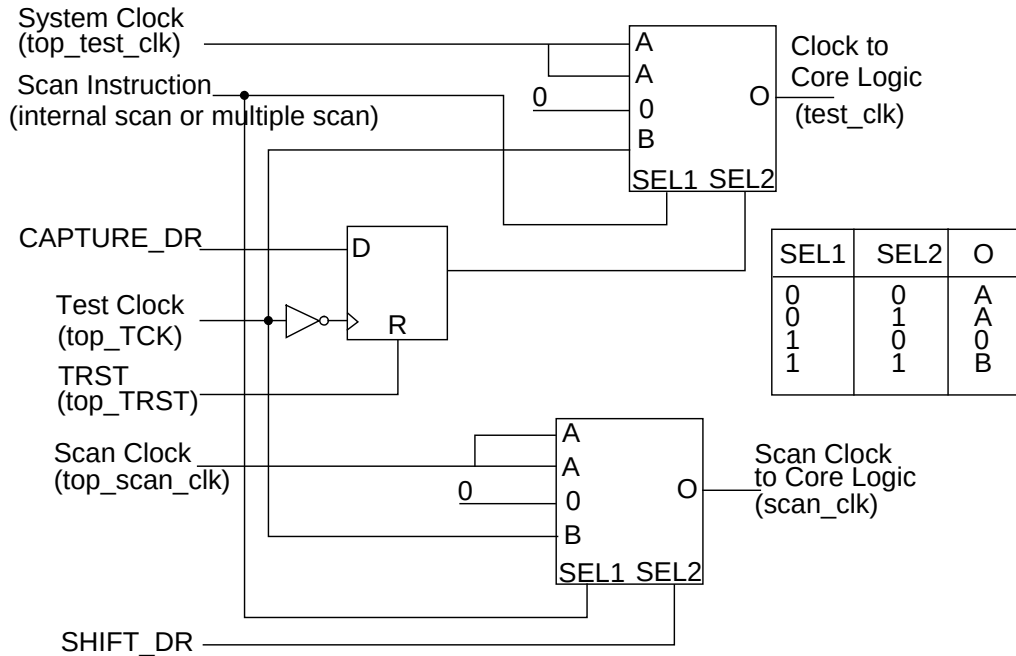
You must specify which type of internal scan architecture your design uses. During the connection to boundary scan circuitry, BSDArchitect creates clocking circuitry depending on the specified internal scan architecture. [Figure 4-35](#) shows the boundary scan architecture created for clocking the internal scan circuitry of a mux-DFF design.

Figure 4-35. Clocking Circuitry Created for Mux-DFF Architecture



In [Figure 4-35](#), the multiplexer connects the system clock to the core logic clock before loading the scan instruction in the instruction register. For more information on scan instructions, see [“Defining the Scan Instruction”](#) on page 123. After the scan instruction loads in the instruction register, the TAP causes the connection of TCK to the core logic clock during the CAPTURE and SHIFT_DR states.

[Figure 4-36](#) shows the boundary scan architecture created for clocking the internal scan circuitry of a clocked-scan design.

Figure 4-36. Clocking Circuitry Created for Clocked Scan Architecture

In [Figure 4-36](#), before loading the scan instruction (internal scan or multiple scan) in the instruction register, both the system clock and the scan clock connect to the core logic clock. After the scan instruction loads in the instruction register, TCK connects to the core logic clock during the TAP controller CAPTURE_DR state and the scan clock connects to the core logic during the SHIFT_DR state.

When you choose the internal scan architecture, you must also decide the mode in which you are going to test the chip. If you want the ability to exercise the chip's internal scan circuitry by using the internal scan ports directly, you should choose *standalone* mode. If you choose the standalone testing mode, BSDArchitect creates extra circuitry to give you the option of direct access to the internal scan ports at the top-level of the design. Standalone mode provides the most efficient method for internal chip testing.

If you only want access to the internal scan circuitry via the TAP, and you do *not* want BSDArchitect to add the extra circuitry for the option of testing in standalone mode, you should choose *nostandalone* mode. Nostandalone mode provides a means to restrict internal chip access, except through the use of boundary scan circuitry, as some testers may require.

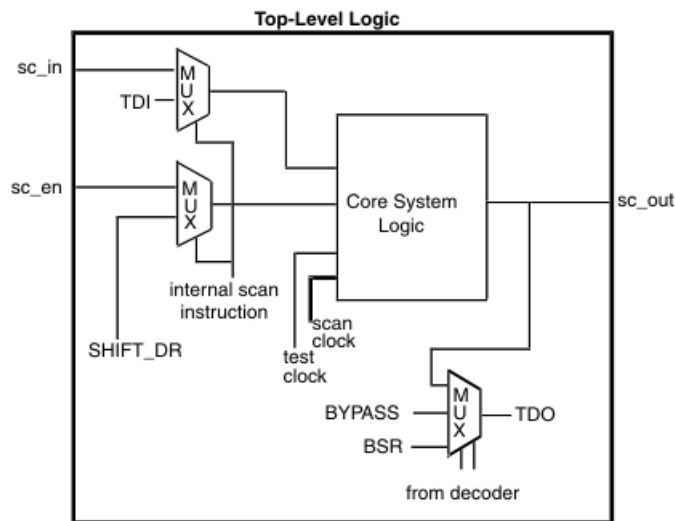
Note



If using nonstandalone scan architecture, a fault at the reset pin for the scan cells will not be detected. This is because for that architecture only one clock can be defined during capture in Tessent FastScan: the test clock by using “Add Capture Clock test_clk -atpg”. In order to detect a fault at the reset pin, the reset pin needs to be defined as a clock, but this is not allowed when using nonstandalone scan architecture.

The default circuitry allows testing in standalone mode, as shown in [Figure 4-37](#).

Figure 4-37. Default Architecture for Testing Mode



[Figure 4-37](#) shows the default connections for a core design containing a single scan chain. The circuitry shown allows you to access the internal scan chain either directly (through `sc_in`, `sc_out`, and `sc_en`) or by using TAP signals. The thin lines show circuitry that is valid if your design uses the mux-DFF architecture. The clock signal shown with the heavier line is only valid if your design uses the clocked-scan architecture.

To specify the internal scan architecture and testing mode, you use the Set Iscan Interface command. For example:

```
set iscan interface -type mux_scan -sen sc_en -mode stand_alone -tclk sclk
```

This example specifies that the type of scan architecture is mux-DFF, that the testing mode should allow for standalone testing (direct access to the scan chain, without going through the TAP), that the scan enable is the `sc_en` port, and that the test clock is the `sclk` port.

Specifying the Internal Scan Chain

If you want BSDArchitect to access the internal scan chain, you must define a register name and the input and output ports for the register. The following is an example of specifying an internal scan chain using the Add Core Register command:

```
add core register int_scan_reg sc1_i sc2_o
```

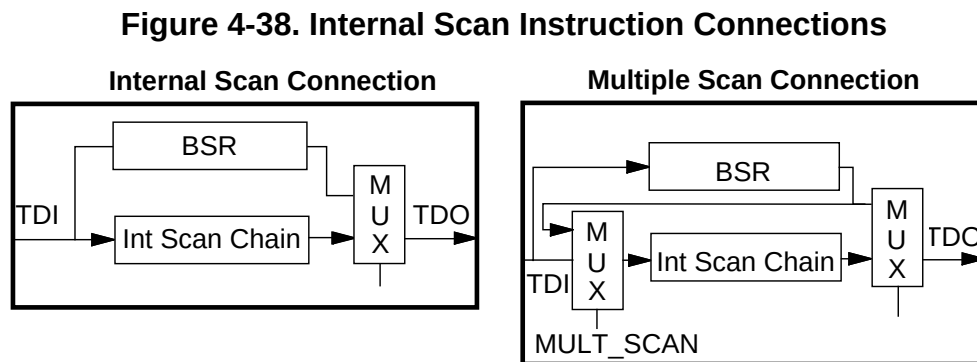
This command gives the name “int_scan_reg” to the combined chain bounded by the input port sc1_i and the output port sc2_o.

Defining the Scan Instruction

The Scan Instruction signal shown in Figures 4-35, 4-36, and 4-37 can be any scan instruction name so long as it is defined as either an internal or multiple scan instruction. These are user-defined instructions for the purpose of accessing the internal scan circuitry. You must first define a register using the Add Core Register command. You then add the instruction and associate it with the defined register using the Add Bscan Instruction command.

The internal scan instruction connects the internal scan register between the TDI and TDO ports. The multiple scan instruction connects both the boundary scan register (BSR) and the internal scan register in series between TDI and TDO.

Figure 4-38 shows simplified circuit diagrams of the internal scan and multiple scan instructions.



The circuitry created for the multiple scan instruction includes a multiplexer, controlled by the multiple scan signal, at the input of the internal scan chain. One input of the mux is the output of the BSR and the other input is TDI. If you want to use the standalone mode for testing, BSDArchitect adds an additional mux to connect the internal scan chain with a primary input.

For example, to connect both the internal scan chain and boundary scan register in series between TDI and TDO, you define the following multiple scan instruction:

```
add bscan instruction mult_scan -reg_name scan_reg -mult_scan_instr
```

Note that adding instructions with the internal scan or multiple scan instruction types require the use of the `-reg_name` argument to define the internal scan register, because neither the Connect Iscan Chains nor the Add Core Register command provides this information.

Combining Multiple Chains

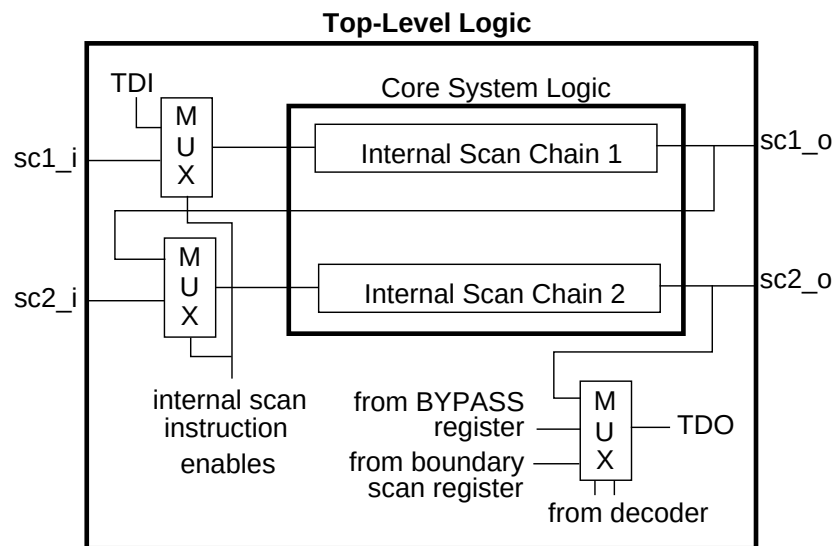
BSDArchitect can handle multiple internal scan chains, but you must first combine them into a single scan chain, as shown in [Figure 4-39](#).

You can connect multiple scan chains into a single chain by using the Connect Iscan Chains command. All internal scan chain related ports have associated boundary scan cells during standalone testing. In no-standalone mode, these ports do not appear at the top level and do not have associated boundary scan cells. The following example shows how to connect two scan chains into a single chain:

```
connect iscan chains sc1_i sc1_o sc2_i sc2_o
```

The input and output ports of the first scan chain are `sc1_i` and `sc1_o`, respectively. The input and output ports of the second scan chain are `sc2_i` and `sc2_o`, respectively. This command connects the first scan chain to the second scan chain so that the input of the combined chain is `sc1_i` and the output of the combined chain is `sc2_o`.

Figure 4-39. Connection of Multiple Scan Chains



Chapter 5

Output Testing and Verification

This chapter outlines how to use BSDArchitect to synthesize and verify boundary scan circuitry for your design. For specific information on BSDArchitect functionality or commands, refer to the [BSDArchitect Reference Manual](#).

Understanding BSDArchitect Output Files

The BSDArchitect tool can create a number of different output files. The following sections describe each file. By default, when you save your design with the [Save Bscan](#) command, the tool generates the following:

- [HDL Description](#)
- [Top-Level Interface Description](#)
- [HDL Test Bench](#)
- [BSDL Description](#)

Additional commands are available to create the following:

- [Synthesis Script](#)
- [Parametric Tests](#)
- [Test Vectors](#)
- [ATPG Setup Files](#)

HDL Description

The HDL description file contains the design with inserted boundary scan circuitry. This file is created by the Save Bscan command. You can specify a buffered or unbuffered, mapped or unmapped HDL description. You may want to specify a mapped file if your design uses technology-mapped cells or you are using the Internal flow.

The default file name of an unmapped HDL description is `<prefix>_bscan.<suffix>`, where `<prefix>` is the design name specified at invocation and `<suffix>` is the design format extension. The default file name for a mapped file is `<prefix>_map.<suffix>`. You can specify different file names using the [Setup File Naming](#) command.

In general, the mapped output uses IEEE 1149.1 cells from the specified ASIC library. This output contains direct instantiation of ASIC library cells. For more information, refer to “[Technology Mapping](#)” on page 62.

Note



The HDL description file is not created in the BSDL flow.

Top-Level Interface Description

The top-level interface description file contains a black box model with the inputs and outputs for the top-level boundary scan core entity/module. This file is created by the Save Bscan command.

The default file name of the top-level interface description is `<prefix>_top.<suffix>`, where `<prefix>` is the design name specified at invocation and `<suffix>` is the design format extension. You can specify a different file name using the [Setup File Naming](#) command.

HDL Test Bench

The HDL test bench file contains tests for the boundary scan logic functionality at the RTL level to verify that the generated boundary scan logic is correct. The parametric tests will be included in this file by default, but you can choose to have BSDArchitect write these test into a separate file. For more information, see “[Parametric Tests](#)” on page 128. The HDL test bench is created by the Save Bscan command.

The default file name of the HDL test bench is `<prefix>_tb.<suffix>`, where `<prefix>` is the design name specified at invocation and `<suffix>` is the design format extension. You can specify a different file name using the [Setup File Naming](#) command.

The test bench does *not* test the BSDL, check for full compliance with the IEEE 1149.1 standard, or fault grade the boundary scan circuitry. For more detailed information on the test bench functions, see “[Troubleshooting the Test Bench Simulation](#)” on page 144.

The test bench performs the following tests:

- **Verification of User-Defined Instructions**

The test bench verifies all user-defined instructions (defined by the [Add Bscan Instruction](#) command) that target core registers, external registers, and the boundary scan register.

- **Verification of Public Instructions**

The test bench verifies the following predefined instructions: SAMPLE, PRELOAD, BYPASS, EXTEST, IDCODE, USERCODE, CLAMP, and HIGHZ. It does not test the INTEST or RUNBIST instructions.

When AC pins are used, the test bench also verifies the instructions EXTEST_PULSE and EXTEST_TRAIN.

- **Verification of Boundary Scan Register Operation**

The test bench verifies the shift, capture, update, and mode capabilities of the boundary scan registers.

- **Verification of Instruction Logic**

The test bench verifies the load and unload capability of the instruction register. It also tests the instruction decode circuitry.

- **Verification of TAP Controller Transitions**

The test bench implicitly verifies the TAP controller transitions during the testing of the public instructions. The test bench also explicitly does some testing of other transitions.

- **Verification of the Reset Operation**

The test bench verifies the reset operation by resetting the design, unloading the data register, and checking the unloaded data.

Additional test bench files (that contain a subset of the original test bench) can be created using the [Save Testbench](#) command.

BSDL Description

The BSDL description file contains the design with boundary scan circuitry written in BSDL. It can be used with board testing tools. This file is created by the Save Bscan command.

The default file name of the BSDL description file is `<prefix>_bscan.bsd`, where `<prefix>` is the design name specified at invocation. You can specify a different file name using the [Setup File Naming](#) command.

Synthesis Script

BSDArchitect can generate a synthesis script for synthesizing the HDL output file from BSDArchitect. This file is not written out in the BSDL flow. Use the optional -Synthesis_script switch for the [Save Bscan](#) command. The script can use either Design Compiler or Leonardo. You can specify a file name or use the default name:

- `<prefix>_leonardo_syn.do` for a Leonardo script
- `<prefix>_dc_syn.do` or `<prefix>_dc_syn_tcl.do` for a Design Compiler script

Parametric Tests

Parametric tests use parametric measurement units to ensure that the I/O hardware works within specified limits. I/O parametric tests are important because they test the ability of a chip to work correctly when placed in an environment with other chips.

BSDArchitect generates the following two classes of parametric tests: [Typical Pattern Set](#) and [Checkerboard Pattern Set](#).

To generate these tests, issue the following command *before* inserting the boundary scan circuitry (issuing the Run command):

```
set testbench parameters -parametric_test typical
```

The parametric tests are included in the HDL test bench file and the test vectors file. You can also write the parametric test into a separate test bench file using the Save Testbench command. For more information, refer to the [Set Testbench Parameters](#) command in the *BSDArchitect Reference Manual*. To write the parametric test vectors to a file, you can use the -Parametric switch with the [Save Patterns](#) command.

Typical Pattern Set

In a typical pattern set, the tests use all low voltages (0), all high voltages (1), and highz (z) as follows:

- **VIH Test** — Apply all 1s at primary inputs and capture at bscan cells.
- **VIL Test** — Apply all 0s at primary inputs and capture at bscan cells.
- **VOH Test** — Apply all 1s from bscan cells to primary outputs.
- **VOL Test** — Apply all 0s from bscan cells to primary outputs.

Checkerboard Pattern Set

For the checkerboard pattern set, the tests use checkerboard patterns. For example, “0101010...” and “1010101...”. In this set, the tool also generates highz patterns. Additionally, the test performs bridge testing between pads and ports.

Test Vectors

You can write the test bench file as test vectors with the [Save Patterns](#) command.

ATPG Setup Files

If the circuit contains internal scan circuitry controlled by the boundary scan circuitry, you can use BSDArchitect to generate the following two ATPG setup files:

- **Dofile** — Used for setting up scan circuitry information.
- **Test procedure file** — Used for specifying the operation of the scan circuitry. The “[Test Procedure Files](#)” section in the *Scan and ATPG User’s Manual* discusses test procedure files in more detail.

The file names of the ATPG setup files are `<root_name>.testproc` and `<root_name>.dofile`, where `<root_name>` is specified with the [Save Atpg Setup](#) command.

This capability assumes the TAP controller conforms to IEEE 1149.1-2001 standard and that the design contains a single internal scan chain. This would be the case if you use BSDArchitect to create the boundary scan circuitry and connect it to the internal scan circuitry.

To save ATPG setup files, issue the following command:

```
save atpg setup mux_scan_stand
```

In this case, the root name of the files will be “mux_scan_stand.” BSDArchitect produces the following four files:

- `mux_scan_stand_standalone.dofile`
- `mux_scan_stand_standalone.testproc`
- `mux_scan_stand_nostandalone.dofile`
- `mux_scan_stand_nostandalone.testproc`

Using BSDArchitect in a BSDL Flow

Use this procedure to check your BSDL file against the IEEE 1149.1 standard and generate the output files.

Prerequisites

- Your design file must be written in BSDL format.

Procedure

1. Invoke [BSDArchitect](#) including the `-Bsd` switch.

```
shell> <Tessent_Tree_Path>/bin/bsdarchitect test.bsd -bsd
```
2. (Optional) Make any customizations to the test bench. For more information, refer to the [Set Testbench Parameters](#) command.
3. Check the boundary scan circuitry for compliance to the standard by issuing the `run` command.

```
run
```

4. Save your design and write out the test bench. For more options on output files, see [“Creating Output Files”](#) on page 135. BSDArchitect will not write an HDL description file in the BSDL flow.

save bscan -replace

5. Exit BSDArchitect.

exit

Related Topics

[Verifying the HDL Boundary Scan Circuitry](#)

[Troubleshooting the Test Bench Simulation](#)

Inserting Boundary Scan Circuitry

Use this procedure to check your design for errors and insert the specified boundary scan circuitry. The boundary scan circuitry will only be added if no errors are found. Only one successful boundary scan insertion is allowed per BSDArchitect session. To perform another insertion, reinvoke BSDArchitect or use the Reset State command.

Prerequisites

- You must have specified any customizations for your design and output files. For more information on available customizations, refer to [“Boundary Scan Customizations”](#) on page 61 and [“Understanding BSDArchitect Output Files”](#) on page 125.

Procedure

1. Issue the Run command. BSDArchitect will check your design for errors and insert boundary scan circuitry if no errors are found.

run

2. Verify the boundary scan circuitry by issuing the following two commands after the Run command completes successfully:

report bscan status
report bscan cell

```
-----  
                          BSDArchitect  Report  
-----  
Design : my_mod  
-----  
Asynchronous TAP Reset : Yes  
Device Identification Register : No  
User Identification Register : No  
Instruction Register Width : 4
```

INSTRUCTION REPORT:

Instruction	Target Register	Opcode
EXTEST	BOUNDARY	0000
BYPASS	BYPASS	1111
SAMPLE	BOUNDARY	0001
PRELOAD	BOUNDARY	0001

PORT REPORT:

No.	Top-Port	BSC	Mode	Type	Cell Name	BSR Instance	Internal
1	EN1	YES	Both	bc_1	bc_1	bsr_i1	NO
2	OUT_IO[3]	YES	Both	bc_1	bc_1	bsr_i1	NO
3	OUT_IO[2]	YES	Both	bc_1	bc_1	bsr_i1	NO
4	OUT_IO[1]	YES	Both	bc_1	bc_1	bsr_i1	NO
5	OUT_IO[0]	YES	Both	bc_1	bc_1	bsr_i1	NO
6	PAD_OUT[1]	YES	Both	bc_1	bc_1	bsr_i1	NO
7	PAD_OUT[0]	YES	Both	bc_1	bc_1	bsr_i1	NO
8	Q	YES	Both	bc_1	bc_1	bsr_i1	NO
9	CL	YES	Both	bc_1	bc_1	bsr_i1	NO
10	D	YES	Both	bc_1	bc_1	bsr_i1	NO
11	IN_IO[3]	YES	Both	bc_1	bc_1	bsr_i1	NO
12	IN_IO[2]	YES	Both	bc_1	bc_1	bsr_i1	NO
13	IN_IO[1]	YES	Both	bc_1	bc_1	bsr_i1	NO
14	IN_IO[0]	YES	Both	bc_1	bc_1	bsr_i1	NO
15	PAD_IN[1]	YES	Both	bc_1	bc_1	bsr_i1	NO
16	PAD_IN[0]	YES	Both	bc_1	bc_1	bsr_i1	NO

Total no. of BS cells : 16

Note: Port ordering is from TDO to TDI.

Related Topics

[Creating Output Files](#)

Defining a Test Setup Procedure

Use this procedure to set up an initialization preamble for boundary scan test patterns with a [test_setup](#) procedure in the test procedure file.

Prerequisites

- JTAG circuitry is inserted. For more information, see “[Inserting Boundary Scan Circuitry](#)” on page 130.
- A hand-written test procedure file containing a test setup procedure. For more information, see the [Test_Setup \(Optional\)](#) section in the *Tessent Common Resources Manual for ATPG Products*.
- The following is an example of a test setup procedure:

```
set time scale 1.0 ns ;
timeplate tpo =
    force_pi 0 ;
    measure_po 190 ;
    pulse TCK 100 100 ;
    period 200 ;
end;

procedure test_setup =
timeplate tp0;
// Apply reset procedure
// Test cycle one
cycle =
    force TMS 1;
    force TDI 0;
    force TRST 0;
    pulse TCK;
end;
```

Procedure

1. Generate test vectors. For example:

run

2. Read the test procedure file into BSDArchitect. For example:

read procfile procfile

3. Save test vectors that contain the test_setup procedure. For example:

save pattern pattern1.wgl -format wgl -rep
save pattern pattern1.stil -format stil -rep -parameter parameter

The test_setup events are added in the beginning of the pattern set and also in the beginning of the native testbench (Verilog/VHDL).

4. Save the RTL, testbench, BSDL and other output files as needed.

Related Topics

[Read Procfile](#)

[Write Procfile](#)

Example Test Setup for STIL Test Vectors

The following example illustrates a test_setup procedure and the resulting STIL test vectors and test bench.

Following is a test procedure file with a setup_test procedure defined:

```
set time scale 1 ns;
set strobe_window time 1;
timeplate tp0 =
force_pi 100;
measure_po 200;
pulse TCK 300 100;
period 500;
end;

procedure test_setup =
timeplate tp0;
// Apply reset procedure
// Test cycle one
cycle =
    force TMS 1;
    force TDI 0;
    force TRST 0;
    pulse TCK;
end;

// "TMS"=0 change to run-test-idle
// Test cycle two
cycle =
    force TMS 0;
    force TRST 1;
    pulse TCK;
end;

// "TMS"=1 change to select-DR
// Test cycle three
cycle =
    force TMS 1;
    pulse TCK;
end;

// "TMS"=1 change to select-IR
// Test cycle four
cycle =
    force TMS 1;
    pulse TCK;
end;

// "TMS"=0 change to capture-IR
// Test cycle five
cycle =
    force TMS 0;
    pulse TCK;
end;

// "TMS"=0 change to shift-IR
// Test cycle six
cycle =
    force TMS 0;
```

```
        pulse TCK;
    end;
    // load MULT_SCAN instruction "1000" in IR
    // Test cycle seven
    cycle =
        force TMS 0;
        pulse TCK;
    end;
    // Test cycle eight
    cycle =
        force TMS 0;
        pulse TCK;
    end;
    // Test cycle nine
    cycle =
        force TMS 0;
        pulse TCK;
    end;

    // Last shift in Exit-IR Stage
    // Test cycle ten
    // Change to shift-dr stage for shifting in data
    // "TMS" = 11100
    // "TMS"=1 change to update-IR state

    cycle =
        force TMS 1;
        force TDI 1;
        pulse TCK;
    end;
    // Test cycle eleven
    cycle =
        force TMS 1;
        force TDI 1;
        pulse TCK;
    end;
    // "TMS"=1 change to select-DR state
    // Test cycle twelve
    cycle =
        force TMS 1;
        pulse TCK;
    end;
    // "TMS"=0 change to capture-DR state
    // Test cycle thirteen
    cycle =
        force TMS 0;
        pulse TCK;
    end;
    // "TMS"=0 change to shift-DR state
    // Test cycle fourteen
    cycle =
        force TMS 0;
        force TEST_MODE 1;
        force RESETN 1;
        pulse TCK;
    end;
end;
```

If this test_setup is read into BSDArchitect and STIL patterns are saved in structural format using parameter STIL_STRUCTURAL 1, the following test_setup section is generated in the STIL pattern file:

```
Procedures {
  "test_setup" {
    W tset_tp0;
    C { _po_ = \r18 X ; tck = 0;      }
    V { tdi = 0; tms = 1; trst = 0; tck = 1;      }
    V { tms = 0; trst = 1; tck = 1;      }
    V { tms = 1; tck = 1;      }
    V { tms = 1; tck = 1;      }
    V { tms = 0; tck = 1;      }
    V { tms = 0; tck = 1;      }
    V { tms = 0; tck = 1;      }
    V { tms = 0; tck = 1;      }
    V { tms = 0; tck = 1;      }
    V { tdi = 1; tms = 1; tck = 1;      }
    V { tdi = 1; tms = 1; tck = 1;      }
    V { tms = 1; tck = 1;      }
    V { tms = 0; tck = 1;      }
    V { TEST_MODE = 1; RESETN = 1; tms = 0; tck = 1;      }
  }
}
```

The equivalent cycles are present in WGL vectors as well.

Creating Output Files

Use this procedure to create the output files for your design. Many of the steps in this procedure are optional, depending on the types of files you want. For more information on the output file types and options, refer to “[Understanding BSDArchitect Output Files](#)” on page 125.

Prerequisites

- You must have previously issued the Run command without errors.

Procedure

1. (Optional) Generate the test vectors.
save patterns filename -format wgl -replace
2. Save the design and generate the default output files (HDL description, top-level interface description, HDL test bench, and BSDL description).
save bscan -replace
3. (Optional) Generate additional test bench files for a subset of test bench functionality.
save testbench -parametric -file parametric_tb.v -replace

4. (Optional) Generate the ATPG setup files.
save atpg setup file_name_root -replace
5. Exit BSDArchitect.
exit

Related Topics

[Verifying the HDL Boundary Scan Circuitry](#)

Verifying the HDL Boundary Scan Circuitry

Use this procedure to verify that the boundary scan circuitry functions as you expect. The examples in this section use ModelSim and Design Compiler. Where applicable, the different Verilog and VHDL design examples are shown.

Prerequisites

- You must have completed boundary scan insertion, created all output files, and exited BSDArchitect.
- You will need the following files for verification: technology-mapped or component libraries (if applicable), core logic HDL model, boundary scan HDL model, and HDL test bench.

Procedure

1. Compile the mapped libraries. Any libraries used need to be compiled prior to the BSDArchitect files. The following example shows how to compile the library using ModelSim from the directory containing the HDL source of the target technology (assuming the *modelsim.ini* file contains a mapping between the *h4c_lib* label and the location *h4c*, which is the compiled library destination):

```
shell> <modelsim tree>/bin/vlib h4c
shell> <modelsim tree>/bin/vmap h4c_lib h4c
shell> <modelsim tree>/bin/vcom -work h4c_lib -check_synthesis
h4c.vhd
```

Note



If you have custom pads in your design, first compile the BSDArchitect-generated source code and then compile the libraries.

2. Define your work directory (for placing compiled boundary scan models).

```
shell> cd /user/jdoe/bs
shell> <modelsim tree>/bin/vlib work
```

3. Compile the HDL source files including the boundary scan component libraries, the core logic HDL entity, the HDL model with boundary scan, and the test bench. If you do not already have a compiled model for your core logic, you need to compile this model before you compile any outputs of BSDArchitect. For instance, you may already have a synthesized design which has internal scan circuitry added, and you may have created an HDL model of this design just for input to BSDArchitect. In this case, you would now compile that model. Before you can run the test bench, you must first compile the HDL source code generated by BSDArchitect. After you compile the necessary components and models, compile the test bench for use by the simulator.

Verilog:

```
shell> cd /user/jdoe/bs/verilog_models
shell> <modelsim tree>/bin/vlog -work work aha.v aha_bscan.v
shell> <modelsim tree>/bin/vlog aha_bscan_tb.v
```

VHDL:

```
shell> cd /user/jdoe/bs/vhdl_models
shell> <modelsim tree>/bin/vcom -check_synthesis aha.vhd
shell> <modelsim tree>/bin/vcom -check_synthesis aha_bscan.vhd
shell> <modelsim tree>/bin/vcom aha_bscan_tb.vhd
```

4. Simulate the test bench.

Verilog:

```
shell> <modelsim tree>/bin/vsim -lib work aha_bscan_tb -t ps
vsim1> run -all
vsim1> quit -f
```

VHDL:

```
shell> <modelsim tree>/bin/vsim -lib work aha_bscan_tb
vsim1> run -all
vsim1> quit -f
```

5. Synthesize the design. This step creates a gate-level netlist for your design with boundary scan.

- a. Invoke Design Compiler.

```
shell> design_analyzer
```

- b. Synthesize and optimize the design.

Input Core File = aha.v or aha.vhd
Bscan logic file = aha_bscan.v or aha_bscan.vhd
Output netlist = aha_bscan_netlist.v or aha_bscan_netlist.vhd

The following Design Compiler dofile assumes that you have completed all of the required environment setup steps.

Verilog:

```
include "dc_h4c.setup"
read -f verilog {aha.v aha_bscan.v}
current_design find (design, aha_top_bscan)
uniquify
compile -map_effort medium
write -f verilog -h -o aha_bscan_netlist.v
exit
```

VHDL:

```
include "dc_h4c.setup"
read -f vhd {aha.vhd aha_bscan.vhd}
current_design find (design, aha_top_bscan)
uniquify
compile -map_effort medium
write -f vhd -h -o aha_bscan_netlist.vhd
exit
```

Related Topics

[Verifying the Gate-Level Boundary Scan Logic](#)

[Troubleshooting the Test Bench Simulation](#)

Verifying the Gate-Level Boundary Scan Logic

At this point in the design flow, you have both an HDL model and gate-level model for the boundary scan logic with the core ASIC logic. Previously you have verified the behavior of the HDL boundary scan model. Now, after synthesizing this model and performing any necessary optimizations, you are ready to test the gate-level boundary scan circuitry.

This procedure is nearly identical to the procedure for testing the HDL boundary scan model using the test bench. As before, you could use a gate-level simulator to verify the operation of the boundary scan model.

Prerequisites

- You must have a complete design netlist (from verifying the HDL circuitry).

Procedure

1. Define the work directory (for placing the compiled boundary scan models).

```
shell> cd /user/jdoe/bs
shell> <modelsim tree>/bin/vlib work
```

2. Compile the gate_level_models and the test bench.

Verilog:

```
shell> cd /user/jdoe/bs/gate_level_models
shell> <modelsim tree>/bin/vlog -work work aha_bscan_netlist.v
shell> <modelsim tree>/bin/vlog aha_bscan_tb.v
```

VHDL:

```
shell> cd /user/jdoe/bs/gate_level_models
shell> <modelsim tree>/bin/vcom -check_synthesis
aha_bscan_netlist.vhd
shell> <modelsim tree>/bin/vcom aha_bscan_tb.vhd
```

3. Simulate the test bench.

Verilog:

```
shell> <modelsim tree>/bin/vsim -lib work aha_bscan_tb -t ps
vsim1> run -all
vsim1> quit -f
```

VHDL:

```
shell> <modelsim tree>/bin/vsim -lib work aha_bscan_tb
vsim1> run -all
vsim1> quit -f
```

Related Topics

[Troubleshooting the Test Bench Simulation](#)

Initializing TAP States for Gate-Level Simulations

If you generated a TAP controller without a reset pin by using the [Set Bscan Reset OFF](#) command, then you must manually initialize the following registers at the beginning of gate-level simulation:

- `.../tap_i/tap_ctrl_i/reg_pstate_reg[3]`
- `.../tap_i/tap_ctrl_i/reg_pstate_reg[2]`
- `.../tap_i/tap_ctrl_i/reg_pstate_reg[1]`
- `.../tap_i/tap_ctrl_i/reg_pstate_reg[0]`

Otherwise, the gate-level simulation will fail.

Use the following procedure at the beginning of the gate-level simulation to initialize the registers:

```
vsim1> force -freeze sim:/cpu_bscan_tb/chip/tap_i/tap_ctrl_i/reg_pstate_0 /Q St1
vsim2> force -freeze sim:/cpu_bscan_tb/chip/tap_i/tap_ctrl_i/reg_pstate_1 /Q St1
vsim3> force -freeze sim:/cpu_bscan_tb/chip/tap_i/tap_ctrl_i/reg_pstate_2 /Q St1
vsim4> force -freeze sim:/cpu_bscan_tb/chip/tap_i/tap_ctrl_i/reg_pstate_3 /Q St1
vsim5> run 300
vsim6> noforce sim:/cpu_bscan_tb/chip/tap_i/tap_ctrl_i/reg_pstate_0 /Q
vsim7> noforce sim:/cpu_bscan_tb/chip/tap_i/tap_ctrl_i/reg_pstate_1 /Q
vsim8> noforce sim:/cpu_bscan_tb/chip/tap_i/tap_ctrl_i/reg_pstate_2 /Q
vsim9> noforce sim:/cpu_bscan_tb/chip/tap_i/tap_ctrl_i/reg_pstate_3 /Q
vsim10> run -all
```

AC Port Verification

The following sections apply only if you have AC pins in your design.

AC Test Receiver Verification

In accordance with the IEEE standard 1149.6, AC pins in a design should be testable, and all AC pins that receive data into an IC must be equipped with test receivers. Single-ended AC pins have one test receiver, and differential AC channels have one test receiver per leg. For more information about the standard, refer to the IEEE Std 1149.6-2003 document, available through the IEEE.

BSDArchitect includes a digital model of the analog test receiver as part of the input pad on an AC port and covers a subset of the test receiver model shown in Figure 48 of the IEEE Std 1149.6-2003 document. The generated test bench tests for proper operation of the test receiver in both level-sensitive and edge-detection modes. That is, the behavior of the test receiver is tested with both the EXTEST and EXTEST_PULSE/EXTEST_TRAIN instructions.

During the EXTEST instruction, the AC input pads operate in level-sensitive mode, and the test receiver only responds to logic levels (a valid logic 1 or a valid logic 0). If the input to the test receiver is Z (invalid float state), the output of the test receiver is the previously captured valid logic level. See sections 4.8 and 4.9 of the IEEE Std 1149.6-2003 document for details.

During the EXTEST_PULSE instruction, the AC input pads operate in edge-sensitive mode, and the test receiver responds to signal transitions. If the input to the test receiver is 0/1, the pad output is 0/1. If the input to the test receiver is Z, the behavior is similar to that during the EXTEST instruction. See section 4.7 of the IEEE Std 1149.6-2003 document for details.

The behavior of the test receiver during the EXTEST_TRAIN instruction is the same as during the EXTEST_PULSE instruction.

The following is the Verilog model of BSDArchitect's implementation of the test receiver:

```
module ac_test_receiver (ac_mode, hyst_clk, hyst_preset, jtag_data, ipad);
input  ipad, ac_mode, hyst_clk, hyst_preset;
output jtag_data;
reg    jtag_data;

always @ (posedge ipad or negedge ipad or posedge hyst_clk) begin
    if (~ac_mode)
    begin
        if (ipad === 1'bz)
        begin
            if (hyst_clk)
                jtag_data <= hyst_preset;
            end
        else if ((ipad === 1'bx) || (ipad === 1'b0) || (ipad === 1'b1))
            jtag_data <= ipad;
        else
            jtag_data <= 1'bx;
        end
    else if (ac_mode)
    begin
        if (ipad === 1'bz)
        begin
            if (hyst_clk)
                jtag_data <= hyst_preset;
            end
        else if (ipad === 1'bx)
            jtag_data <= 1'bx;
        else if ((ipad === 1'b0) || (ipad === 1'b1))
        begin
            if (hyst_clk)
                jtag_data <= hyst_preset;
            else
                jtag_data <= ipad;
            end
        else
            jtag_data <= 1'bx;
        end
    end
end
```

```
        jtag_data <= 1'bx;
    end

    endmodule
```

All AC pads use the test receiver internally. In the case of differential pads, each leg of the pad has its own test receiver. The following example shows an AC differential input pad model with two test receivers:

```
module ac_diff_in (pin_p, pin_n, din, din_p, din_n, ac_mode, bsc_data_p,
    bsc_data_n, hyst_clk);
    input pin_p, pin_n, ac_mode, bsc_data_p, bsc_data_n, hyst_clk;
    output din, din_p, din_n;
    wire data_p, data_n;

    ac_test_receiver ac_test_receiver_p(.ac_mode(ac_mode),
        .hyst_clk(hyst_clk),
        .hyst_preset(bsc_data_p),
        .ipad(pin_p),
        .jtag_data(data_p)
    );

    ac_test_receiver ac_test_receiver_n(.ac_mode(ac_mode),
        .hyst_clk(hyst_clk),
        .hyst_preset(bsc_data_n),
        .ipad(pin_n),
        .jtag_data(data_n)
    );

    assign din = (pin_p ^ pin_n) === 1'b1 ? pin_p : 1'bx;
    assign din_p = data_p;
    assign din_n = data_n;
endmodule
```

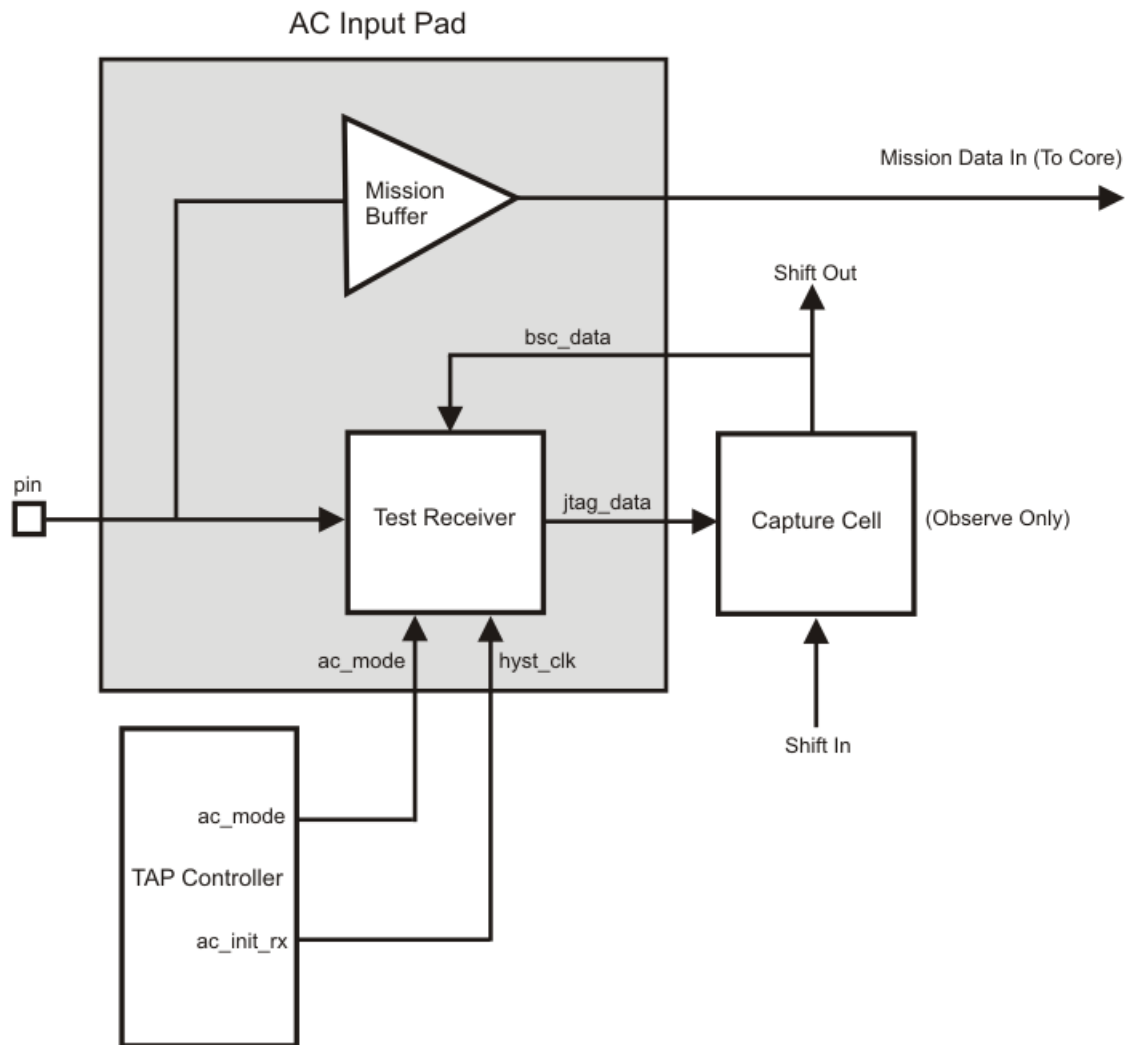
Related Topics

[JTAG Logic Tests](#)

Changes to BSDArchitect-generated RTL for Designs With AC Pads

Each AC pad type has four test ports: `ac_mode`, `bsc_data` (hysteresis preset), `hyst_clk` (hysteresis clock), and `jtag_data` (test receiver output). [Figure 5-1](#) shows the connections between an AC input pad with test receiver, the corresponding BC_4 boundary scan cell, and the TAP controller. Note that `ac_mode` and `ac_init_rx` are just two of several TAP ports.

Figure 5-1. Connections Between an AC Input Pad with Test Receiver, the Corresponding BC_4 Boundary Scan Cell, and the TAP Controller



AC Selection Cell Verification

In accordance with IEEE standard 1149.6 requirements, BSDArchitect verifies that:

- Each AC selection cell controls at least one output pin.
- Each boundary scan cell associated with an output pin has no more than one AC selection cell controlling it.
- Each AC selection cell complies with the structure defined in the standard. Each cell is composed of a shift stage and an update stage. See [Figure 4-22](#) for a diagram of the AC_SelX cell and [Figure 4-23](#) for a diagram of the AC_SelU cell.

- Each AC selection cell is sequentially updated with a 0, and any associated boundary scan cell that is controlled by the selection cell responds to valid logic levels. This is tested with both the EXTEST_PULSE and EXTEST_TRAIN instruction to make sure that the AC selection cells behave as if EXTEST is in effect.
- Each AC_SelU cell can capture the data from serial _input or capture the data from its update stage.
- Each AC selection cell captures the data as specified by its position in the input BSDL file.

Related Topics

[Setting Up the 1149.6 Hardware](#)

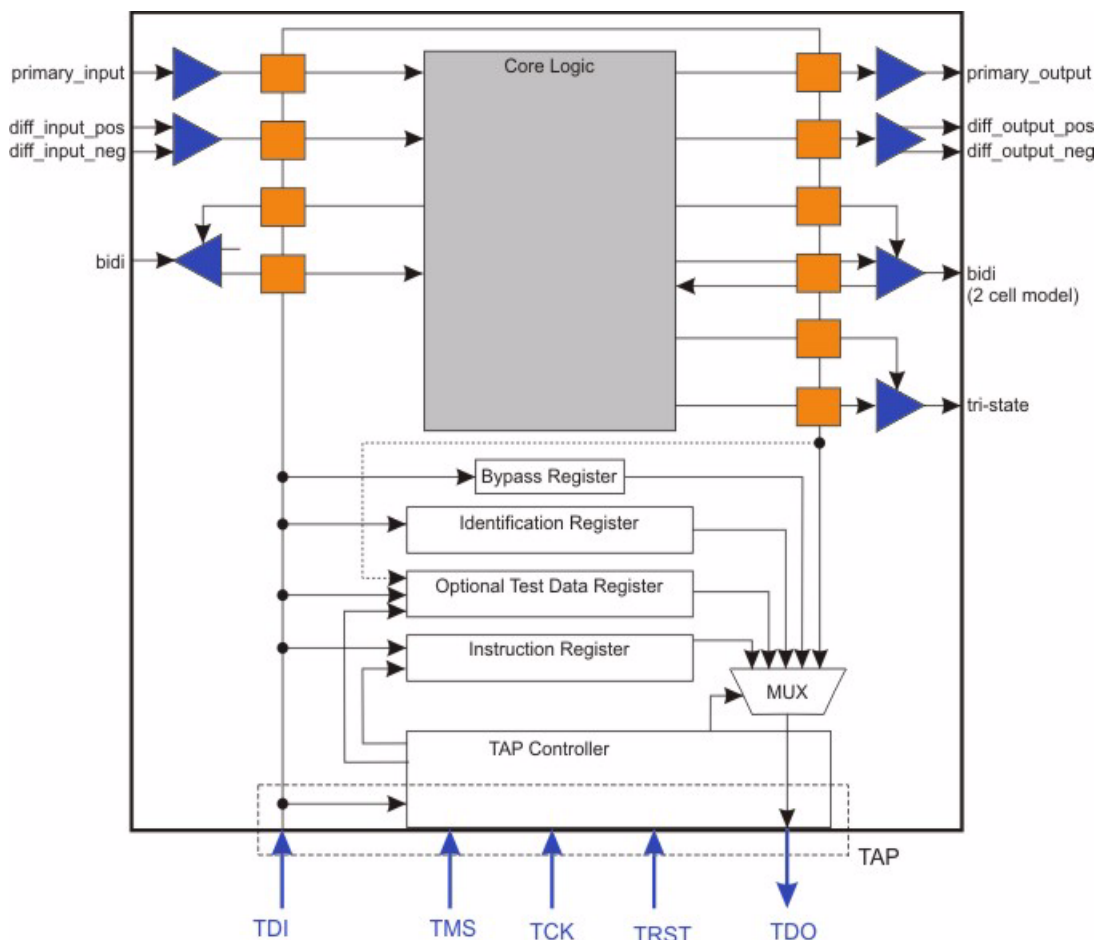
Troubleshooting the Test Bench Simulation

This section describes each test in the test bench in detail and is useful for troubleshooting errors encountered during simulation.

Boundary Scan Test Architecture

[Figure 5-2](#) shows a simple boundary scan implementation. The orange rectangles represent the boundary scan cells and the blue triangles represent the I/O pads in the design. The different input configurations are shown on the left and the output configurations are shown on the right. When you simulate the test bench, the test values may be applied or read in parallel at the top-level input or output ports or shifted in through the boundary scan or data registers. In the following sections “apply” refers to the parallel data given to the input or output ports. “Shift” refers to the series data given to the registers (Boundary Scan, Identification, Bypass, etc.).

Figure 5-2. Test Architecture



JTAG Logic Tests

The following sections describe the tests performed when you simulate your test bench. For more information about the TAP states, refer to “[TAP Controller State Machine](#)” on page 15.

By default, the test bench will include Mode-Mux, SAMPLE, EXTEST, PRELOAD, and BYPASS. When AC pins are used, the test bench also includes EXTEST_PULSE and EXTEST_TRAIN. Additional instructions are tested when created by the [Add Bscan Instruction](#) command.

Note



RUNBIST and INTEST are not tested. All unassigned opcodes are tested for BYPASS instruction, except if your design shares a functional pin with TDO. For more information, refer to “[Sharing Functional Pins With TDO](#)” on page 58.

Any pin without a boundary scan cell can be constrained to a specific value using the [Add Pin Constraint](#) command.

Initialization

1. Perform an asynchronous reset on TAP controller.
 - a. Set TMS = 1 and TRST = 0.
 - b. Wait one clock cycle.
 - c. Set TMS = 0 and TRST=1.
2. Initialize the input boundary scan cells.
 - a. Apply a 1 to all primary input ports to 1.
 - b. Apply a 1 to all differential positive inputs and apply a 0 to all differential negative inputs.
 - c. Apply a Z to all bidirectional ports.

The following example shows the output for initialization when you simulate the test bench:

```
# 1. Initialization
# -----
#      250 OK   Initializing BS logic by resetting TAP controller
#     1050 OK   Initializing PI of input bscells
#     1250 OK   *** Boundary Scan logic has been reset ***
#
```

Check Instruction Register

After the TAP reset, the default instruction should be loaded into the instruction register. The default instruction is IDCODE. If you did not add the IDCODE instruction, then BYPASS will be the default. If the IDCODE instruction is loaded, the identification register will be selected. If the BYPASS instruction is loaded, the bypass register will be selected.

1. Change state to Shift-DR.
2. Shift out one bit from the selected register through TDO.
3. Check the shifted out bit at TDO.
 - 0 — Bypass instruction is selected. The bypass register is loaded with a 0 value at reset.
 - 1 — Idcode instruction. The LSB of the identification register is always 1.

```
# 2. Check instruction register after TAP reset
# -----
#     1250 OK   Checking Instruction Register after TAP reset
#     1850 OK   *** Active instruction is BYPASS ***
#
```

Test for Capture Pattern

1. Load the BYPASS opcode.
2. Change state to Shift-IR.
3. Shift out the contents of instruction register through TDO.
4. Verify the shifted out pattern against the reference pattern.
5. Set TDO to Z. TDO should be set to Z when not shifting.

```
# 3. Test for capture-pattern
# -----
#      3250 OK   Checking for capture pattern
#      4050 OK   *** Capture pattern shifted out properly ***
#      4050 OK   *** TDO is set to 'Z' ***
#
```

Run Through all TAP States

1. Synchronous reset the TAP controller.
 - a. Set TMS = 1.
 - b. Pulse TCK 5 times.
2. Transition from Select-IR to Run-Test/Idle state.
 - a. Change state to Shift-IR.
 - b. Shift in a predefined pattern into the instruction register.
 - c. Verify the 2 LSB shifted out at TDO.
 - d. Change state to Pause-IR and Stay for a while.
 - e. Return to the Shift-IR state. Shift out and verify the rest of the predefined pattern at TDO.
 - f. Return to RUn-Test/Idle by traversing TAP states.
3. Transition from Select-DR to Run-Test/Idle state.
 - a. Load the EXTEST opcode into the instruction register.
 - b. Change state to Shift-DR.
 - c. Shift in a predefined pattern into the boundary scan register.
 - d. Verify the 2 LSB shifted out at TDO.
 - e. Change state to Pause-DR and Stay for a while.
 - f. Return to the Shift-DR state. Shift out and verify the rest of the predefined pattern at TDO.

- g. Return to Run-Test/Idle by traversing TAP states.

```
# 4. Run through all TAP controller states
# -----
#      4450 OK   Synchronously resetting TAP controller
#      6050 OK   Running all TAP controller states and state transitions
#      6850 OK   Checking instruction register shift with pause
#      10850 OK  *** Instruction register shifts properly ***
#      15050 OK  Checking boundary register shift with pause
#      62650 OK  *** Boundary register shifts properly ***
#
```

Test BYPASS Instruction

1. Load the BYPASS opcode into the instruction register.
2. Change state to Shift-DR.
3. Shift out one bit from the bypass register and verify a 0 at TDO.
4. Shift in a predefined pattern into the bypass register at TDI.
5. Verify the predefined pattern shifted out at TDO.
6. Return to Run-Test/Idle state and set TDO to Z.

```
# 5. Test for instruction: BYPASS
# -----
#      110650 OK  Testing opcode: 1111 for BYPASS instruction
#      110650 OK  *** loading opcode: 1111***
#      113250 OK  **** Checking bypass register output
#      115650 OK  Data check OK *** Overshifted bypass register output
# equals the inserted value 11110011111
#      115650 OK  *** TDO is set to 'Z' ***
#
```

Test SAMPLE Instruction

1. Load the SAMPLE opcode into the instruction register. The SAMPLE instruction targets the boundary scan register.
2. Apply the checkerboard pattern “1010...” to the input ports. Apply a Z to all tri-state and bidirectional ports.
3. Change state to Shift-DR.
4. Shift out and verify the checkerboard pattern at TDO.
5. Change state out of Shift-DR and set TDO to Z.
6. Apply the checkerboard pattern “0101...” to the input ports. Apply a Z to all tri-state and bidirectional ports.
7. Change state to Shift-DR.

8. Shift out and verify the checkerboard pattern at TDO.
9. Change state out of Shift-DR and set TDO to Z.

```
# 6. Test for instruction: SAMPLE
# -----
# 116050 OK   Testing opcode: 0001 for SAMPLE instruction
# 116050 OK   *** loading opcode: 0001***
# 118050 OK   *** setting input signals to '...1010' ***
# 118650 OK   **** Checking boundary register serial output with
sampled inputs '...1010'
# 163050 OK   Data check OK *** Pattern '...1010' applied to input
bscells shifted out through boundary register
# 163050 OK   *** TDO is set to 'Z' ***
# 163450 OK   *** setting bidi-input signals to 'z' ***
# 163450 OK   *** setting input signals to '...0101' ***
# 164050 OK   **** Checking boundary register serial output with
sampled inputs '...0101'
# 208450 OK   Data check OK *** Pattern '...0101' applied to input
bscells shifted out through boundary register
# 208450 OK   *** TDO is set to 'Z' ***
```

Test PRELOAD Instruction

1. Apply a Z to all bidirectional ports.
2. Load the PRELOAD opcode into the instruction register.
3. Change state to Shift-DR.
4. Shift in a predefined 12-bit pattern at TDI. Pulse TCK for 7 cycles more than the length of the boundary scan register.
5. Verify the 7 bits of predefined pattern at TDO for the last 7 cycles.

```
# 208850 OK   *** setting bidi-input signals to 'z' ***
# 208850 OK   Test for the PRELOAD functionality in SAMPLE_PRELOAD
instruction
# 208850 OK   *** loading opcode: 0001***
# 255850 OK   *** Boundary register shifts properly in preload mode***
# 255850 OK   *** setting bidi-input signals to 'z' ***
```

Test EXTEST Instruction

1. Test the input ports:
 - a. Load the EXTEST opcode into the instruction register. The EXTEST instruction targets the boundary scan register.
 - b. Create a pattern (Para_pattern) that can switch off tri-state, bidirectional, and open-collector input ports.
 - c. Change state to Shift-DR.

- d. Shift in Para_pattern at TDI and return state to Run-Test/Idle.
 - e. Apply the checkerboard “1010...” pattern on input ports and apply a Z on bidirectional ports.
 - f. Change state to Shift-DR.
 - g. Shift out and verify the checkerboard pattern at TDO. Bidirectional ports with pull down should be 0 and bidirectional ports with pull up should be 1.
 - h. Shift in Para_pattern at TDI and return state to Run-Test/Idle.
 - i. Apply the checkerboard “0101...” pattern on input ports and apply a Z on bidirectional ports.
 - j. Change state to Shift-DR.
 - k. Shift out and verify the checkerboard pattern at TDO. Bidirectional ports with pull down should be 0 and bidirectional ports with pull up should be 1.
2. Test the output ports:
- a. Create a pattern that activates the control pins for tri-state, bidirectional, and open-collector output ports.
 - b. Shift in a pattern of all zeroes “0000...,” with the generated active pattern, into the boundary scan register.
 - c. Verify a 0 value on all output ports (simple, differential, tri-state, bidirectional, and open-collector).
 - d. Shift in a pattern of all ones “1111...,” with the generated active pattern, into the boundary scan register.
 - e. Verify a 1 value on all output ports.
 - f. Shift in a checkerboard “1010...” pattern, with the generated active pattern, into the boundary scan register.
 - g. Verify the checkerboard pattern on all output ports.
 - h. Shift in a checkerboard “0101...” pattern, with the generated active pattern, into the boundary scan register.
 - i. Verify the checkerboard pattern on all output ports.

```
# 7. Test for instruction: EXTEST
# -----
# 255850 OK   Testing opcode: 0000 for EXTEST instruction
# 255850 OK   *** loading opcode: 0000***
# 257850 OK   Testing on input ports
# 257850 OK   Shifting pattern 'z'
# 303250 OK   *** setting input signals to '...1010' ***
# 303850 OK   **** Checking boundary register serial output with
sampled inputs '..1010'
# 348250 OK   Data check OK *** Pattern '..1010' applied to input
bscells shifted out through boundary register
# 348250 OK   *** TDO is set to 'Z' ***
# 348650 OK   Shifting pattern 'z'
# 394050 OK   *** setting input signals to '...0101' ***
# 394650 OK   **** Checking boundary register serial output with
sampled inputs '...0101'
# 439050 OK   Data check OK *** Pattern '...0101' applied to input
bscells shifted out through boundary register
# 439050 OK   *** TDO is set to 'Z' ***
# 439450 OK   Testing on output ports
# 439450 OK   Creating active pattern to enable tri/bidi/oc and oc
bidis
# 439450 OK   Shifting in active and zero pattern in to the BSR
# 484850 OK   Checking on simple outputs, bidis, tristates oc, oc bidis
and diff outputs for zero
# 484850 OK   Data check OK *** All outputs are set to '0'
# 484850 OK   Creating active pattern to enable tri/bidi/oc and oc
bidis
# 484850 OK   Shifting in active and zero pattern into the BSR
# 530250 OK   Data check OK *** All outputs are set to '1'
# 530250 OK   Creating checker board pattern of 0101...
# 530250 OK   Shifting in pattern of 0101...into the BSR
# 575650 OK   Data check OK *** All outputs are set to '...0101'
# 575650 OK   Creating checker board pattern of 1010...
# 575650 OK   Shifting in pattern of 1010...into the BSR
# 621050 OK   Data check OK *** All outputs are set to '..1010'
# 621050 OK   *** setting bidi-input signals to 'z' ***
#
```

3. Test the AC test receiver:

- a. Create a pattern of all zeroes “00...0” corresponding to ac-input, ac-diff-in, ac-bidi, and ac-bidi-diff boundary scan cell positions. The pattern is created so that the corresponding control cells of the ac-bidi and ac-bidi-diff are OFF, which means these pads operate in input mode.
- a. Shift in a pattern of all zeroes “00...0,” with the control cells for ac-bidi and ac-bidi-diff OFF, into the boundary scan register.
- b. Change state to Run-Test/Idle.
- c. Apply “ZZ...Z” to all of the ac-input, ac-diff-in, ac-bidi, and ac-bidi-diff ports.
- d. Change state to Shift-DR.

- e. Shift out and verify the “00...0” pattern at TDO corresponding to the four AC pad types.
- f. Change state to Run-Test/Idle.
- g. Create a pattern of all ones “11...1” corresponding to ac-input, ac-diff-in, ac-bidi, and ac-bidi-diff boundary scan cell positions. The pattern is created so that the corresponding control cells of the ac-bidi and ac-bidi-diff are OFF, which means that these pads operate in input mode.
- h. Shift in a pattern of all ones “11...1,” with the control cells for ac-bidi and ac-bidi-diff OFF, into the boundary scan register.
- i. Change state to Run-Test/Idle.
- j. Apply “ZZ...Z” to all of the ac-input, ac-diff-in, ac-bidi, and ac-bidi-diff ports.
- k. Change state to Shift-DR.
- l. Shift out and verify the “11...1” pattern at TDO corresponding to the four AC pad types.

```
# 76850 OK   Testing input receiver on all ac input, diff-in, bidi and
bidi-diff ports in dcmode
# 76850 OK   shifting pattern '0' for all ac ports configured as input
# 79650 OK   *** setting all ac ports to tristate
# 80250 OK   **** Checking boundary register serial output with ac
inputs set to '0'
# 82050 OK   Data check OK *** Pattern '00...0' applied to ac bscells
configured as input shifted out through boundary register
# 82050 OK   *** TDO is set to 'Z' ***
# 82450 OK   shifting pattern '1' for all ac ports configured as input
# 85250 OK   *** setting all ac ports to tristate
# 85850 OK   **** Checking boundary register serial output with ac
inputs set to '1'
# 87650 OK   Data check OK *** Pattern '11...1' applied to ac bscells
configured as input shifted out through boundary register
# 87650 OK   *** TDO is set to 'Z' ***
```

Test HIGHZ Instruction

1. Load the HIGHZ opcode into the instruction register. The HIGHZ instruction targets the bypass register.
2. Verify all output ports are in an inactive state:
 - a. Verify a Z on tri-state, bidirectional, and open-collector ports.
 - b. Verify a 0 on tri-state, bidirectional, and open-collector ports with pull down.
 - c. Verify a 1 on tri-state, bidirectional, and open-collector ports with pull up.
 - d. Verify a 0 on differential ports with power down.

- e. Verify a 1 on differential ports with power up.
 - f. Normal differential output ports are not checked.
3. Change state to Shift-DR.
 4. Shift in a predefined pattern into the bypass register at TDI and verify the same pattern at TDO after one cycle.

Test for KEEPERS

1. Load the EXTEST opcode into the instruction register.
2. Create a pattern of all ones “1111...” for ports with keepers.
3. Set all tri-state, bidirectional, and open-collector ports to an active state.
4. Change state to Shift-DR.
5. Shift in the ones pattern into the boundary scan register at TDI.
6. Load the HIGHZ opcode into the instruction register.
7. Verify the outputs with keepers.
8. Load the EXTEST opcode into the instruction register.
9. Create a pattern of all zeros “0000...” for ports with keepers.
10. Set all tri-state, bidirectional, and open-collector ports to an active state.
11. Change state to Shift-DR.
12. Shift in the zeros pattern into the boundary scan register at TDI.
13. Load the HIGHZ opcode into the instruction register.
14. Verify the outputs with keepers.

Test CLAMP Instruction

1. Load the SAMPLE opcode into the instruction register.
2. Prepare pattern of all zeros (zero_pat) for output ports. Set all tri-state, bidirectional, and open-collector ports to an active state.
3. Change state to Shift-DR.
4. Shift in the zero_pat pattern into the boundary scan register at TDI.
5. Load the CLAMP opcode into the instruction register. The CLAMP instruction targets the bypass register.
6. Change state to Shift-DR.

7. Shift in a predefined pattern into the bypass register at TDI and observe the same pattern at TDO after one cycle.
8. Verify that all output ports are set to 0.

Test User-Defined Instructions

1. Load user-defined instruction opcode into the instruction register. This instruction will target the register you specified with the Add Bscan Instruction command.
2. Change state to Shift-DR.
3. Shift in a predefined pattern to the selected register at TDI by pulsing TCK for 'n + 1' cycles, where 'n' is the size of the selected register.
4. Shift out and verify the predefined pattern at TDO.

Test MBIST Architect Instructions

1. Load the MBIST opcode into the instruction register.
2. Change state to Shift-DR.
3. Shift in the MBIST start pattern at TDI.
4. Change state to Run-Test/Idle and wait for the MBIST test to complete.
5. Synchronize TCK with the Test Clock (for forcing TMS).
 - If TCK == 1, skip 1 negedge + 1/4 of TCK period.
 - If TCK == 0, skip 1 posedge + 1 negedge + 1/4 of TCK period.
6. Wait for a specified number of cycles.
7. Change state to Capture-DR to capture the MBIST test response.
8. Change state to Shift-DR.
9. Shift out and verify the response at TDO (end at Update-DR state).
10. Repeat the above steps for multiple MBIST test patterns, if required.

Test INT_SCAN and MULT_SCAN Instructions

1. Load the INT_SCAN or MULT_SCAN opcode into the instruction register. The INT_SCAN instruction targets the internal scan chain (core register) you specified with the Add Bscan Instruction command. The MULT_SCAN instruction targets an internal scan chain in series with the boundary scan register.
2. Change state to Shift-DR.

3. Shift in a predefined 4-bit pattern at TDI for N+4 cycles, where N is the length of the selected register.
4. Verify the predefined pattern at TDO.

Test EXTEST_PULSE Instruction

1. Test the output ports:
 - a. Load the EXTEST_PULSE opcode into the instruction register.
 - b. Create a pattern of all ones “1111...” and set all ports to an active state.
 - c. Change state to Shift-DR.
 - d. Shift in the ones pattern into the boundary scan register at TDI.
 - e. Change state to Update-DR (pattern applied on PO).
 - f. Change state to Run-Test/Idle and verify that the pattern is inverted on PO.
 - g. Remain in Run-Test/Idle for 3 cycles and verify that none of the POs are transitioning.
 - h. Change state to Update-DR and verify that all POs are restored to their original value.
 - i. Create a pattern of all zeros “0000...” and set all ports to an active state.
 - j. Change state to Shift-DR.
 - k. Shift in the zeros pattern into the boundary scan register at TDI.
 - l. Change state to Update-DR (pattern applied on PO).
 - m. Change state to Run-Test/Idle and verify that the pattern is inverted on PO.
 - n. Remain in Run-Test/Idle for 3 cycles and verify that none of the POs are transitioning.
 - o. Change state to Update-DR and verify that all POs are restored to their original value.

```
# 94850 OK   Testing opcode: 0010 for EXTEST_PULSE instruction
# 94850 OK   *** loading opcode: 0010 ***
# 99850 OK   **** Checking ac_cell outputs for all 0's, after entering
into run-test-idle
# 99850 OK   Data check OK *** All outputs are set to '0'
# 99850 OK   **** Checking ac_cell outputs for all 0's, while staying
in run-test-idle
# 100050 OK  Data check OK *** All outputs are set to '0'
# 100250 OK  Data check OK *** All outputs are set to '0'
# 100450 OK  Data check OK *** All outputs are set to '0'
# 100650 OK  **** Checking ac_cell outputs for all 1's, after exiting
the run-test-idle
```

```
# 100650 OK Data check OK *** All outputs are set to '1'
# 106450 OK **** Checking ac_cell outputs for all 1's, while entering
into run-test-idle
# 106450 OK Data check OK *** All outputs are set to '1'
# 106450 OK **** Checking ac_cell outputs for all 1's, while staying
in run-test-idle
# 106650 OK Data check OK *** All outputs are set to '1'
# 106850 OK Data check OK *** All outputs are set to '1'
# 107050 OK Data check OK *** All outputs are set to '1'
# 107250 OK **** Checking ac_cell outputs for all 0's, while leaving
the run-test-idle
# 107250 OK Data check OK *** All outputs are set to '0'
```

2. Test the AC test receiver:

- a. Create a pattern of all undefines "XX...X" corresponding to ac-input, ac-diff-in, ac-bidi, and ac-bidi-diff boundary scan cell positions. The pattern is created so that the corresponding control cells of the ac-bidi and ac-bidi-diff are OFF, which means these pads operate in input mode.
- b. Shift in a pattern of all undefines "XX...X," with the control cells for ac-bidi and ac-bidi-diff OFF, into the boundary scan register.
- c. Create a pattern of all zeros "00...0" (value of 1 for n-leg observe cells) corresponding to the ac-input, ac-diff-in, ac-bidi, and ac-bidi-diff pins.
- d. Create a pattern of all ones "11...1" (value of 0 for n-leg observe cells) corresponding to the ac-input, ac-diff-in, ac-bidi, and ac-bidi-diff pins.
- e. Shift in a pattern of all ones "11...1" (value of 0 for n-leg observe cells), with the control cells for ac-bidi and ac-bidi-diff OFF, into the boundary scan register.
- f. Apply the "11...1" pattern to the AC pins.
- g. Wait a clock cycle.
- h. Apply the "00...0" pattern to the AC pins.
- i. Change to Shift-DR state.
- j. Shift out and verify the "00...0" pattern at TDO corresponding to the four AC pad types.
- k. Apply the "ZZ...Z" pattern to the AC pins.
- l. Shift in a pattern of all zeros "00...0" (value of 1 for n-leg observe cells), with the control cells for ac-bidi and ac-bidi-diff OFF, into the boundary scan register.
- m. Shift out and verify the "00...0" pattern at TDO corresponding to the differential AC pad types.
- n. Shift in a pattern of all zeros "00...0" (value of 1 for n-leg observe cells), with the control cells for ac-bidi and ac-bidi-diff OFF, into the boundary scan register.

- o. Apply the "00...0" pattern to the AC pins.
- p. Wait a clock cycle.
- q. Apply the "11..." pattern to the AC pins.
- r. Change to Shift-DR state.
- s. Shift out and verify the "11...1" pattern at TDO corresponding to the four AC pad types.
- t. Apply the "ZZ...Z" pattern to the AC pins.
- u. Shift in a pattern of all ones "11...1" (value of 0 for n-leg observe cells), with the control cells for ac-bidi and ac-bidi-diff OFF, into the boundary scan register.
- v. Shift out and verify the "11...1" pattern at TDO corresponding to the differential AC pad types.
- w. Create a pattern of all undefines "XX...X" corresponding to the ac-input, ac-diff-in, ac-bidi, and ac-bidi-diff pins.
- x. Shift in a pattern of all undefines "XX...X," with the control cells for ac-bidi and ac-bidi-diff OFF, into the boundary scan register.
- y. Create a pattern of zeros "0" for p-leg observe cells and ones "1" for n-leg observe cells corresponding to the ac-input, ac-diff-in, ac-bidi, and ac-bidi-diff pins.
- z. Shift in a pattern of zeros "0" for p-leg observe cells and ones "1" for n-leg observe cells, with the control cells for ac-bidi and ac-bidi-diff OFF, into the boundary scan register.
- aa. Apply the "00...0" pattern to the AC pins and wait a clock cycle.
- ab. Apply the "11...1" pattern to the AC pins and wait a clock cycle.
- ac. Apply the "ZZ...Z" pattern to the AC pins and wait a clock cycle.
- ad. Change to Shift-DR state.
- ae. Shift out and verify the zeros "0" for p-leg observe cells and ones "1" for n-leg observe cells pattern at TDO corresponding to the four AC pad types.
- af. Shift in a pattern of all undefines "XX...X," with the control cells for ac-bidi and ac-bidi-diff OFF, into the boundary scan register.
- ag. Create a pattern of ones "1" for p-leg observe cells and zeros "0" for n-leg observe cells corresponding to the ac-input, ac-diff-in, ac-bidi, and ac-bidi-diff pins.
- ah. Shift in a pattern of ones "1" for p-leg observe cells and zeros "0" for n-leg observe cells with the control cells for ac-bidi and ac-bidi-diff OFF, into the boundary scan register.
- ai. Apply the "11...1" pattern to the AC pins and wait a clock cycle.

- aj. Apply the "00...0" pattern to the AC pins and wait a clock cycle.
- ak. Apply the "ZZ...Z" pattern to the AC pins and wait a clock cycle.
- al. Change to Shift-DR state.
- am. Shift out and verify the ones "1" for p-leg observe cells and zeros "0" for n-leg observe cells pattern at TDO corresponding to the four AC pad types.

```
# 109250 OK   Testing input receiver on all ac input, diff-in, bidi and
bidi-diff ports in acmode
# 109250 OK   *** setting all ac-bidis and ac-bidi-diffs to input mode
# 110050 OK   shifting pattern '1' for all ac ports configured as input
# 111850 OK   *** applying a falling edge on all ac input ports
# 112050 OK   capturing data into bscells
# 112650 OK   **** Checking boundary register serial output
# 114450 OK   *** TDO is set to 'Z' ***
# 114850 OK   *** setting all ac ports to tristate
# 118250 OK   **** Checking boundary register serial output with ac
inputs set to '0'
# 120050 OK   Data check OK *** Pattern '00...0' applied to ac bscells
configured as input shifted out through boundary register
# 120050 OK   *** TDO is set to 'Z' ***
# 120450 OK   shifting pattern '0' for all ac ports configured as input
# 124250 OK   *** applying a rising edge on all ac input ports
# 124450 OK   capturing data into bscells
# 125050 OK   **** Checking boundary register serial output
# 125850 OK   *** TDO is set to 'Z' ***
# 126250 OK   *** setting all ac ports to tristate
# 129650 OK   **** Checking boundary register serial output with ac
inputs set to '1'
# 131450 OK   Data check OK *** Pattern '11...1' applied to ac bscells
configured as input shifted out through boundary register
# 131450 OK   *** TDO is set to 'Z' ***
# 131850 OK   *** setting all ac-bidis and ac-bidi-diffs to input mode
# 134650 OK   Testing hysteresis value of '0' while applying '1' on all
ac input ports
# 134650 OK   shifting pattern '0/1' for all p/n-leg ac ports configured
as input
# 138050 OK   capturing data into bscells
# 138650 OK   **** Checking boundary register serial output
# 140450 OK   *** TDO is set to 'Z' ***
# 140850 OK   *** setting all ac-bidis and ac-bidi-diffs to input mode
# 143650 OK   Testing hysteresis value of '1' while applying '0' on all
ac input ports
# 144650 OK   shifting pattern '1/0' for all p/n-leg ac ports configured
as input
# 147050 OK   capturing data into bscells
# 147650 OK   **** Checking boundary register serial output
# 149450 OK   *** TDO is set to 'Z' ***
```

Test EXTEST_TRAIN Instruction

1. Test the output ports:

- a. Load the EXTEST_TRAIN opcode into the instruction register.
- b. Create a pattern of all ones “1111...” and set all ports to an active state.
- c. Change state to Shift-DR.
- d. Shift in the ones pattern into the boundary scan register at TDI.
- e. Change state to Update-DR (pattern applied on PO).
- f. Change state to Run-Test/Idle and verify that the pattern is inverted on PO.
- g. Remain in Run-Test/Idle for 3 cycles and verify that the pattern on PO is getting inverted on each neg-edge of TCK.
- h. Change state to Update-DR and verify that all POs are restored to their original value.
- i. Create a pattern of all zeros “0000...” and set all ports to an active state.
- j. Change state to Shift-DR.
- k. Shift in the zeros pattern into the boundary scan register at TDI.
- l. Change state to Update-DR (pattern applied on PO).
- m. Change state to Run-Test/Idle and verify that the pattern is inverted on PO.
- n. Remain in Run-Test/Idle for 3 cycles and verify that the pattern on PO is getting inverted on each neg-edge of TCK.
- o. Change state to Update-DR and verify that all POs are restored to their original value.

```
# 94850 OK    Testing opcode: 0010 for EXTEST_TRAIN instruction
# 94850 OK    *** loading opcode: 0010 ***
# 99850 OK    **** Checking ac_cell outputs for all 0's, after entering
into run-test-idle
# 99850 OK    Data check OK *** All outputs are set to '0'
# 99850 OK    **** Checking ac_cell outputs for all 0's, while staying
in run-test-idle
# 100050 OK   **** // Verify that all ac output ports are inverted
# 100050 OK   Data check OK *** All outputs are set to '1'
# 100250 OK   **** // Verify that all ac output ports are inverted
# 100250 OK   Data check OK *** All outputs are set to '0'
# 100450 OK   **** // Verify that all ac output ports are inverted
# 100450 OK   Data check OK *** All outputs are set to '1'
# 100650 OK   **** // Verify that all ac output ports are inverted
# 100650 OK   Data check OK *** All outputs are set to '0'
# 100850 OK   **** Checking ac_cell outputs for all 1's, after exiting
the run-test-idle
# 100850 OK   Data check OK *** All outputs are set to '1'
# 106650 OK   **** Checking ac_cell outputs for all 1's, after entering
into run-test-idle
# 106650 OK   Data check OK *** All outputs are set to '1'
```

```
# 106650 OK **** Checking ac_cell outputs are inverting while staying
in run-test-idle
# 106850 OK **** // Verify that all ac output ports are inverted
# 106850 OK Data check OK *** All outputs are set to '0'
# 107050 OK **** // Verify that all ac output ports are inverted
# 107050 OK Data check OK *** All outputs are set to '1'
# 107250 OK **** // Verify that all ac output ports are inverted
# 107250 OK Data check OK *** All outputs are set to '0'
# 107450 OK **** // Verify that all ac output ports are inverted
# 107450 OK Data check OK *** All outputs are set to '1'
# 107650 OK **** Checking ac_cell outputs for all 0's, after exiting
the run-test-idle
# 107650 OK Data check OK *** All outputs are set to '0'
```

2. Test the AC test receiver:

Note



The steps performed when testing the AC test receiver are identical between the EXTEST_PULSE and EXTEST_TRAIN instructions. Refer to “Test the AC test receiver” in the “[Test EXTEST_PULSE Instruction](#)” section.

Test Mode-Mux

1. Load the SAMPLE opcode into the instruction register.
2. Change state to Capture-DR to capture the core outputs.
3. Change state to Shift-DR.
4. Shift out the sampled core data at TDO and store it in an array (bsreg_value).
5. Invert the sampled core data and store it in an array (inv_bsreg_value).
6. Change state to Shift-DR.
7. Shift in the inverted data into the boundary scan register at TDI.
8. Change state to Update-DR and load the data into update register of the boundary scan cells. At this point, one input of mode mux (PI-PO) contains the sampled data (core data) and other input (Update Reg to PO) contains the inverted data.
9. Load the EXTEST opcode into the instruction register.
10. Verify that the inverted data appears on all POs.

```
# 8. Test MODE-MUX for all instructions
# -----
# 623650 OK **** Shifting out the sampled core logic output signal
values
# 668450 OK Shifting inverted core logic signals into the update
register of bsr cells
#
# 713850 OK Testing Mode-mux during EXTEST instruction
# 713850 OK *** loading instruction EXTEST ***
# 715850 OK Data check OK *** All outputs are set to the update
register value
#
```

Test Unused Opcodes for BYPASS Instruction

1. Load the unused opcode into the instruction register.
2. Change state to Shift-DR.
3. Shift out one bit from the bypass register and verify a 0 at TDO.
4. Shift in a predefined pattern into the bypass register at TDI.
5. Verify the predefined pattern shifted out at TDO.
6. Return to Run-Test/Idle state and set TDO to Z.

```
# 9. Test unassigned opcodes for equivalence to bypass
# -----
# 721850 OK *** loading opcode: 0111***
# 724450 OK **** Checking bypass register output
# 726850 OK Data check OK *** Overshifted bypass register output
equals the inserted value 11110011111
# 726850 OK *** TDO is set to 'Z' ***
# 727250 OK *** loading opcode: 1011***
# 729850 OK **** Checking bypass register output
# 732250 OK Data check OK *** Overshifted bypass register output
equals the inserted value 11110011111
# 732250 OK *** TDO is set to 'Z' ***
# 732650 OK *** loading opcode: 1101***
# 735250 OK **** Checking bypass register output
# 737650 OK Data check OK *** Overshifted bypass register output
equals the inserted value 11110011111
# 737650 OK *** TDO is set to 'Z' ***
...

```

Parametric Testing

The following sections describe the parametric tests that may be included in the test bench.

Typical

1. Load the EXTEST opcode into the instruction register.

2. Test the input ports:
 - a. Create an inactive pattern (para_pattern_z).
 - b. Change state to Shift-DR.
 - c. Shift in the para_pattern_z pattern into the boundary scan register at TDI.
 - d. Apply a 0 to all input ports.
 - e. Change state to Shift-DR (through Update-DR).
 - f. Shift out and verify a 0 for all input port positions at TDO.
 - g. Shift in the para_pattern_z pattern into the boundary scan register at TDI.
 - h. Apply a 1 to all input ports.
 - i. Change state to Shift-DR (through Update-DR).
 - j. Shift out and verify a 1 for all input port positions at TDO.
 - k. Force all bidirectional ports to Z.
3. Test the output ports:
 - a. Create an active pattern for control cells and a zero pattern for data cells (para_pattern_0) for all output ports.
 - b. Change state to Shift-DR.
 - c. Shift in the para_pattern_0 pattern into the boundary scan register at TDI.
 - d. Change state to Run-Test/Idle (through Update-DR).
 - e. Verify that 0 is observed on all output ports.
 - f. Create an active pattern for control cells and a ones pattern for data cells (para_pattern_1) for all output ports.
 - g. Change state to Shift-DR.
 - h. Shift in the para_pattern_1 pattern into the boundary scan register at TDI.
 - i. Change state to Run-Test/Idle (through Update-DR).
 - j. Verify that 1 is observed on all output ports.
 - k. Load the HIGHZ opcode into the instruction register.
 - l. Verify that all outputs are set to Z.

Checkerboard

1. Load the EXTEST opcode into the instruction register.

2. Test the input ports:
 - a. Create an inactive pattern (para_pattern_z).
 - b. Change state to Shift-DR.
 - c. Shift in the para_pattern_z pattern into the boundary scan register at TDI.
 - d. Apply a checkerboard pattern “0101...” to all input ports.
 - e. Change state to Shift-DR (through Update-DR).
 - f. Shift out and verify the checkerboard pattern for all input port positions at TDO.
 - g. Shift in the para_pattern_z pattern into the boundary scan register at TDI.
 - h. Apply a checkerboard pattern “1010...” to all input ports.
 - i. Change state to Shift-DR (through Update-DR).
 - j. Shift out and verify the checkerboard pattern for all input port positions at TDO.
3. Test the output ports:
 - a. Create a checkerboard pattern “0101...” for all output ports.
 - b. Change state to Shift-DR.
 - c. Shift in the checkerboard pattern into the boundary scan register at TDI.
 - d. Change state to Run-Test/Idle (through Update-DR).
 - e. Verify that the checkerboard pattern is observed on the output ports.
 - f. Create a checkerboard pattern “1010...” for all output ports.
 - g. Change state to Shift-DR.
 - h. Shift in the checkerboard pattern into the boundary scan register at TDI.
 - i. Change state to Run-Test/Idle (through Update-DR).
 - j. Verify that the checkerboard pattern is observed on all output ports.
 - k. Load the HIGHZ opcode into the instruction register.
 - l. Verify that all outputs are set to Z.

Ground Bounce Set

1. Test the input ports:
 - a. Load the EXTTEST opcode into the instruction register.
 - b. Create an inactive pattern (para_pattern_z).

- c. Change state to Shift-DR.
 - d. Shift in the para_pattern_z pattern into the boundary scan register at TDI.
 - e. Change state to Run-Test/Idle.
 - f. Apply a 0 to all inputs.
 - g. Wait for 1 TCK cycle.
 - h. Apply a 1 to all inputs.
 - i. Change state to Shift-DR.
 - j. Shift out and verify that a 1 was observed on all input positions at TDO.
 - k. Set all bidirectional ports to an inactive state.
2. Test the output ports:
 - a. Load the EXTEST opcode into the instruction register.
 - b. Create a zeros pattern (para_pattern_0) for all outputs.
 - c. Change state to Shift-DR.
 - d. Shift in the para_pattern_0 pattern into the boundary scan register at TDI.
 - e. Change state to Run-Test/Idle.
 - f. Reload the EXTEST opcode into the instruction register.
 - g. Create a ones pattern (para_pattern_1) for all outputs.
 - h. Change state to Shift-DR.
 - i. Shift in the para_pattern_1 pattern into the boundary scan register at TDI.
 - j. Change state to Run-Test/Idle (through Update-DR).
 - k. Verify that a 1 was observed on all output ports.

VCC Bounce Set

This is the same as the Ground bounce test except that pattern order is reversed:

1. Test the input ports:
 - a. Load the EXTEST opcode into the instruction register.
 - b. Create an inactive pattern (para_pattern_z).
 - c. Change state to Shift-DR.
 - d. Shift in the para_pattern_z pattern into the boundary scan register at TDI.

- e. Change state to Run-Test/Idle.
 - f. Apply a 1 to all input ports.
 - g. Wait for 1 TCK cycle.
 - h. Apply a 0 to all input ports.
 - i. Change state to Shift-DR.
 - j. Shift out and verify that a 0 was observed on all input positions at TDO.
 - k. Set all bidirectional ports to an inactive state.
2. Test the output ports:
- a. Load the EXTEST opcode into the instruction register.
 - b. Create a ones pattern (para_pattern_1) for all outputs.
 - c. Change state to Shift-DR.
 - d. Shift in the para_pattern_1 pattern into the boundary scan register at TDI.
 - e. Change state to Run-Test/Idle.
 - f. Reload the EXTEST opcode into the instruction register.
 - g. Create a zeros pattern (para_pattern_0) for all outputs.
 - h. Change state to Shift-DR.
 - i. Shift in the para_pattern_0 pattern into the boundary scan register at TDI.
 - j. Change state to Run-Test/Idle (through Update-DR).
 - k. Verify that a 0 was observed on all output ports.

Chapter 6

Existing Boundary Scan Register Integration

The ever-increasing size and complexity of designs often require reusable blocks or designs. These blocks, which come from such sources as independent IP vendors, EDA tool suppliers, or semiconductor companies, may be either soft, firm, or hard cores (IP).

Note



Although sometimes referred to as *reusable blocks*, *reusable designs*, *reused cores*, *IP macros*, *BSR segment cores*, *modules*, or *reused block modules*, we refer to them as **existing blocks** in this documentation.

This topic describes how to work with an existing block that contains a boundary scan register (BSR) and then integrate it with a new top-level design. BSDArchitect recognizes an existing block that has pre-configured 1149.1/1149.6-compliant JTAG circuitry, connects the existing block's boundary scan chain with the chip-level boundary scan cells and inserts the TAP, generates test benches to simulate the capabilities of and compliance with the 1149.1/1149.6 IEEE standard, and generates WGL/STIL patterns and BSDL for compliance checking and DC parametric tests.

Figure 6-1 shows how, in the Internal flow, BSDArchitect connects the existing block's boundary scan register segment with the other boundary scan cells in the design. The blue line shows the BSR in the existing block, and the red line shows the top-level BSR that incorporates the existing block's BSR.

BSDArchitect Process Guide, v2015.4
December 2015

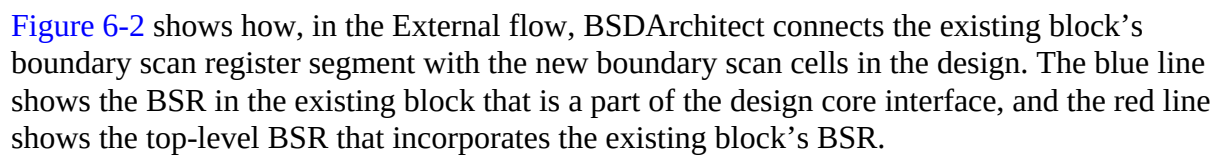
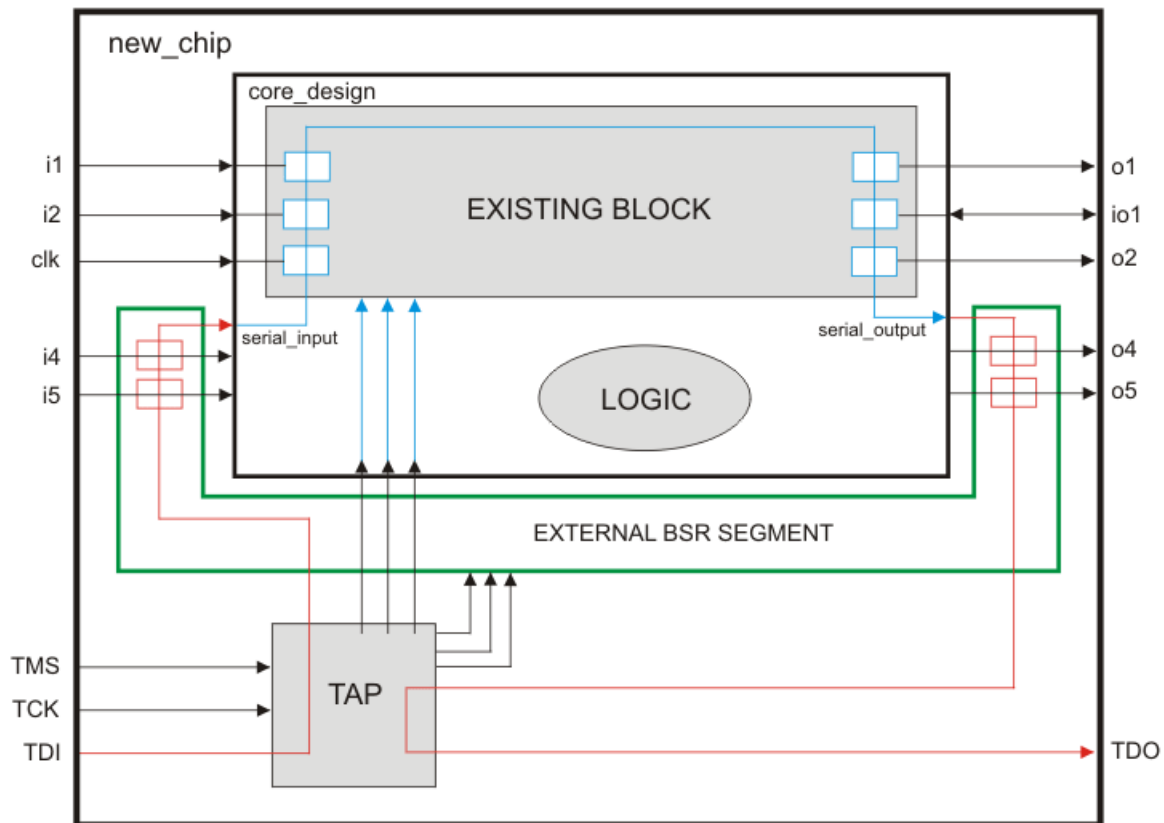


Figure 6-2. External Flow Connections of the Existing Block's BSR Segment with the Design's New Boundary Scan Cells

Working with Existing Blocks

A complex chip may have one or more occurrences of an existing block. These blocks can be the same type or different types and can interact with circuit elements within the chip or with the external interface. The existing block must have a Verilog netlist and a BSDL file.

If present in the existing block, the TAP must be able to be bypassed so that the block's BSR and control signals are directly accessible from the interface. The **serial_input** of the beginning cell of the BSR and **serial_output** of the ending cell of the BSR should be available at the interface. If the **TDI/TDO** of the existing block's TAP is provided instead, you must be able to select the **TDO** mux in such a way that the **serial_output** of the last cell in the existing block's BSR always drives out though the **TDO** directly.

To bypass the TAP controller in the existing block, you can tie some pins to known values throughout the test. This ensures that only the BSR segment of the block is directly available to the periphery of the existing block and that the BSR segment can shift values correctly, assuming that the signals at the interface are configured correctly.

Existing block pins with boundary scan cells and pads cannot be shared; they must have a one-to-one mapping to the top level or terminate inside of the design.

The existing block may have all types of pins, but the pins' connections must be handled differently from the rest of the circuit. Following is an explanation of how BSDArchitect handles these various pins.

I/O's Mapped to the Top Level of the Device

The following pins are mapped to the top level; that is, they can be traced to the top in the Internal flow. BSDArchitect assumes that all pins have boundary scan pads and cells of the appropriate type and does not do any checking to make sure that the pads conform with the data provided in the BSDL file or even if a pad exists. You can, however, add a pad that conforms to the correct port type.

- **BSDL Linkage Pins** — Non-digital pins such as power, ground, no-connects, and analog are present as linkage pins in the input BSDL. These pins remain untouched and appear as linkage pins in the top-level BSDL.
- **Clock Pins** — Aside from the test clock *TCK*, any pins specified as clock are treated as free-running in the test bench.
- **Compliance Pins** — Existing compliance pins from the BSDL file become top-level compliance pins, retaining their state patterns. Pins without boundary scan cells or pads that are designated as compliance pins become top-level compliance pins.
- **Power Pins** — Pins without boundary scan cells or pads that are designated as power pins must have power pads added and are reflected as linkage pins in the BSDL file.
- **Other Pins** — Other pins can be single bits or vectors. Individual vector pins all must be connected; none can be left floating. The port type of the existing block and the port type to which it is mapped at the top level must be identical (that is, input to input, output to output, inout to inout).

The other pins may have boundary scan cells, pads, or neither:

- **Pins with boundary scan cells and pads** — The BSDL file for the existing block contains the port type and boundary scan cell information.
- **Pins with boundary scan cells but without pads** — The BSDL file for the existing block contains the boundary scan cell information. Insert pads with the [Add Pad Cell](#) command. For details on usage, see the documentation for that command.
- **Pins with pads but without boundary scan cells** — If the existing block is fixed (hard IP), BSDArchitect ignores these pins because the boundary scan cells cannot be inserted after the pad. If the existing block is not fixed (soft IP), the boundary scan cells can be inserted.

- **Pins without boundary scan cells and pads** — If the existing block is fixed (hard IP), BSDArchitect ignores these pins because the boundary scan cells and pads cannot be inserted. If the existing block is not fixed (soft IP), the boundary scan cells and pads can be inserted.
- **TAP Pins** — BSDArchitect ignores the existing block's TAP pins unless the pins provide access to or operate the JTAG BSR segment as specified with the -Connect switch of the [Add Existing Block](#) command. Some cases of the latter are when:
 - TDI provides serial_input to the BSR chain.
 - TDO provides serial_output from the BSR chain.
 - TRST provides a global asynchronous reset.
 - TCK provides the test clock for the boundary scan cell clock.

Note

TMS cannot be used because the TAP operation is bypassed for this functionality.

I/O's Not Mapped to the Top Level of the Device

The following pins of the existing block may either go to the TAP, connect to any user-defined internal location, or be left open.

- **Access Pins** — The serial_input or serial_output access of the BSR segment, the mode signals, or signals such as updatedr, clockdr, shiftdr, and ac_mode are connected to the appropriate TAP pins.
- **Other Pins** — Other pins can be single bits or vectors. They may have boundary scan cells, pads, or neither:
 - **Pins with pads and internal boundary scan cells** — BSDArchitect ignores pins with pads because pads are not necessary for internal pins. The pins with internal boundary scan cells are not used for JTAG testing except that they pass on the values to the next cell for serial shifting. In the BSDL file, these pins have the function type *internal*. According to the IEEE standard, only the boundary scan cell types BC_1, BC_2, BC_3, and BC_4 can be internal.
 - **Pins with internal boundary scan cells but without pads** — The BSDL file for the existing block contains the boundary scan cell information. Insert pads with the [Add Pad Cell](#) command. For details on usage, see the documentation for that command.
 - **Pins with pads but without boundary scan cells** — BSDArchitect ignores these pins as non-functional, and they do not appear in the test bench or patterns.

- **Pins without boundary scan cells and pads** — If you choose to insert boundary scan cells and pads, insert pads external to the existing block and insert boundary scan cells as part of the top-level BSR chain.
- **All pins are internal** — BSDArchitect handles the entire chain as an internal BSR. That is, BSDArchitect does not use the pins for JTAG testing except that they pass on the values to the next cell for serial shifting. In the BSDL file, these pins have the function type *internal*. According to the IEEE standard, only boundary scan cell types BC_1, BC_2, BC_3, and BC_4 can be internal.

How the Commands Relate to Existing Blocks

Not all BSDArchitect commands have applicability to existing blocks. Some have relevance, while others have implications only for the top level of the design. The Set commands have impact only on the settings of the TAP controller or soft logic available on the chip or any new JTAG hardware that BSDArchitect inserts. The current scenario and whether the existing block is hard or soft determine a command's relevance. Existing blocks that are hard cannot be modified.

The following is a list of commands with a brief description of their applicability to existing blocks. Two commands, Load Design Block and Add Existing Block, apply specifically to existing blocks. Assume that if a command applies to pins on an existing block, the pins are traceable to the top. For complete details on the functionality of each command, see the *BSDArchitect Reference Manual*.

Add Ac Port	Applies to top-level pins or existing block pins when AC pads and/or boundary scan cells can be inserted.
Add Bscan Cell	Specifies boundary scan cells for existing block pins that do not already have boundary scan cells.
Add Bypass Mux	Adds a bypass mux only on the BSR segment that BSDArchitect adds. A bypass mux cannot be added to BSR segments that are between two existing blocks.
Add Datacontrol Port	Applies only to the top-level ports, not to ports on the existing block.
Add Existing Block	Specifies the location of the existing block as well as the connections to that block.
Add Pad Cell	Specifies whether the existing block has a pad cell.
Add Pin Constraint	Specifies the constraint value for any pin without a boundary scan cell. The pin is held to this value during the test.
Add Port Connection	Makes the connections between the existing block ports and the pins inside the top level.

Add Testbench Clocks	Specifies an existing block pin that is mapped to the top as a free-running clock for the test bench.
Load Design Block	Defines an existing block and specifies a BSDL file as its interface.
Load Pin Map	Loads a pin map file that can contain existing block instances as well as pin names.
Set Ac Parameters	Applies to modifiable parameters of the top level but not to parameters of the existing block. For example, the low pass and high pass time constants can be specified for AC pad test receivers for AC pads that BSDArchitect adds, but for AC pins of existing blocks, the data comes from the BSDL file.
Set Bist Interface	Applies to the top level that can be modified but not to the BIST interface of the existing block.
Set Bscan Cell	Specifies the boundary scan cell types that BSDArchitect inserts. These cells are external to the existing block.
Set Bsr Instances	Applies to BSR instances that BSDArchitect adds.
Set Pad Cell	Inserts pad cells external to the existing block.
Set Pad Instances	Applies to pad instances that BSDA adds.
Set Pad Order	Applies to pad instances that BSDA adds.
Set Port Order	Applies only to the top-level ports because the BSDL file defines the port order for the existing block. The port order inside the existing block cannot be modified.

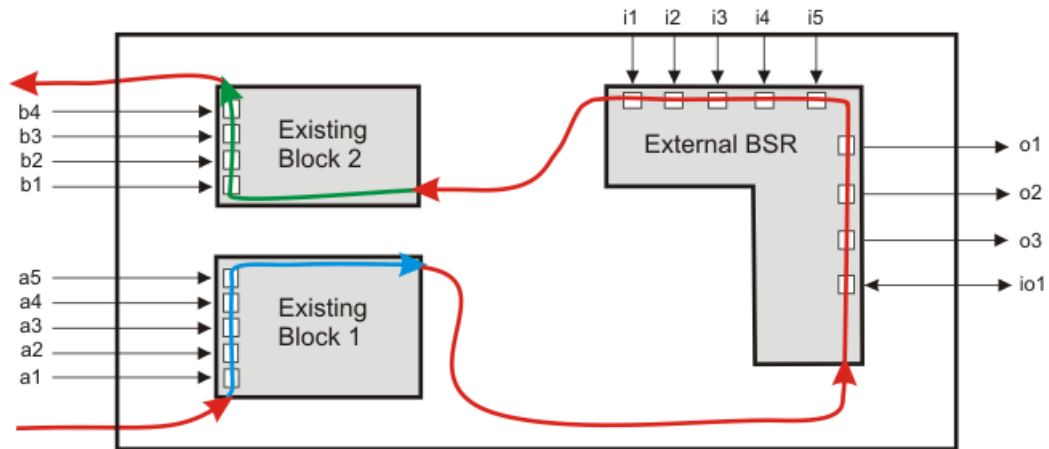
Specifying the Routing Order With Existing Block BSRs

Use the [Set Port Order](#) command to specify the routing order with existing block BSRs. You can set the port order by specifying existing block instances and individual port names either at the command line or in a separate file.

The following command example, along with its graphical representation in [Figure 6-3](#), shows how to specify the routing order of multiple existing blocks in the External flow. In the command example, *existing_block1_instance* is the instance path name for Existing Block 1, and *existing_block2_instance* is the instance path name for Existing Block 2.

```
set port order existing_block1_instance io1 o3 o2 o1 i5 i4 i3 i2 i1 existing_block2_instance
```

Figure 6-3. Example of Routing Order With Multiple Existing Blocks in the External Flow



There are several ways to get help when setting up and using Tessent software tools. Depending on your need, help is available from documentation, online command help, and Mentor Graphics Support.

Documentation

The Tessent software tree includes a complete set of documentation and help files in PDF format. Although you can view this documentation with any PDF reader, if you are viewing documentation on a Linux file server, you must use only Adobe® Reader® versions 8 or 9, and you must set one of these versions as the default using the MGC_PDF_READER variable in your *mgc_doc_options.ini* file.

For more information, refer to “[Specifying Documentation System Defaults](#)” in the *Managing Mentor Graphics Tessent Software* manual.

You can download a free copy of the latest Adobe Reader from this location:

<http://get.adobe.com/reader>

You can access the documentation in the following ways:

- **Shell Command** — On Linux platforms, enter **mgcdocs** at the shell prompt or invoke a Tessent tool with the -Manual invocation switch. This option is available only with Tessent Shell and the following classic point tools: Tessent FastScan, Tessent TestKompress, Tessent Diagnosis, and DFTAdvisor.
- **File System** — Access the Tessent bookcase directly from your file system, without invoking a Tessent tool. From your product installation, invoke Adobe Reader on the following file:

```
$MGC_DFT/doc/pdfdocs/_bk_tessent.pdf
```

- **Application Online Help** — You can get contextual online help within most Tessent tools by using the “help -manual” tool command:

```
> help dofile -manual
```

This command opens the appropriate reference manual at the “dofile” command description.

Mentor Graphics Support

Mentor Graphics software support includes software enhancements, access to comprehensive online services with SupportNet, and the optional On-Site Mentoring service.

For details, refer to this page:

<http://supportnet.mentor.com/about>

If you have questions about a software release, you can log in to SupportNet and search thousands of technical solutions, view documentation, or open a Service Request online:

<http://supportnet.mentor.com>

If your site is under current support and you do not have a SupportNet login, you can register for SupportNet by filling out a short form here:

<http://supportnet.mentor.com/user/register.cfm>

All customer support contact information is available here:

<http://supportnet.mentor.com/contacts/supportcenters/index.cfm>

Appendix B

Cell Type Assignments for Bidirectional Ports

Background

By default, the BSDArchitect tool follows the IEEE 1149.1 standard for bidirectional (bidi) ports. If you have a configured bidi port, then the tool adds the following:

- Control Port — Adds a combined type BC_2_A, BC_2_A_EXT, or BC_2_B.
- Bidi-out Port — Adds a BC_7 or BC_7_LOW cell type.
- Bidi-in Port — Assigns no cell.

When using IEEE 1149.6 features, the BSDArchitect tool adds the following by default:

- AC Bidi-out Port — Adds an AC_7 or AC_7_LOW cell type.
- AC Bidi-in Port — Assigns no cell.

Consequently, this configuration allows the tool to skip a cell on the bidi-input port and allows the same testability as you would have if there was a cell (for example, BC_1 which is a 3-cell model) each on each port.

To meet this combination, the tool needs more circuits than that of the standard BC_2 cell and, depending on if the bidi-control (or AC) is active high or low, the tool also needs a different BC_7 (or AC_7) cell.

Note



BC_7 (or AC_7) is for active high, and BC_7_LOW (or AC_7_LOW) is for active low pad.

Understanding Cell Type Assignments

When using the BSDArchitect External flow, you declare and configure bidi ports using the [Add Tristate Port](#) command. This command *automatically* assigns one of the following boundary scan cell types to the bidi port:

- [BC_2_A Cell](#) — A non-IEEE standard cell type for use with a combined I/O bidi control pin. The tool assigns this cell type when the bidi control pad is *active low*.

- [BC_2_A_EXT Cell](#) — A non-IEEE standard cell type similar to BC_2_A for use with a combined I/O bidi control pin. The tool assigns this cell type when the bidi-control is an external control signal.
- [BC_2_B Cell](#) — A non-IEEE standard cell type similar to BC_2_A for use with a combined I/O bidi control pin. The tool assigns this cell type when the bidi control pad is *active high*.
- [BC_7 Cell](#) — An IEEE-specified cell type. The tool assigns this cell on the output of a bidi when the bidi pad is *active high*.
- [BC_7_LOW Cell](#) — A non-IEEE standard cell type similar to BC_7. The tool assigns this cell on the output of a bidi when the bidi pad is *active low*.
- [AC_7 Cell](#) — An IEEE 1149.6-specified cell type. The tool assigns this cell on the output of a bidi when the bidi pad is *active high*.
- [AC_7_LOW Cell](#) — A non-IEEE standard cell type similar to AC_7. The tool assigns this cell on the output of a bidi when the bidi pad is *active low*.

You can modify these cell types assignments to a bidi port using the [Add Bscan Cell](#) command.

Cell Type Correspondence

[Table B-1](#) lists the BC_2_* to BC_7* cell type correspondence.

Table B-1. BC_2_* to BC_7* Cell Type Correspondence

Desired Output When INTEST or RUNBIST Instruction is Present	Active Value of Bidi Cell	BC_2_* Cell Added	Required Output Cell Type
HIGHZ	Low	BC_2_B	BC_7_LOW
Cell Value	Low	BC_2_A	BC_7_LOW
HIGHZ	High	BC_2_A	BC_7
Cell Value	High	BC_2_B	BC_7

Appendix C

Setting Up the 1149.6 Hardware

BSDArchitect includes enhancements in support of the IEEE standard 1149.6, which addresses boundary scan testing of AC-coupled differential nets. You can use these enhancements with both the External and Internal Boundary Scan flows. For more information about the standard, refer to the IEEE 1149.6 document, available through the IEEE.

AC Boundary Scan Cells Support

BSDArchitect supports these IEEE 1149.6 boundary scan cells:

- [AC_1 Cell](#)
- [AC_2 Cell](#)
- [AC_7 Cell](#)
- [AC_7_LOW Cell](#)
- [AC_8 Cell](#)
- [AC_9 Cell](#)
- [AC_10 Cell](#)
- [AC_SelX Cell](#)
- [AC_SelU Cell](#)

AC Pad Cells

BSDArchitect provides the following extended generic AC versions of the input, bidirectional, and differential input pads. These pads are identical to the corresponding DC versions but contain additional pins.

- [AC Input Pad](#)
- [AC Bidirectional Pad](#)
- [AC Differential Input Pad](#)
- [AC Bidi Differential Pad](#)

Note



All AC pads use the AC test receiver internally. For more information, refer to the “[AC Test Receiver Verification](#)” section.

Support for EXTEST_PULSE and EXTEST_TRAIN Instructions

BSDArchitect generates IEEE 1149.6-compliant RTL and BSDL if you:

- Define at least one of the IEEE 1149.6-mandated instructions EXTEST_PULSE or EXTEST_TRAIN.
- Designate at least one pin as an AC driver.
- Define AC ports and omit the EXTEST_PULSE or EXTEST_TRAIN instructions. In this case, BSDArchitect by default adds the EXTEST_PULSE instruction and generates the 1149.6 compliant RTL.



Tip: Use the [Add Bscan Instruction](#) command to add these instructions manually.

Setting Up the AC Boundary Scan Ports

The IEEE 1149.6 specification states that you use AC cells exclusively on output ports; however for the inputs, the specification recommends that you use a test receiver, which is part of the pad. Therefore, you must use the following commands to identify the AC pads to which you want to allocate AC resources:

- [Add Ac Port](#)
- [Delete Ac Port](#)
- [Report Ac Port](#)
- [Set Ac Parameters](#)

Before associating an AC resource with a port, you must designate the port as an AC port.

Adding AC Boundary Scan Cells

Specify AC boundary scan cells with these commands:

- [Add Bscan Cell](#)
- [Delete Bscan Cell](#)

- [Set Bscan Cell](#)

When using the IEEE 1149.6 standard, adhere to the [Boundary Scan Rules Checking](#) guidelines for the AC cells.

Specifying AC/DC Selection Cells

The IEEE 1149.6 standard introduces an optional AC/DC selection cell. This optional selection cell reverts an AC pin or a set of AC pins to EXTEST behavior when BSDArchitect loads either the EXTEST_PULSE or EXTEST_TRAIN instruction.

These optional selection cells are part of the BSR but are internal cells. For example, you can use AC_SelX and AC_SelU for internal cells, which are not associated with any port but remain part of the BSR.

Specify the AC/DC selection cells using the Add Bscan Cell command with the following additional AC/DC selection cell-specific syntax:

```
ADD BScan Cell ... cell_to_select/AC_SelX[BSR_cell_position] ...
```

or

```
ADD BScan Cell ... cell_to_select/AC_SelU[BSR_cell_position] ...
```

The *BSR_cell_position* is a number in square brackets that corresponds to the position of the AC_Sel[X|U] cell in the BSR chain; the *BSR_cell_position* controls the AC cell's AC/DC behavior.

If you omit the square brackets, BSDArchitect creates a dedicated AC_SelX/AC_SelU for each cell.

Example 1

AC_SelX cell at position after cell 5 in the BSR is overriding the ac_mode control of cell AC_1 on port my_out_port:

```
add ac port my_out_port  
add bscan cell AC_1/AC_SelX[5] my_out_port
```

Example 2

AC_SelU at position after cell 4 is controlling the AC_7 cell on the bidi output port:

```
add bscan cell AC_7/AC_SelU[4] my_bidi_output_port
```

Example 3

All AC cells on port type output should have a common AC_SelX cell, which is located at position 4:

```
set bscan cell -out AC_1/AC_SelX[4] -tri_out AC_1/AC_SelU[3]
```

Example 4

AC_SelX will be the selection cell for port out_port and positioned before the primary cell AC_1 meant for port out_port:

```
add bscan cell AC_1/AC_SelX out_port
```

Note



For information on AC selection cell verification, refer to the “[AC Selection Cell Verification](#)” section.

Specifying Observe Only Cells

The IEEE 1149.6 standard requires a cell to have the negative and the positive leg of the differential pair. Alternatively, a cell can have an AC resource on the input pad. You also can have an optional cell to monitor the input buffer using syntax similar to the following examples:

Example 1

Note that the input_pad/differential pair (port P7 in this case) has to be an AC type.

```
add bscan cell BC_4 P7/din_observe
```

```
add bscan cell AC_1 P7
```

Example 2

Differential pair P5-P6.

```
add bscan cell BC_4 P6/din_p
```

```
add bscan cell BC_4 P6/din_n
```

```
add bscan cell AC_1 P6/din // or Omit the word din for the cell (1149.1 type)
```

Summary of Modifications to Report Commands

The following BSDArchitect report commands support the IEEE 1149.6 implementation:

- REPort BScan Cell
- REPort BScan Status
- REPort PIn Map
- REPort PIn Name
- REPort TEchnology Cell
- REPort POrt Connection -usable_signals

For complete information, see the *BSDArchitect Reference Manual*.

— A —

AC boundary scan cells, adding, [180](#)
 AC boundary scan ports, setting up, [180](#)
 AC pads, [107](#)
 AC port verification, [140](#)
 AC selection cell verification, [143](#)
 AC test receiver verification, [140](#)
 AC_1 cell type, [87](#)
 AC_10 cell type, [97](#)
 AC_2 cell type, [88](#)
 AC_7 cell type, [90](#)
 AC_7_LOW cell type, [92](#)
 AC_8 cell type, [94](#)
 AC_9 cell type, [95](#)
 AC_SelU cell type, [99](#)
 AC_SelX cell type, [98](#)
 AC/DC selection cells, specifying, [181](#)
 ATPG setup files, [128](#)

— B —

BC_1 cell type, [67](#)
 BC_10 cell type, [85](#)
 BC_2 cell type, [69](#)
 BC_2_A cell type, [70](#)
 BC_2_A_EXT cell type, [72](#)
 BC_3 cell type, [74](#)
 BC_4 cell type, [75](#)
 BC_5 cell type, [76](#)
 BC_7 cell type, [78](#)
 BC_8 cell type, [82](#)
 BC_9 cell type, [83](#)
 BC_X cell type, [86](#)
 Bidirectional ports
 cell type assignments, [177](#)
 important issues, [30](#)
 Boundary scan
 architecture, [13](#)
 board example, [11](#)
 cells, [13](#)

 customizing, [61](#)
 internal design flow, [43](#)
 pin mapping, [111](#)
 register, [14](#)
 specification, [39](#)
 verification, [136](#)

Boundary scan cell mode, [67](#)
 Boundary Scan Register, existing, [167](#)

BSDArchitect
 design issues, [30](#)
 external design flow, [23](#)
 features, [18](#)
 invocation, [39](#), [44](#)
 limitations, [36](#)
 output model, [24](#)
 recommended practices, [38](#)
 reserved words, [38](#)

BSDL, [18](#)
 BSDL flow, [20](#)

Bypass
 instruction, [15](#)
 register, [14](#)

— C —

Clamp instruction, [15](#)

Commands
 add bscan cell, [112](#)
 add bscan instruction, [117](#)
 add core register, [117](#), [123](#)
 add differential port, [106](#), [112](#)
 add external register, [57](#)
 add opencollector port, [103](#), [112](#)
 add pad cell, [47](#), [100](#), [112](#)
 add port connection, [56](#)
 add technology cell, [47](#), [100](#), [101](#)
 add tristate port, [102](#), [112](#)
 connect core chains, [56](#)
 connect iscan chains, [56](#), [124](#)
 delete nontop port, [58](#)
 dofile, [40](#), [61](#)

- exit, [40](#)
- load pin map, [55](#), [111](#)
- report circuit components, [46](#)
- report pad cell, [48](#)
- run, [40](#), [46](#)
- save bscan, [40](#)
- save patterns, [128](#)
- set controller naming, [47](#)
- set dofile abort, [21](#)
- set iscan interface, [122](#)
- set pad cell, [47](#)
- set port order, [112](#)
- set technology mapping, [47](#)
- Continuation character, [20](#)
- Control port notation, [51](#)
- Control signal, [33](#)
- Custom boundary scan, [61](#)
- D —
- Design
 - BSDL flow, [20](#)
 - External flow, [19](#), [23](#)
 - external flow example, [40](#)
 - internal flow, [19](#), [43](#)
 - internal flow example, [45](#)
 - issues, [30](#)
- Dofile, [40](#), [61](#)
 - batch mode, [20](#)
 - set dofile abort, [21](#)
- Dummy scan pins, [119](#)
- E —
- Enable signal, [33](#)
- Entity port logic type, [30](#)
- Escaped identifiers, [30](#)
- Existing block
 - applicable commands, [172](#)
 - working with, [169](#)
- Existing block BSR, specifying routing order, [173](#)
- External boundary scan synthesis, [23](#)
- External design flow, [23](#)
- External flow, [19](#)
 - existing block BSR, [168](#)
- External register, [57](#)
- Extest instruction, [14](#)
- Extest_pulse instruction, [15](#), [180](#)
- Extest_train instruction, [15](#), [180](#)
- F —
- Flows
 - BSDL, [20](#)
 - external, [19](#), [23](#)
 - internal, [19](#)
- G —
- Ground ports, [55](#)
- H —
- Highz instruction, [15](#)
- I —
- I/O Pads. See Pads
- Idcode instruction, [15](#)
- Input
 - I/O pad descriptions, [43](#), [44](#)
 - pin mapping file, [111](#)
 - Verilog netlist, [43](#)
- Instruction
 - bypass, [15](#)
 - clamp, [15](#)
 - extest, [14](#)
 - extest_pulse, [15](#), [180](#)
 - extest_train, [15](#), [180](#)
 - highz, [15](#)
 - idcode, [15](#)
 - intest, [15](#), [36](#)
 - preload, [14](#)
 - runbist, [15](#), [36](#)
 - sample, [14](#)
 - sample_preload, [14](#)
 - usercode, [15](#)
- Instruction register, [14](#)
- Instructions
 - user-defined, [117](#)
- Internal boundary scan synthesis, [43](#)
- Internal flow, [19](#)
 - control port notation, [51](#)
 - existing block BSR, [167](#)
 - inputs, [43](#)
 - output, [44](#)
 - power and ground ports, [55](#)
 - tdo pin sharing, [58](#)

top-level port connections, [56](#)
 uniquification, [59](#)
 unsupported commands, [60](#)
 Internal scan circuitry, [36](#), [118](#)
 Intest instruction, [15](#), [36](#)
 Invocation, [39](#), [44](#)

— J —

JTAG, [11](#)
 JTAG ports, [47](#)

— L —

Library switch, [44](#)
 Limitations, [36](#)
 Line continuation character, [20](#)

— N —

Nostandalone mode, [121](#)

— O —

Observe only cells, specifying, [182](#)

Output

ATPG setup file, [128](#)
 example, [26](#), [28](#)
 model, [24](#)
 types, [24](#), [125](#)

— P —

Pads

ac bidi differential, [110](#)
 ac bidirectional, [109](#)
 ac differential input, [108](#)
 ac input, [108](#)
 bidirectional, [102](#)
 differential input, [106](#)
 differential output, [106](#)
 input, [100](#)
 open collector output, [103](#)
 output, [101](#)
 setting up, [99](#) to [106](#)
 tristate, [102](#)

Parametric testing, [128](#)

Pin mapping, [55](#), [111](#)

Pinmap file

contents, [112](#)
 example, [26](#), [116](#)
 loading, [111](#)

syntax, [113](#)

Power ports, [55](#)

Preload instruction, [14](#)

— R —

Register

boundary scan, [14](#)
 bypass, [14](#)
 instruction, [14](#)
 test data, [14](#)

Report commands supporting 1149.6, [182](#)

Reserved words, [38](#)

Runbist instruction, [15](#), [36](#)

Running the tool

BSDL, [129](#)
 external, [40](#)

— S —

Sample instruction, [14](#)

Sample_preload instruction, [14](#)

Scan

architectures, [120](#)
 connecting, [118](#)

Scan cells, boundary, [13](#)

Scan chains, [124](#)

Specparams, [44](#)

Standalone mode, [121](#)

— T —

TAP, [14](#)

TAP controller, [14](#)

TDO pin sharing, [58](#)

Technology mapping

port mapping, [63](#)

Test access port, [14](#)

Test access port controller, [14](#)

Test bench, [24](#), [126](#)

Test data register, [14](#)

Test receiver verification, [140](#)

Test setup procedure, defining, [131](#)

Test vectors, [18](#)

Top-down design flows, [19](#)

Tri-state ports, important issues, [30](#)

— U —

Uniquification, [59](#)

Usercode instruction, [15](#)

User-defined instructions, [117](#)

— V —

Verifying boundary scan

gate-level, [138](#)

overview, [136](#)

Verilog escaped identifiers, [30](#)

Verilog support, [37](#)

VHDL entity, [30](#)

Third-Party Information

For information about third-party software included with this release of Tessent products, refer to the [*Third-Party Software for Tessent Products*](#).



End-User License Agreement

The latest version of the End-User License Agreement is available on-line at:
www.mentor.com/eula

IMPORTANT INFORMATION

USE OF ALL SOFTWARE IS SUBJECT TO LICENSE RESTRICTIONS. CAREFULLY READ THIS LICENSE AGREEMENT BEFORE USING THE PRODUCTS. USE OF SOFTWARE INDICATES CUSTOMER'S COMPLETE AND UNCONDITIONAL ACCEPTANCE OF THE TERMS AND CONDITIONS SET FORTH IN THIS AGREEMENT. ANY ADDITIONAL OR DIFFERENT PURCHASE ORDER TERMS AND CONDITIONS SHALL NOT APPLY.

END-USER LICENSE AGREEMENT ("Agreement")

This is a legal agreement concerning the use of Software (as defined in Section 2) and hardware (collectively "Products") between the company acquiring the Products ("Customer"), and the Mentor Graphics entity that issued the corresponding quotation or, if no quotation was issued, the applicable local Mentor Graphics entity ("Mentor Graphics"). Except for license agreements related to the subject matter of this license agreement which are physically signed by Customer and an authorized representative of Mentor Graphics, this Agreement and the applicable quotation contain the parties' entire understanding relating to the subject matter and supersede all prior or contemporaneous agreements. If Customer does not agree to these terms and conditions, promptly return or, in the case of Software received electronically, certify destruction of Software and all accompanying items within five days after receipt of Software and receive a full refund of any license fee paid.

1. ORDERS, FEES AND PAYMENT.

- 1.1. To the extent Customer (or if agreed by Mentor Graphics, Customer's appointed third party buying agent) places and Mentor Graphics accepts purchase orders pursuant to this Agreement (each an "Order"), each Order will constitute a contract between Customer and Mentor Graphics, which shall be governed solely and exclusively by the terms and conditions of this Agreement, any applicable addenda and the applicable quotation, whether or not those documents are referenced on the Order. Any additional or conflicting terms and conditions appearing on an Order or presented in any electronic portal or automated order management system, whether or not required to be electronically accepted, will not be effective unless agreed in writing and physically signed by an authorized representative of Customer and Mentor Graphics.
- 1.2. Amounts invoiced will be paid, in the currency specified on the applicable invoice, within 30 days from the date of such invoice. Any past due invoices will be subject to the imposition of interest charges in the amount of one and one-half percent per month or the applicable legal rate currently in effect, whichever is lower. Prices do not include freight, insurance, customs duties, taxes or other similar charges, which Mentor Graphics will state separately in the applicable invoice. Unless timely provided with a valid certificate of exemption or other evidence that items are not taxable, Mentor Graphics will invoice Customer for all applicable taxes including, but not limited to, VAT, GST, sales tax, consumption tax and service tax. Customer will make all payments free and clear of, and without reduction for, any withholding or other taxes; any such taxes imposed on payments by Customer hereunder will be Customer's sole responsibility. If Customer appoints a third party to place purchase orders and/or make payments on Customer's behalf, Customer shall be liable for payment under Orders placed by such third party in the event of default.
- 1.3. All Products are delivered FCA factory (Incoterms 2010), freight prepaid and invoiced to Customer, except Software delivered electronically, which shall be deemed delivered when made available to Customer for download. Mentor Graphics retains a security interest in all Products delivered under this Agreement, to secure payment of the purchase price of such Products, and Customer agrees to sign any documents that Mentor Graphics determines to be necessary or convenient for use in filing or perfecting such security interest. Mentor Graphics' delivery of Software by electronic means is subject to Customer's provision of both a primary and an alternate e-mail address.

2. **GRANT OF LICENSE.** The software installed, downloaded, or otherwise acquired by Customer under this Agreement, including any updates, modifications, revisions, copies, documentation and design data ("Software") are copyrighted, trade secret and confidential information of Mentor Graphics or its licensors, who maintain exclusive title to all Software and retain all rights not expressly granted by this Agreement. Mentor Graphics grants to Customer, subject to payment of applicable license fees, a nontransferable, nonexclusive license to use Software solely: (a) in machine-readable, object-code form (except as provided in Subsection 5.2); (b) for Customer's internal business purposes; (c) for the term of the license; and (d) on the computer hardware and at the site authorized by Mentor Graphics. A site is restricted to a one-half mile (800 meter) radius. Customer may have Software temporarily used by an employee for telecommuting purposes from locations other than a Customer office, such as the employee's residence, an airport or hotel, provided that such employee's primary place of employment is the site where the Software is authorized for use. Mentor Graphics' standard policies and programs, which vary depending on Software, license fees paid or services purchased, apply to the following: (a) relocation of Software; (b) use of Software, which may be limited, for example, to execution of a single session by a single user on the authorized hardware or for a restricted period of time (such limitations may be technically implemented through the use of authorization codes or similar devices); and (c) support services provided, including eligibility to receive telephone support, updates, modifications, and revisions. For the avoidance of doubt, if Customer provides any feedback or requests any change or enhancement to Products, whether in the course of receiving support or consulting services, evaluating Products, performing beta testing or otherwise, any inventions, product improvements, modifications or developments made by Mentor Graphics (at Mentor Graphics' sole discretion) will be the exclusive property of Mentor Graphics.
3. **ESC SOFTWARE.** If Customer purchases a license to use development or prototyping tools of Mentor Graphics' Embedded Software Channel ("ESC"), Mentor Graphics grants to Customer a nontransferable, nonexclusive license to reproduce and distribute executable

files created using ESC compilers, including the ESC run-time libraries distributed with ESC C and C++ compiler Software that are linked into a composite program as an integral part of Customer's compiled computer program, provided that Customer distributes these files only in conjunction with Customer's compiled computer program. Mentor Graphics does NOT grant Customer any right to duplicate, incorporate or embed copies of Mentor Graphics' real-time operating systems or other embedded software products into Customer's products or applications without first signing or otherwise agreeing to a separate agreement with Mentor Graphics for such purpose.

4. BETA CODE.

- 4.1. Portions or all of certain Software may contain code for experimental testing and evaluation (which may be either alpha or beta, collectively "Beta Code"), which may not be used without Mentor Graphics' explicit authorization. Upon Mentor Graphics' authorization, Mentor Graphics grants to Customer a temporary, nontransferable, nonexclusive license for experimental use to test and evaluate the Beta Code without charge for a limited period of time specified by Mentor Graphics. Mentor Graphics may choose, at its sole discretion, not to release Beta Code commercially in any form.
- 4.2. If Mentor Graphics authorizes Customer to use the Beta Code, Customer agrees to evaluate and test the Beta Code under normal conditions as directed by Mentor Graphics. Customer will contact Mentor Graphics periodically during Customer's use of the Beta Code to discuss any malfunctions or suggested improvements. Upon completion of Customer's evaluation and testing, Customer will send to Mentor Graphics a written evaluation of the Beta Code, including its strengths, weaknesses and recommended improvements.
- 4.3. Customer agrees to maintain Beta Code in confidence and shall restrict access to the Beta Code, including the methods and concepts utilized therein, solely to those employees and Customer location(s) authorized by Mentor Graphics to perform beta testing. Customer agrees that any written evaluations and all inventions, product improvements, modifications or developments that Mentor Graphics conceived or made during or subsequent to this Agreement, including those based partly or wholly on Customer's feedback, will be the exclusive property of Mentor Graphics. Mentor Graphics will have exclusive rights, title and interest in all such property. The provisions of this Subsection 4.3 shall survive termination of this Agreement.

5. RESTRICTIONS ON USE.

- 5.1. Customer may copy Software only as reasonably necessary to support the authorized use. Each copy must include all notices and legends embedded in Software and affixed to its medium and container as received from Mentor Graphics. All copies shall remain the property of Mentor Graphics or its licensors. Customer shall maintain a record of the number and primary location of all copies of Software, including copies merged with other software, and shall make those records available to Mentor Graphics upon request. Customer shall not make Products available in any form to any person other than Customer's employees and on-site contractors, excluding Mentor Graphics competitors, whose job performance requires access and who are under obligations of confidentiality. Customer shall take appropriate action to protect the confidentiality of Products and ensure that any person permitted access does not disclose or use Products except as permitted by this Agreement. Customer shall give Mentor Graphics written notice of any unauthorized disclosure or use of the Products as soon as Customer becomes aware of such unauthorized disclosure or use. Except as otherwise permitted for purposes of interoperability as specified by applicable and mandatory local law, Customer shall not reverse-assemble, reverse-compile, reverse-engineer or in any way derive any source code from Software. Log files, data files, rule files and script files generated by or for the Software (collectively "Files"), including without limitation files containing Standard Verification Rule Format ("SVRF") and Tcl Verification Format ("TVF") which are Mentor Graphics' trade secret and proprietary syntaxes for expressing process rules, constitute or include confidential information of Mentor Graphics. Customer may share Files with third parties, excluding Mentor Graphics competitors, provided that the confidentiality of such Files is protected by written agreement at least as well as Customer protects other information of a similar nature or importance, but in any case with at least reasonable care. Customer may use Files containing SVRF or TVF only with Mentor Graphics products. Under no circumstances shall Customer use Products or Files or allow their use for the purpose of developing, enhancing or marketing any product that is in any way competitive with Products, or disclose to any third party the results of, or information pertaining to, any benchmark.
- 5.2. If any Software or portions thereof are provided in source code form, Customer will use the source code only to correct software errors and enhance or modify the Software for the authorized use. Customer shall not disclose or permit disclosure of source code, in whole or in part, including any of its methods or concepts, to anyone except Customer's employees or on-site contractors, excluding Mentor Graphics competitors, with a need to know. Customer shall not copy or compile source code in any manner except to support this authorized use.
- 5.3. Customer may not assign this Agreement or the rights and duties under it, or relocate, sublicense, or otherwise transfer the Products, whether by operation of law or otherwise ("Attempted Transfer"), without Mentor Graphics' prior written consent and payment of Mentor Graphics' then-current applicable relocation and/or transfer fees. Any Attempted Transfer without Mentor Graphics' prior written consent shall be a material breach of this Agreement and may, at Mentor Graphics' option, result in the immediate termination of the Agreement and/or the licenses granted under this Agreement. The terms of this Agreement, including without limitation the licensing and assignment provisions, shall be binding upon Customer's permitted successors in interest and assigns.
- 5.4. The provisions of this Section 5 shall survive the termination of this Agreement.

6. **SUPPORT SERVICES.** To the extent Customer purchases support services, Mentor Graphics will provide Customer with updates and technical support for the Products, at the Customer site(s) for which support is purchased, in accordance with Mentor Graphics' then current End-User Support Terms located at <http://supportnet.mentor.com/supportterms>.

7. LIMITED WARRANTY.

- 7.1. Mentor Graphics warrants that during the warranty period its standard, generally supported Products, when properly installed, will substantially conform to the functional specifications set forth in the applicable user manual. Mentor Graphics does not warrant that Products will meet Customer's requirements or that operation of Products will be uninterrupted or error free. The

warranty period is 90 days starting on the 15th day after delivery or upon installation, whichever first occurs. Customer must notify Mentor Graphics in writing of any nonconformity within the warranty period. For the avoidance of doubt, this warranty applies only to the initial shipment of Software under an Order and does not renew or reset, for example, with the delivery of (a) Software updates or (b) authorization codes or alternate Software under a transaction involving Software re-mix. This warranty shall not be valid if Products have been subject to misuse, unauthorized modification, improper installation or Customer is not in compliance with this Agreement. MENTOR GRAPHICS' ENTIRE LIABILITY AND CUSTOMER'S EXCLUSIVE REMEDY SHALL BE, AT MENTOR GRAPHICS' OPTION, EITHER (A) REFUND OF THE PRICE PAID UPON RETURN OF THE PRODUCTS TO MENTOR GRAPHICS OR (B) MODIFICATION OR REPLACEMENT OF THE PRODUCTS THAT DO NOT MEET THIS LIMITED WARRANTY. MENTOR GRAPHICS MAKES NO WARRANTIES WITH RESPECT TO: (A) SERVICES; (B) PRODUCTS PROVIDED AT NO CHARGE; OR (C) BETA CODE; ALL OF WHICH ARE PROVIDED "AS IS."

7.2. THE WARRANTIES SET FORTH IN THIS SECTION 7 ARE EXCLUSIVE. NEITHER MENTOR GRAPHICS NOR ITS LICENSORS MAKE ANY OTHER WARRANTIES EXPRESS, IMPLIED OR STATUTORY, WITH RESPECT TO PRODUCTS PROVIDED UNDER THIS AGREEMENT. MENTOR GRAPHICS AND ITS LICENSORS SPECIFICALLY DISCLAIM ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NON-INFRINGEMENT OF INTELLECTUAL PROPERTY.

8. **LIMITATION OF LIABILITY.** EXCEPT WHERE THIS EXCLUSION OR RESTRICTION OF LIABILITY WOULD BE VOID OR INEFFECTIVE UNDER APPLICABLE LAW, IN NO EVENT SHALL MENTOR GRAPHICS OR ITS LICENSORS BE LIABLE FOR INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES (INCLUDING LOST PROFITS OR SAVINGS) WHETHER BASED ON CONTRACT, TORT OR ANY OTHER LEGAL THEORY, EVEN IF MENTOR GRAPHICS OR ITS LICENSORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. IN NO EVENT SHALL MENTOR GRAPHICS' OR ITS LICENSORS' LIABILITY UNDER THIS AGREEMENT EXCEED THE AMOUNT RECEIVED FROM CUSTOMER FOR THE HARDWARE, SOFTWARE LICENSE OR SERVICE GIVING RISE TO THE CLAIM. IN THE CASE WHERE NO AMOUNT WAS PAID, MENTOR GRAPHICS AND ITS LICENSORS SHALL HAVE NO LIABILITY FOR ANY DAMAGES WHATSOEVER. THE PROVISIONS OF THIS SECTION 8 SHALL SURVIVE THE TERMINATION OF THIS AGREEMENT.

9. **HAZARDOUS APPLICATIONS.** CUSTOMER ACKNOWLEDGES IT IS SOLELY RESPONSIBLE FOR TESTING ITS PRODUCTS USED IN APPLICATIONS WHERE THE FAILURE OR INACCURACY OF ITS PRODUCTS MIGHT RESULT IN DEATH OR PERSONAL INJURY ("HAZARDOUS APPLICATIONS"). EXCEPT TO THE EXTENT THIS EXCLUSION OR RESTRICTION OF LIABILITY WOULD BE VOID OR INEFFECTIVE UNDER APPLICABLE LAW, IN NO EVENT SHALL MENTOR GRAPHICS OR ITS LICENSORS BE LIABLE FOR ANY DAMAGES RESULTING FROM OR IN CONNECTION WITH THE USE OF MENTOR GRAPHICS PRODUCTS IN OR FOR HAZARDOUS APPLICATIONS. THE PROVISIONS OF THIS SECTION 9 SHALL SURVIVE THE TERMINATION OF THIS AGREEMENT.

10. **INDEMNIFICATION.** CUSTOMER AGREES TO INDEMNIFY AND HOLD HARMLESS MENTOR GRAPHICS AND ITS LICENSORS FROM ANY CLAIMS, LOSS, COST, DAMAGE, EXPENSE OR LIABILITY, INCLUDING ATTORNEYS' FEES, ARISING OUT OF OR IN CONNECTION WITH THE USE OF MENTOR GRAPHICS PRODUCTS IN OR FOR HAZARDOUS APPLICATIONS. THE PROVISIONS OF THIS SECTION 10 SHALL SURVIVE THE TERMINATION OF THIS AGREEMENT.

11. INFRINGEMENT.

11.1. Mentor Graphics will defend or settle, at its option and expense, any action brought against Customer in the United States, Canada, Japan, or member state of the European Union which alleges that any standard, generally supported Product acquired by Customer hereunder infringes a patent or copyright or misappropriates a trade secret in such jurisdiction. Mentor Graphics will pay costs and damages finally awarded against Customer that are attributable to such action. Customer understands and agrees that as conditions to Mentor Graphics' obligations under this section Customer must: (a) notify Mentor Graphics promptly in writing of the action; (b) provide Mentor Graphics all reasonable information and assistance to settle or defend the action; and (c) grant Mentor Graphics sole authority and control of the defense or settlement of the action.

11.2. If a claim is made under Subsection 11.1 Mentor Graphics may, at its option and expense: (a) replace or modify the Product so that it becomes noninfringing; (b) procure for Customer the right to continue using the Product; or (c) require the return of the Product and refund to Customer any purchase price or license fee paid, less a reasonable allowance for use.

11.3. Mentor Graphics has no liability to Customer if the action is based upon: (a) the combination of Software or hardware with any product not furnished by Mentor Graphics; (b) the modification of the Product other than by Mentor Graphics; (c) the use of other than a current unaltered release of Software; (d) the use of the Product as part of an infringing process; (e) a product that Customer makes, uses, or sells; (f) any Beta Code or Product provided at no charge; (g) any software provided by Mentor Graphics' licensors who do not provide such indemnification to Mentor Graphics' customers; or (h) infringement by Customer that is deemed willful. In the case of (h), Customer shall reimburse Mentor Graphics for its reasonable attorney fees and other costs related to the action.

11.4. THIS SECTION 11 IS SUBJECT TO SECTION 8 ABOVE AND STATES THE ENTIRE LIABILITY OF MENTOR GRAPHICS AND ITS LICENSORS, AND CUSTOMER'S SOLE AND EXCLUSIVE REMEDY, FOR DEFENSE, SETTLEMENT AND DAMAGES, WITH RESPECT TO ANY ALLEGED PATENT OR COPYRIGHT INFRINGEMENT OR TRADE SECRET MISAPPROPRIATION BY ANY PRODUCT PROVIDED UNDER THIS AGREEMENT.

12. TERMINATION AND EFFECT OF TERMINATION.

12.1. If a Software license was provided for limited term use, such license will automatically terminate at the end of the authorized term. Mentor Graphics may terminate this Agreement and/or any license granted under this Agreement immediately upon written notice if Customer: (a) exceeds the scope of the license or otherwise fails to comply with the licensing or confidentiality provisions of this Agreement, or (b) becomes insolvent, files a bankruptcy petition, institutes proceedings for liquidation or winding up or enters into an agreement to assign its assets for the benefit of creditors. For any other material breach of any

provision of this Agreement, Mentor Graphics may terminate this Agreement and/or any license granted under this Agreement upon 30 days written notice if Customer fails to cure the breach within the 30 day notice period. Termination of this Agreement or any license granted hereunder will not affect Customer's obligation to pay for Products shipped or licenses granted prior to the termination, which amounts shall be payable immediately upon the date of termination.

- 12.2. Upon termination of this Agreement, the rights and obligations of the parties shall cease except as expressly set forth in this Agreement. Upon termination, Customer shall ensure that all use of the affected Products ceases, and shall return hardware and either return to Mentor Graphics or destroy Software in Customer's possession, including all copies and documentation, and certify in writing to Mentor Graphics within ten business days of the termination date that Customer no longer possesses any of the affected Products or copies of Software in any form.
13. **EXPORT.** The Products provided hereunder are subject to regulation by local laws and United States ("U.S.") government agencies, which prohibit export, re-export or diversion of certain products, information about the products, and direct or indirect products thereof, to certain countries and certain persons. Customer agrees that it will not export or re-export Products in any manner without first obtaining all necessary approval from appropriate local and U.S. government agencies. If Customer wishes to disclose any information to Mentor Graphics that is subject to any U.S. or other applicable export restrictions, including without limitation the U.S. International Traffic in Arms Regulations (ITAR) or special controls under the Export Administration Regulations (EAR), Customer will notify Mentor Graphics personnel, in advance of each instance of disclosure, that such information is subject to such export restrictions.
14. **U.S. GOVERNMENT LICENSE RIGHTS.** Software was developed entirely at private expense. The parties agree that all Software is commercial computer software within the meaning of the applicable acquisition regulations. Accordingly, pursuant to U.S. FAR 48 CFR 12.212 and DFAR 48 CFR 227.7202, use, duplication and disclosure of the Software by or for the U.S. government or a U.S. government subcontractor is subject solely to the terms and conditions set forth in this Agreement, which shall supersede any conflicting terms or conditions in any government order document, except for provisions which are contrary to applicable mandatory federal laws.
15. **THIRD PARTY BENEFICIARY.** Mentor Graphics Corporation, Mentor Graphics (Ireland) Limited, Microsoft Corporation and other licensors may be third party beneficiaries of this Agreement with the right to enforce the obligations set forth herein.
16. **REVIEW OF LICENSE USAGE.** Customer will monitor the access to and use of Software. With prior written notice and during Customer's normal business hours, Mentor Graphics may engage an internationally recognized accounting firm to review Customer's software monitoring system and records deemed relevant by the internationally recognized accounting firm to confirm Customer's compliance with the terms of this Agreement or U.S. or other local export laws. Such review may include FlexNet (or successor product) report log files that Customer shall capture and provide at Mentor Graphics' request. Customer shall make records available in electronic format and shall fully cooperate with data gathering to support the license review. Mentor Graphics shall bear the expense of any such review unless a material non-compliance is revealed. Mentor Graphics shall treat as confidential information all information gained as a result of any request or review and shall only use or disclose such information as required by law or to enforce its rights under this Agreement. The provisions of this Section 16 shall survive the termination of this Agreement.
17. **CONTROLLING LAW, JURISDICTION AND DISPUTE RESOLUTION.** The owners of certain Mentor Graphics intellectual property licensed under this Agreement are located in Ireland and the U.S. To promote consistency around the world, disputes shall be resolved as follows: excluding conflict of laws rules, this Agreement shall be governed by and construed under the laws of the State of Oregon, U.S., if Customer is located in North or South America, and the laws of Ireland if Customer is located outside of North or South America. All disputes arising out of or in relation to this Agreement shall be submitted to the exclusive jurisdiction of the courts of Portland, Oregon when the laws of Oregon apply, or Dublin, Ireland when the laws of Ireland apply. Notwithstanding the foregoing, all disputes in Asia arising out of or in relation to this Agreement shall be resolved by arbitration in Singapore before a single arbitrator to be appointed by the chairman of the Singapore International Arbitration Centre ("SIAC") to be conducted in the English language, in accordance with the Arbitration Rules of the SIAC in effect at the time of the dispute, which rules are deemed to be incorporated by reference in this section. Nothing in this section shall restrict Mentor Graphics' right to bring an action (including for example a motion for injunctive relief) against Customer in the jurisdiction where Customer's place of business is located. The United Nations Convention on Contracts for the International Sale of Goods does not apply to this Agreement.
18. **SEVERABILITY.** If any provision of this Agreement is held by a court of competent jurisdiction to be void, invalid, unenforceable or illegal, such provision shall be severed from this Agreement and the remaining provisions will remain in full force and effect.
19. **MISCELLANEOUS.** This Agreement contains the parties' entire understanding relating to its subject matter and supersedes all prior or contemporaneous agreements. Some Software may contain code distributed under a third party license agreement that may provide additional rights to Customer. Please see the applicable Software documentation for details. This Agreement may only be modified in writing, signed by an authorized representative of each party. Waiver of terms or excuse of breach must be in writing and shall not constitute subsequent consent, waiver or excuse.